

Aula 04 - Prof. Diego Carvalho e Renato da Costa

*SEFAZ-AL (Auditor Fiscal da
Administração Tributária Estadual -
AFTE) Inteligência Artificial e Ciência de
Dados - 2026 (Pós-Edital)*
**Autor:
Diego Carvalho, Emannuelle
Gouveia Rolim, Equipe Informática
e TI, Renato da Costa**

29 de Agosto de 2026

Índice

1) SQL - Teoria	3
2) SQL - Questões Comentadas - CEBRASPE	75
3) SQL - Lista de Questões - CEBRASPE	94



APRESENTAÇÃO

Fala, galera!

A matéria de SQL inicia-se de forma simples e intuitiva, mas possui uma curva de aprendizado que atinge níveis elevados de complexidade teórica e prática. Os comandos básicos das sublinguagens exigem certa memorização (decoreba), mas são curtos e de fácil assimilação. No entanto, o estudante deve estar preparado para tópicos práticos e matemáticos complexos, como Álgebra Relacional e subconsultas com EXISTS, considerado o ponto mais difícil de compreender, sendo muito recomendado o uso de videoaulas.

Para o estudante em pré-edital, com tempo sobrando, o maior custo-benefício está em esgotar todo o conteúdo do edital. Ele deve sim focar nos temas de incidência altíssima como o comando SELECT e suas cláusulas (FROM, WHERE, GROUP BY, HAVING, ORDER BY). Contudo, obrigatoriamente deve incluir os assuntos de baixa ou baixíssima ocorrência, como views, triggers, stored procedures, CTEs e funções de janela. Nada deve ser descartado, pois qualquer item pode ser o diferencial.

Para o aluno em pós-edital, que possui tempo escasso, a estratégia muda drasticamente para focar apenas nas faixas de incidência média, alta e altíssima. Deve ser dada prioridade total e obrigatória ao comando SELECT, operadores (AND, OR, NOT) e JOIN. Além disso, os comandos DML como INSERT, UPDATE e DELETE precisam ser consolidados. Em contrapartida, devem ser descartados tópicos de incidência baixíssima, como Stored Procedures, ordem de precedência, CTEs e funções analíticas.

Prof. Diego Carvalho

 **PROFESSOR DIEGO CARVALHO - [WWW.INSTAGRAM.COM/PROFESSORDIEGOCARVALHO](https://www.instagram.com/professordiegocarvalho)**

SQL: UMA RELAÇÃO COMPLICADA



SQL

Conceitos Básicos

INCIDÊNCIA EM PROVA: BAIXA

SQL (Structured Query Language) é a linguagem padrão para manipular bancos de dados relacionais. Permite consultar, inserir, alterar e excluir dados em tabelas, seguindo uma abordagem declarativa: o usuário informa o que deseja e o sistema decide como executar. Linguagens de programação são meios de comunicação entre humanos e computadores. Podem seguir diferentes paradigmas, sendo os principais:

- Procedural: descreve passo a passo como alcançar um resultado.
- Declarativo: define o que se quer obter, sem detalhar o processo.

SQL segue o paradigma declarativo. O foco é no resultado, não no processo. Apesar de existirem extensões com elementos procedurais (como PL/SQL e Transact-SQL), o foco será na versão padrão definida pelo ANSI, com eventuais menções a esses dialetos.

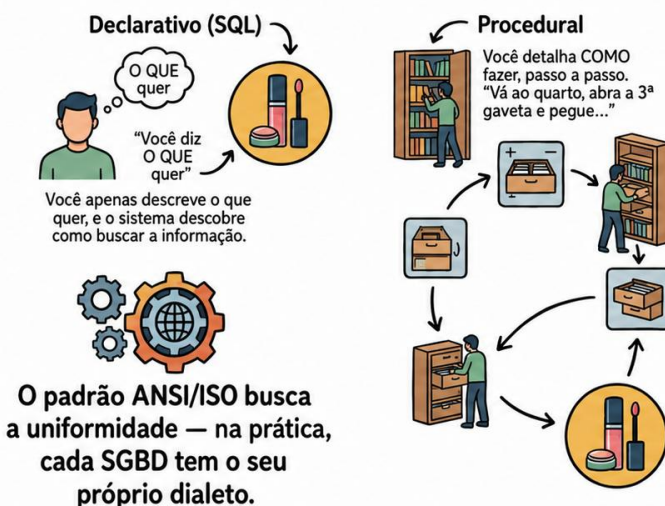
SQL: A Linguagem Para Conversar com Bancos de Dados

O que é SQL?
A Analogia do Garçom

Como o SQL Funciona?
O Foco no Resultado



SQL é uma linguagem declarativa: o foco é no resultado, não no processo.



Tipos de Dados

INCIDÊNCIA EM PROVA: BAIXA

Antes de tratar as sub-linguagens do SQL, é essencial entender os tipos de dados. Em uma tabela, cada coluna deve ter um tipo de dado definido, o que determina os valores permitidos e assegura a integridade das informações – como impedir que datas sejam inseridas em colunas numéricas. SQL oferece tipos de dados como texto, números, datas e valores lógicos. Apesar de variações entre SGBDs (como MySQL, PostgreSQL e SQL Server), as categorias principais são semelhantes.



Tipos Textuais

Também conhecido como literal, esse tipo de dado é basicamente uma sequência ou cadeia de caracteres (também chamada de string). Vejamos:

TIPO TEXTUAL	DESCRIÇÃO	EXEMPLO
CHAR(N)	Tipo usado para armazenar cadeias de caracteres de tamanho fixo. Mesmo que o conteúdo tenha menos caracteres que o definido, o banco completa o restante com espaços em branco. É útil para códigos, siglas e dados com comprimento sempre constante.	CODIGO CHAR(5)
VARCHAR(N)	Armazena cadeias de caracteres de tamanho variável, ocupando apenas o espaço necessário para a string inserida, até o limite definido. É ideal para nomes, endereços e qualquer informação textual cujo comprimento não seja uniforme e possa variar bastante.	NOME VARCHAR(255)
TEXT	Usado para guardar grandes blocos de texto, sem limite prático ou com limite muito extenso. É adequado para descrições longas, artigos, comentários, anotações e demais conteúdos textuais amplos que ultrapassem facilmente o tamanho permitido pelos tipos tradicionais.	COMENTARIOS TEXT
CLOB (Character Large Object)	Tipo destinado ao armazenamento de grandes quantidades de texto, geralmente muito maiores que campos comuns. É apropriado para documentos extensos, livros, relatórios completos ou qualquer conteúdo textualmente volumoso que exija capacidade de armazenamento elevada.	LIVRO CLOB

Quando trabalhamos com textos no SQL, tem um detalhe importante: os valores textuais, chamados de strings, devem sempre estar entre aspas simples (' '). Por exemplo, se você quiser buscar um cliente chamado João, o comando seria assim:

SINTAXE DO COMANDO

```
SELECT * FROM CLIENTES WHERE NOME = 'JOÃO';
```

Agora, aspas duplas (" ") servem para outro propósito. Elas são usadas – dependendo do SGBD – para delimitar identificadores, como nomes de colunas ou tabelas que contenham espaços, caracteres especiais ou até palavras reservadas. Exemplo:

SINTAXE DO COMANDO

```
SELECT "Primeiro Nome", "Último Nome" FROM CLIENTES;
```

A distinção entre letras maiúsculas e minúsculas em SQL depende do contexto. Para valores textuais, como nomes e e-mails, essa diferenciação está vinculada à configuração da collation do banco de dados. Se a collation for sensível a maiúsculas e minúsculas (case-sensitive), sistemas como MySQL, PostgreSQL e SQL Server tratarão, por exemplo, 'joão' e 'João' como valores distintos. Por outro lado, as palavras-chave, como SELECT, FROM e WHERE, não fazem distinção entre maiúsculas e minúsculas. Essas instruções podem ser escritas em qualquer combinação de letras sem comprometer sua execução.

SINTAXE DO COMANDO

```
SELECT nome FROM clientes; -- Correto  
select nome from clientes; -- Correto  
SeLeCt nome FrOm clientes; -- Correto
```



Tipos Numérico

No SQL, os tipos numéricos são usados para armazenar números nas tabelas e se dividem em três categorias principais. Os tipos inteiros guardam números sem casas decimais, como 10, -3 ou 2025, sendo ideais para contagens, quantidades e identificadores. Já os tipos de ponto flutuante permitem valores com casas decimais, como 3.14 ou -0.75, sendo úteis em medições, notas e porcentagens. Por fim, os tipos com precisão exata também lidam com decimais, mas com controle rigoroso sobre o número de dígitos antes e depois da vírgula, o que os torna essenciais em cálculos financeiros. A escolha do tipo adequado depende da precisão necessária e do contexto dos dados.

TIPO NUMÉRICO	DESCRIÇÃO	EXEMPLO
INT / INTEGER	Armazena números inteiros dentro de um intervalo definido pelo SGBD, ocupando normalmente 4 bytes. É amplamente usado para contagens, identificadores e registros que não exigem casas decimais, oferecendo bom equilíbrio entre desempenho e capacidade.	QUANTIDADE INT
SMALLINT	Tipo numérico para inteiros menores que os suportados por INT, ocupando geralmente 2 bytes. É indicado quando os valores são reduzidos e previsíveis, permitindo economia de armazenamento e maior eficiência em tabelas muito grandes ou fortemente acessadas.	IDADE SMALLINT
BIGINT	Armazena inteiros extensos que ultrapassam o limite do INT, usando normalmente 8 bytes. É ideal para grandes contagens, chaves de sistemas amplos ou cenários que acumulam valores continuamente, garantindo suporte a números muito elevados.	VALOR BIGINT
FLOAT	Representa números em ponto flutuante com precisão variável, ocupando cerca de 4 bytes. É útil para valores aproximados, medições e cálculos científicos em que pequenas imprecisões são toleráveis, sendo adequado para cenários de baixa exigência de precisão.	DISTANCIA FLOAT
DOUBLE	Tipo de ponto flutuante com maior precisão e faixa numérica que FLOAT, usando cerca de 8 bytes. É recomendado para cálculos mais detalhados, aplicações científicas e situações em que a precisão é importante, mas ainda se admite mínima variação nos resultados.	PRECO DOUBLE
REAL	Tipo de ponto flutuante semelhante ao FLOAT, variando conforme o SGBD. Geralmente ocupa 4 bytes e serve para representar valores aproximados em cálculos simples, medições e aplicações que não exigem alta precisão, mantendo boa eficiência de armazenamento.	DISTANCIA REAL
DECIMAL(P,S) / NUMERIC(P,S)	Tipos equivalentes que armazenam números com precisão exata, evitando erros de ponto flutuante. P define o total de dígitos e S define a parte decimal. São ideais para valores monetários, cálculos contábeis e registros que exigem total confiabilidade numérica.	SALDO DECIMAL(10,2)

Tipos Binários

O tipo BLOB (Binary Large Object) é utilizado em SQL para armazenar grandes volumes de dados binários, como imagens, vídeos, áudios ou arquivos. Diferente dos tipos VARCHAR ou TEXT, que são voltados para texto, o BLOB armazena sequências de bits representando dados não textuais. Existem variações do BLOB com limites de tamanho distintos, mas sua forma padrão geralmente atende às necessidades da maioria das aplicações.



TIPOS BINÁRIOS	DESCRIÇÃO	EXEMPLO
BLOB (BINARY LARGE OBJECT)	Armazena grandes quantidades de dados binários, como arquivos, imagens, vídeos, áudios ou outros tipos de dados não estruturados. Ele é comumente utilizado em bancos de dados que precisam gerenciar dados multimídia ou outros conteúdos que não são puramente textuais.	ARQUIVO BLOB

Tipos Booleanos

Os tipos booleanos são utilizados para armazenar valores do tipo verdadeiro ou falso. Embora o SQL padrão não tenha um tipo BOOLEAN formalmente definido, a maioria dos SGBDs oferece formas de trabalhar com esses valores – geralmente usando TRUE e FALSE ou 1 e 0. Vale lembrar como funcionam as expressões lógicas, que são bastante usadas em condições:

- **Operação AND:** o resultado é verdadeiro (TRUE) somente se ambas as expressões forem verdadeiras; caso contrário, o resultado é falso (FALSE). Por exemplo, a expressão "TRUE AND FALSE" resultaria em FALSE, porque apenas uma das condições é verdadeira.
- **Operação OR:** o resultado é verdadeiro (TRUE) se pelo menos uma das expressões for verdadeira; caso contrário, o resultado é falso (FALSE). Por exemplo, a expressão "TRUE OR FALSE" resultaria em TRUE, porque uma das condições é verdadeira.
- **Operação NOT:** esse operador inverte o valor de uma expressão lógica, isto é, se a expressão original é TRUE, a negação será FALSE, e vice-versa. Por exemplo, se tivermos a expressão lógica "NOT A", onde A é TRUE, o resultado será FALSE.

Antes de prosseguir, é essencial compreender o conceito de valor NULL. Em SQL, NULL representa a ausência, indefinição ou desconhecimento de um valor. Quando um campo está NULL, significa que nenhum valor foi atribuído àquela posição, o que difere de valores como zero ou uma cadeia de caracteres vazia. Portanto, NULL indica explicitamente que o dado é inexistente ou não foi informado.

INTERPRETAÇÃO	DESCRIÇÃO
VALOR DESCONHECIDO	Trata-se de um valor que existe, mas que não é conhecido. Por exemplo: a data de nascimento de uma pessoa não é conhecida, e por isso é representada por NULL.
VALOR AUSENTE	Trata-se de um valor que existe, mas foi propositalmente retido. Por exemplo: uma pessoa tem um telefone residencial, mas não deseja que ele seja listado, por isso ele é retido e representado como NULL.
VALOR INDEFINIDO	Trata-se de um atributo indefinido para uma determinada tupla. Por exemplo: um atributo CONJUGE seria NULL para uma pessoa que não fosse casada, pois ele não se aplica a essa pessoa.

No SQL, utiliza-se a Lógica de Três Valores (L3V), que inclui os valores TRUE, FALSE e UNKNOWN. Esse terceiro valor, UNKNOWN, aparece principalmente quando há a presença de NULL, que indica ausência de informação e não um valor real. Sempre que uma expressão lógica envolve NULL e não é possível afirmar se é verdadeira ou falsa, o resultado será UNKNOWN.

Isso acontece porque NULL representa incerteza. Por exemplo, se um campo de "telefone residencial" estiver com valor NULL, o sistema não pode concluir se a pessoa não tem telefone, se o dado não foi informado ou se foi perdido. Nesse caso, qualquer comparação com NULL não pode ser avaliada de forma binária, o que afeta diretamente a forma como o SQL trata condições em cláusulas e filtros.



Assim, a L3V é essencial para entender como o SQL processa expressões lógicas, adotando TRUE, FALSE ou UNKNOWN como possíveis resultados.

OPERAÇÕES	RESULTADO
TRUE AND NULL	O operador AND retorna TRUE apenas se ambas as condições forem verdadeiras. Como NULL é desconhecido, o resultado da expressão também será UNKNOWN.
NULL AND TRUE	O operador AND funciona de maneira simétrica, então o resultado ainda será UNKNOWN.
TRUE OR NULL	O operador OR retorna TRUE se pelo menos uma condição for verdadeira. Aqui, como TRUE já está presente, o resultado será TRUE.
NULL OR TRUE	Assim como no caso anterior, TRUE garante que a expressão seja avaliada como TRUE.
FALSE AND NULL	O operador AND retorna FALSE se qualquer uma das condições for falsa. Mesmo que NULL seja desconhecido, FALSE já invalida o resultado.
NULL AND FALSE	Da mesma forma, FALSE sempre domina no caso do operador AND.
FALSE OR NULL	O operador OR precisa de pelo menos uma condição verdadeira para retornar TRUE. Como FALSE é falso e NULL é desconhecido, o resultado será UNKNOWN.
NULL OR FALSE	A lógica aqui é simétrica: como o FALSE não contribui para tornar a expressão verdadeira, o resultado permanece indefinido (UNKNOWN).
! NULL	A negação de NULL não é possível, pois NULL representa um valor desconhecido. Logo, o resultado permanece UNKNOWN.

Por fim, o tipo booleano utiliza apenas 1 byte para seu armazenamento, uma vez que o valor falso é representado por 0 (00000000) e o valor verdadeiro é representado por 1 (00000001).

Tipos de Data/Hora

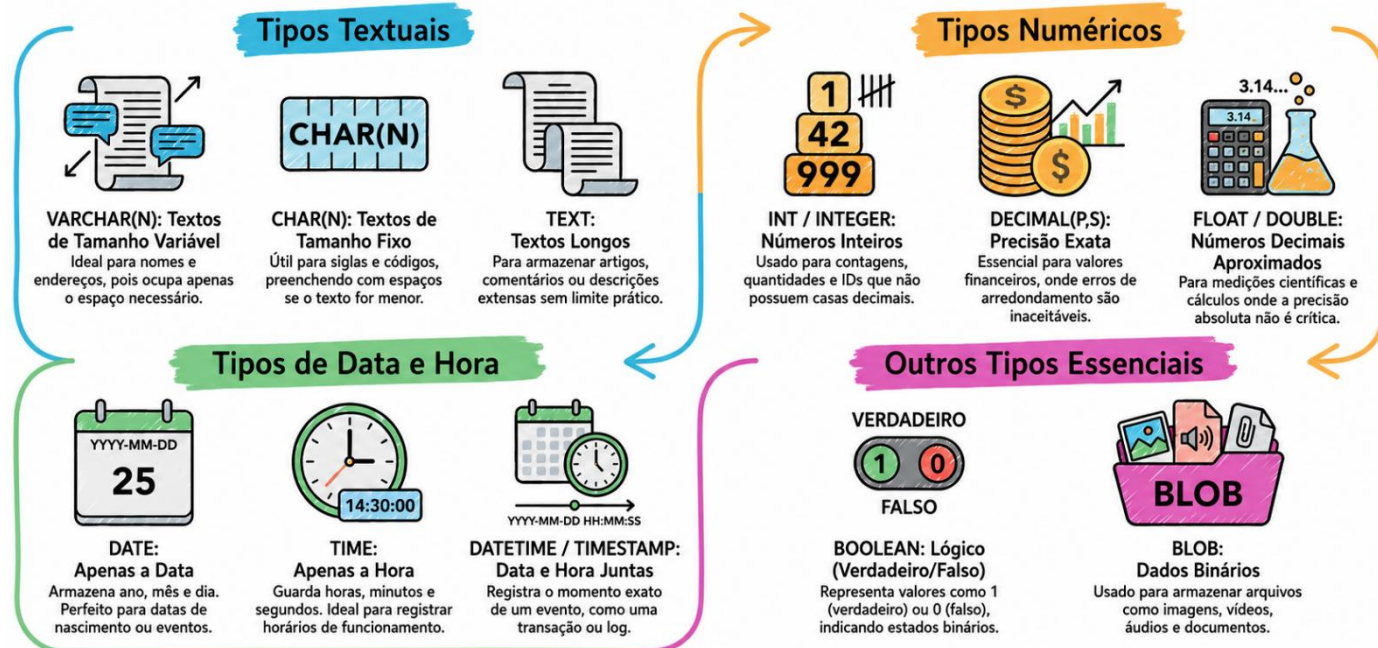
Os tipos de dados de data e hora são utilizados para armazenar informações relacionadas ao tempo, como datas de nascimento, horários de eventos ou registros de criação no sistema. Esses tipos são fundamentais sempre que o tempo precisa ser representado em um banco de dados. Apesar de haver variações entre os SGBDs, como MySQL, PostgreSQL e SQL Server, a estrutura básica é semelhante. Em geral, há tipos específicos para armazenar apenas datas (como DATE), apenas horários (como TIME) e combinações de data e hora (como DATETIME ou TIMESTAMP).

TIPO	DESCRIÇÃO	EXEMPLO
DATE	Armazena apenas a parte da data, incluindo ano, mês e dia, sem qualquer informação de horário. É utilizado para registrar eventos, aniversários, datas de criação ou prazos, garantindo precisão temporal quando o componente de hora não é necessário.	DT_EVENTO DATE
TIME	Armazena exclusivamente a hora, minutos e segundos, sem relação com uma data específica. É ideal para horários de funcionamento, agendamentos, marcação de turnos ou situações em que apenas o tempo do dia é relevante, sem depender de contexto cronológico.	HORA_EVENTO TIME
DATETIME / TIMESTAMP	Armazena simultaneamente data e hora no mesmo campo, permitindo registrar momentos completos no tempo. É amplamente usado para auditoria, logs, transações, cadastros e qualquer evento em que seja essencial saber exatamente quando ocorreu.	DT_TRANSACAO DATETIME
YEAR	Armazena apenas o ano, geralmente usando quatro dígitos. É apropriado para datas simplificadas, como ano de lançamento, ano fiscal, fabricação ou períodos que não exigem meses ou dias. Permite economia de espaço e maior clareza sem sobrecarga temporal.	ANO_LNCMNTO YEAR



Tipos de Dados em SQL: Um Guia Rápido

Em um banco de dados SQL, cada coluna de uma tabela precisa ter um tipo de dado definido. Essa definição é crucial para determinar que tipo de informação pode ser armazenada, garantindo a consistência e a integridade dos dados, como textos, números e datas.



Sublinguagens SQL

SQL é a linguagem usada para executar operações em bancos de dados relacionais por meio de comandos, que são instruções escritas em inglês estruturado, geralmente finalizadas com ponto e vírgula. Ao receber um comando, o SGBD interpreta e executa a ação solicitada. Esses comandos estão organizados em quatro sublinguagens principais: DDL, DML, DCL e TCL, cada uma com uma função específica no gerenciamento do banco de dados.

Entender como cada uma funciona é essencial para o uso eficiente do SQL. Embora seja necessário memorizar os comandos, a maioria é curta, intuitiva e fácil de assimilar com a prática. Atenção: embora a especificação clássica classifique o SELECT como um comando DML (por manipular dados), bancas como o CEBRASPE frequentemente cobram uma divisão mais estrita, classificando-o como DQL (Data Query Language). Fique atento ao enunciado da sua prova!"

DDL (DATA DEFINITION LANGUAGE)

DDL (DATA DEFINITION LANGUAGE)

A DDL (Data Definition Language) é a parte da linguagem SQL responsável pela definição e estruturação dos objetos de um banco de dados, como tabelas, índices, visões e esquemas. Por meio de comandos como CREATE, ALTER e DROP, a DDL permite criar novas estruturas, modificar propriedades existentes ou removê-las por completo, influenciando diretamente o modo como os dados serão armazenados e organizados.

Antes de realizar inserções, consultas ou atualizações em SQL, é necessário definir a estrutura que armazenará os dados. Essa tarefa é feita pela DDL (Data Definition Language), ou Linguagem de Definição de Dados, que reúne os comandos usados para criar, alterar e excluir elementos do banco, como tabelas,



colunas, tipos de dados, restrições, relacionamentos, índices e visões. A DDL define a arquitetura lógica do sistema, organizando como os dados serão armazenados. Ela estabelece o esquema do banco de dados, sendo uma etapa essencial para garantir a integridade e a coerência do sistema.

COMANDOS DDL	DESCRIÇÃO
CREATE TABLE	Comando utilizado para criar tabelas (e outros objetos) de um banco de dados.
DROP TABLE	Comando utilizado para deletar uma tabela (e outros objetos) de um banco de dados.
TRUNCATE TABLE	Comando utilizado para apagar os dados de uma tabela de um banco de dados.
ALTER TABLE	Comando utilizado para manipular colunas ou restrições de um banco de dados.
RENAME TABLE	Comando utilizado para renomear uma tabela de um banco de dados.

IMPORTANTE

Antes de avançar, é importante considerar que o conteúdo desta aula é extenso e interligado. Por isso, alguns conceitos serão apresentados de forma preliminar, com explicações mais detalhadas em momentos apropriados, a fim de evitar sobrecarga e facilitar a compreensão. Neste primeiro momento, o foco será nos comandos voltados à criação e estruturação de tabelas.

Outros objetos do banco serão abordados mais adiante. Também é relevante observar que a sintaxe pode variar entre os diferentes SGBDs. Embora existam muitas ferramentas, a ênfase será nos principais sistemas: Oracle, SQL Server e MySQL. Diferenças relevantes entre eles serão destacadas quando necessário.

Comandos DDL

CREATE TABLE

INCIDÊNCIA EM PROVA: ALTA

O comando utilizado para criar uma nova tabela em um banco de dados é o CREATE TABLE. Por meio dele, define-se toda a estrutura da tabela, incluindo as colunas, os tipos de dados que cada coluna armazenará e, opcionalmente, as restrições necessárias, como chave primária, chave estrangeira e unicidade de valores. Esse comando estabelece a organização dos dados e impõe regras que asseguram a integridade e a consistência das informações. A seguir, apresenta-se sua sintaxe e funcionamento.

SINTAXE DO COMANDO

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1          TIPO_DE_DADO          RESTRIÇÕES  
    NOME_COLUNA2          TIPO_DE_DADO          RESTRIÇÕES  
    NOME_COLUNA3          TIPO_DE_DADO          RESTRIÇÕES  
    ...  
);
```

EXEMPLO DO COMANDO

```
CREATE TABLE ALUNO (  
    NOME          VARCHAR(20)          NOT NULL  
    CPF           INT                   PRIMARY KEY
```



```
SEXO          CHAR(1)          NOT NULL ,
DATA_NASCIMENTO DATE          NOT NULL ,
CIDADE        VARCHAR(50)      NOT NULL ,
VALOR_PAGO    INT              NOT NULL
);
```

RESULTADO DO COMANDO

ALUNO					
NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO

Vamos entender passo-a-passo e linha por linha o comando apresentado:

CREATE TABLE ALUNO (

Esse é o início do comando, onde estamos criando uma tabela com o nome ALUNO. O nome da tabela será utilizado para referenciá-la nas operações subsequentes, como inserção ou consulta de dados.

```
NOME          VARCHAR(20)      NOT NULL ,
```

Aqui, estamos criando uma coluna chamada NOME que vai armazenar dados do tipo texto (string) com, no máximo, 20 caracteres. O tipo de dado VARCHAR(20) é usado quando o tamanho do texto pode variar. A restrição NOT NULL significa que não pode haver registros onde o nome do aluno esteja em branco (ou seja, o campo é obrigatório).

```
CPF           INT              PRIMARY KEY ,
```

A coluna CPF armazena números inteiros (tipo de dado INT). A restrição PRIMARY KEY define que o CPF será a chave primária da tabela, ou seja, não pode haver dois alunos com o mesmo CPF. A chave primária serve para identificar de maneira única cada registro na tabela.

```
SEXO          CHAR(1)          NOT NULL ,
```

A coluna SEXO armazena um valor de exatamente 1 caractere (tipo de dado CHAR(1)), e ela é usada para armazenar informações sobre o sexo do aluno (Ex: "M" para masculino e "F" para feminino). A restrição NOT NULL garante que esse campo não pode ser deixado em branco.

```
DATA_NASCIMENTO DATE          NOT NULL ,
```

A coluna DATA_NASCIMENTO armazena uma data (tipo de dado DATE). A restrição NOT NULL significa que cada aluno deve ter uma data de nascimento registrada.

```
CIDADE        VARCHAR(50)      NOT NULL ,
```

A coluna CIDADE armazena o nome da cidade onde o aluno reside, utilizando o tipo de dado VARCHAR(50), ou seja, uma string de até 50 caracteres. A restrição NOT NULL impede que esse campo seja deixado em branco.



VALOR_PAGO INT NOT NULL

A coluna VALOR_PAGO armazena um valor numérico inteiro (INT) que provavelmente se refere a um pagamento ou mensalidade relacionada ao aluno. A restrição NOT NULL assegura que todo aluno tenha um valor pago registrado.

ALUNO_ESCOLA_ANTIGA					
NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
Alice	111.111.111-11	'F'	01/01/2010	Alexânia	1000,00
Bernardo	222.222.222-22	'M'	02/02/2010	Brasília	2000,00
Caio	333.333.333-33	'M'	03/03/2010	Campinas	3000,00
Denise	444.444.444-44	'F'	04/04/2010	Diadema	4000,00
Elis	555.555.555-55	'F'	05/05/2010	Extrema	5000,00

Agora imagine que você tenha uma tabela chamada ALUNO_ESCOLA_ANTIGA no banco de dados, e ela já está populada¹ com vários dados sobre alunos (conforme tabela anterior). Agora, você quer criar uma nova tabela que tenha a mesma estrutura de dados dessa tabela de referência, mas que pode conter informações filtradas ou um subconjunto dos dados. Para tal, também podemos utilizar a seguinte sintaxe:

SINTAXE DO COMANDO

```
CREATE TABLE NOME_TABELA_NOVA AS
SELECT NOME_COLUNA1, NOME_COLUNA2, NOME_COLUNA3, ...
FROM NOME_TABELA_ANTIGA
WHERE ...
```

Não se preocupem em entender o restante do comando (estudaremos em detalhes mais à frente). Guardem apenas que é possível criar uma tabela a partir de outra pré-existente. Na prática, vamos supor que você queira criar uma nova tabela chamada ALUNO_ESCOLA_NOVA a partir da tabela ALUNO_ESCOLA_ANTIGA, e deseja copiar todas as informações de alunos, mas sem modificar nada. O comando seria o seguinte:

EXEMPLO DO COMANDO

```
CREATE TABLE ALUNO_ESCOLA_NOVA AS
SELECT NOME, CPF, SEXO, DATA_NASCIMENTO, CIDADE, VALOR_PAGO
FROM ALUNO_ESCOLA_ANTIGA
```

Quando você usa esse comando, o SQL cria a tabela nova com a mesma estrutura (colunas) e copia os dados da tabela de referência para a nova tabela, sem alterar a tabela de referência. Ou seja, o banco de dados terá duas tabelas: a ALUNO_ESCOLA_ANTIGA e a ALUNO_ESCOLA_NOVA, ambas com os mesmos dados. Se precisar, você pode adicionar um filtro WHERE para copiar apenas uma parte dos dados, mas não vamos ver isso agora.

RESULTADO DO COMANDO

ALUNO_ESCOLA_NOVA

¹ Esse é o termo técnico que indica que foram inseridos dados na tabela (tabela populada = tabela não vazia).



NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
Alice	111.111.111-11	'F'	01/01/2010	Alexânia	1000,00
Bernardo	222.222.222-22	'M'	02/02/2010	Brasília	2000,00
Caio	333.333.333-33	'M'	03/03/2010	Campinas	3000,00
Denise	444.444.444-44	'F'	04/04/2010	Diadema	4000,00
Elis	555.555.555-55	'F'	05/05/2010	Extrema	5000,00

Para finalizar, vamos falar sobre as regras de nomeação de tabelas – um detalhe que, embora pareça simples, faz toda a diferença. Seguir boas práticas ao dar nomes às suas tabelas ajuda a manter o banco de dados mais organizado, claro e compatível com o SGBD que você estiver usando. Apesar de cada sistema poder ter suas particularidades, existem algumas regras gerais que valem para a maioria dos casos. Vamos dar uma olhada nessas diretrizes para evitar erros e garantir nomes consistentes:

REGRAS DE NOMES DE TABELAS	DESCRIÇÃO
COMEÇAR COM UMA LETRA	A tabela deve começar com uma letra (A-Z ou a-z), não podendo começar com números ou caracteres especiais.
USAR CARACTERES ALFANUMÉRICOS	O nome da tabela pode conter letras (A-Z, a-z), números (0-9) e underlines (_), mas deve evitar caracteres especiais.
EVITAR PALAVRAS RESERVADAS	Não usar palavras reservadas no SQL, como SELECT, INSERT, TABLE, etc., pois podem gerar erros de sintaxe.
SER DESCRITIVO	O nome da tabela deve ser claro e refletir o tipo de dados que ela armazena.
USAR CONVENÇÕES DE NOMENCLATURA	Usar <i>snake_case</i> (com underscores) para tornar o nome da tabela legível. Pode ser usado camelCase, mas <i>snake_case</i> é mais comum.
EVITAR ABREVIACÕES EXCESSIVAS	Evitar abreviações excessivas que dificultem a compreensão do nome da tabela. Prefira nomes completos e descritivos.
LIMITAR COMPRIMENTO DO NOME	Evitar nomes de tabelas muito longos. A maioria dos SGBDs tem um limite de comprimento para os nomes (exemplo: 64 caracteres).
EVITAR O USO DE MAIÚSCULAS	Embora os SGBDs modernos aceitem maiúsculas, é uma boa prática usar minúsculas para evitar problemas de sensibilidade a maiúsculas/minúsculas.
USAR O PLURAL	Usar o plural para indicar que a tabela contém múltiplos registros. Exemplo: clientes, produtos.

DROP TABLE

INCIDÊNCIA EM PROVA: BAIXA

Esse comando é usado para excluir permanentemente uma tabela do banco de dados, removendo sua estrutura e todos os dados associados. Essa ação é irreversível, exceto se houver backup prévio, e por isso deve ser utilizada com cautela. É indicada quando a tabela se torna obsoleta ou deixa de ser necessária no projeto. Não é preciso apagar os dados manualmente, pois o comando faz a exclusão completa. Em muitos SGBDs, sua execução é restrita a usuários com permissões específicas, como forma de evitar exclusões indevidas. A seguir, serão apresentados sua sintaxe e funcionamento.

SINTAXE DO COMANDO

DROP TABLE NOME_DA_TABELA;

EXEMPLO DO COMANDO

DROP TABLE ALUNO_ESCOLA_NOVA;



RESULTADO DO COMANDO

ALUNO_ESCOLA_NOVA					
NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
Alice	111.111.111-11	F	10/10/2010	Alexânia	1000,00
Bernardo	222.222.222-22	M	10/10/2010	Brasília	2000,00
Caio	333.333.333-33	M	10/10/2010	Campinas	3000,00
Denise	444.444.444-44	F	10/10/2010	Diadema	4000,00
Elis	555.555.555-55	F	10/10/2010	Extrema	5000,00



Embora o comando DROP TABLE seja usado especificamente para excluir uma tabela de forma definitiva, o comando DROP, de maneira geral, possui aplicação mais ampla. Ele pode ser utilizado para remover outros objetos do banco de dados, como índices, visões, procedimentos, funções, esquemas, domínios, restrições e até o próprio banco (como no caso do DROP SCHEMA). Esses usos adicionais serão explorados mais adiante, conforme cada objeto for abordado.

TRUNCATE TABLE

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Esse comando é usado para remover todos os registros de uma tabela sem alterar sua estrutura. As colunas, tipos de dados e restrições permanecem intactos, permitindo o uso imediato da tabela após a limpeza. Como essa operação não registra a exclusão de cada linha no log do sistema, ela é muito eficiente em grandes volumes de dados. Diferente do DROP TABLE, que elimina a estrutura e os dados, o TRUNCATE é indicado quando se deseja apenas esvaziar a tabela, mantendo sua definição no banco.

SINTAXE DO COMANDO

TRUNCATE TABLE NOME_DA_TABELA;

EXEMPLO DO COMANDO

TRUNCATE TABLE ALUNO_ESCOLA_NOVA;

RESULTADO DO COMANDO

ALUNO_ESCOLA_NOVA					
NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO

ALTER TABLE

INCIDÊNCIA EM PROVA: BAIXA

O comando ALTER TABLE permite modificar a estrutura de uma tabela já existente sem precisar recriá-la. Com ele, é possível adicionar, remover ou alterar colunas, ajustar tipos de dados, renomear campos e incluir restrições como chaves primárias, estrangeiras ou unicidade. É especialmente útil em atualizações



de sistema, quando a tabela precisa se adaptar a novas exigências sem perda dos dados já armazenados. Diferente do DROP TABLE, que exclui toda a tabela, o ALTER TABLE faz alterações pontuais, preservando a estrutura e os dados. A seguir, será apresentada sua sintaxe.

SINTAXE DO COMANDO

```
ALTER TABLE NOME_DA_TABELA  
ADD COLUMN NOME_COLUNA TIPO_DE_DADO;
```

EXEMPLO DO COMANDO

```
ALTER TABLE ALUNO  
ADD COLUMN EMAIL VARCHAR(255);
```

RESULTADO DO COMANDO

ALUNO						
NOME	CPF	SEXO	DATA_NASCIMENTO	CIDADE	VALOR_PAGO	EMAIL



Vejam que nós adicionamos uma coluna do tipo texto chamada EMAIL capaz de armazenar até 255 caracteres. Já a sintaxe para excluir uma coluna de uma tabela é:

SINTAXE DO COMANDO

```
ALTER TABLE NOME_DA_TABELA  
DROP COLUMN NOME_COLUNA;
```

EXEMPLO DO COMANDO

```
ALTER TABLE ALUNO  
DROP COLUMN SEXO;
```

RESULTADO DO COMANDO

ALUNO					
NOME	CPF	DATA_NASCIMENTO	CIDADE	VALOR_PAGO	EMAIL

Percebam que a coluna SEXO foi excluída do resultado. Agora vamos ver como é a sintaxe para modificar o tipo de dado de uma coluna da tabela:



SINTAXE DO COMANDO (SQL SERVER / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ALTER COLUMN NOME_COLUNA TIPO_DE_DADO;
```

SINTAXE DO COMANDO (MYSQL / ORACLE PRÉ-10G)

```
ALTER TABLE NOME_DA_TABELA  
MODIFY COLUMN NOME_COLUNA TIPO_DE_DADO;
```

SINTAXE DO COMANDO (MYSQL / ORACLE 10G EM DIANTE)

```
ALTER TABLE NOME_DA_TABELA  
MODIFY NOME_COLUNA TIPO_DE_DADO;
```

E por que há três opções de sintaxe diferentes? Porque existem pequenas diferenças dependendo do SGBD utilizado. Vamos ver um exemplo utilizando a sintaxe do SQL Server:

```
ALTER TABLE ALUNO  
ALTER COLUMN CPF VARCHAR(14);
```

Notem que alteramos a coluna CPF de INT para VARCHAR(14). Por fim, é importante mencionar que - por meio desse comando - é possível também inserir ou excluir restrições de uma coluna. Você pode alterar uma coluna para indicar que ela não pode ser nula ou que ela será uma chave estrangeira. Vamos resumir as principais características sobre esse comando:

RENAME TABLE

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

O comando para renomear uma tabela altera apenas seu nome, sem afetar sua estrutura ou os dados armazenados. É utilizado quando se deseja que o nome da tabela represente melhor sua função no sistema. Sua execução pode exigir permissões específicas, geralmente atribuídas ao administrador do banco ou ao proprietário da tabela. Apesar de não modificar os dados, a renomeação pode impactar objetos dependentes, como chaves estrangeiras, visões e procedimentos.

Por isso, em bancos com estruturas mais complexas, essa operação deve ser feita com cautela, considerando possíveis vínculos com o nome original.

SINTAXE DO COMANDO

```
RENAME TABLE NOME_DA_TABELA  
TO NOVO_NOME_DA_TABELA;
```

EXEMPLO DO COMANDO

```
RENAME TABLE ALUNO  
TO ALUNO_ESTRATEGIA;
```



RESULTADO DO COMANDO

ALUNO_ESTRATEGIA					
NOME	CPF	DATA_NASCIMENTO	CIDADE	VALOR_PAGO	EMAIL

Alternativamente, também é possível utilizar o comando ALTER TABLE para modificar o nome de uma determinada tabela. Vejamos a sintaxe:

SINTAXE DO COMANDO

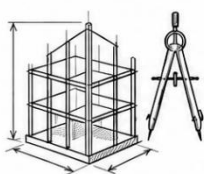
```
ALTER TABLE NOME_DA_TABELA
RENAME TO NOVO_NOME_DA_TABELA;
```

EXEMPLO DO COMANDO

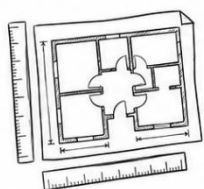
```
ALTER TABLE ALUNO
RENAME TO ALUNO_ESTRATEGIA;
```

SQL DDL: A Arquitetura do seu Banco de Dados

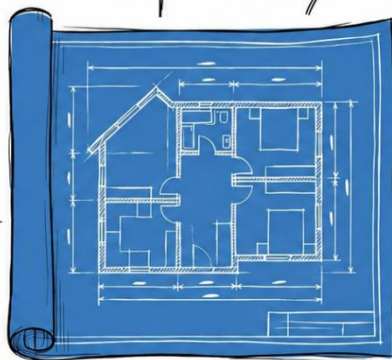
Entendendo a DDL: A Base de Tudo



O que é a Linguagem de Definição de Dados (DDL)?
É a parte do SQL que define a "arquitetura" do banco de dados.



A "Planta Baixa" do Banco de Dados.
Prepara o ambiente onde os dados serão armazenados, definindo tabelas, colunas e regras.



Comandos DDL Essenciais

CREATE TABLE



Cria uma nova tabela, definindo suas colunas e estrutura.

ALTER TABLE



Modifica a estrutura de uma tabela já existente.

TRUNCATE TABLE



Remove todos os dados de uma tabela, mas mantém a estrutura intacta.

DROP TABLE



Exclui permanentemente uma tabela, incluindo sua estrutura e todos os dados.

RENAME TABLE (MySQL)



Altera o nome de uma tabela existente sem afetar os dados. No padrão ANSI:
ALTER TABLE ... RENAME TO ...

Restrições

As restrições (constraints) em SQL são regras aplicadas às colunas de uma tabela para garantir a consistência, validade e confiabilidade dos dados. Elas impedem a inserção ou modificação de



informações que violem critérios previamente definidos. Quando uma ação infringe uma dessas regras, o comando é automaticamente rejeitado.

As restrições são fundamentais para manter a integridade dos dados, evitando duplicações, garantindo o preenchimento de campos obrigatórios e assegurando a coerência entre tabelas relacionadas. Podem ser aplicadas no nível de coluna, afetando um único campo, ou no nível de tabela, abrangendo múltiplas colunas ou a estrutura como um todo.

CONSTRAINT	DESCRIÇÃO
NOT NULL	Garante que uma coluna não possa ter um valor nulo.
UNIQUE	Garante que todos os valores de uma coluna sejam diferentes entre si.
PRIMARY KEY	Garante que todos os valores de uma coluna sejam diferentes entre si e não nulos.
FOREIGN KEY	Garante que ações não destruam links/relacionamentos entre as tabelas.
CHECK	Garante que os valores em uma coluna satisfaçam uma condição específica.
DEFAULT	Define um valor padrão para uma coluna, se nenhum valor for especificado.

NOT NULL

INCIDÊNCIA EM PROVA: MÉDIA

Essa restrição assegura que uma coluna não aceite valores nulos, exigindo que, em qualquer operação de inserção ou atualização, um valor válido seja obrigatoriamente fornecido para aquela coluna. Sua aplicação é fundamental em campos que são essenciais para o funcionamento do sistema, prevenindo o armazenamento de dados incompletos ou indefinidos. Essa restrição pode ser definida no momento da criação da tabela ou adicionada posteriormente por meio de comandos de alteração. Vejamos:

SINTAXE DO COMANDO

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1 TIPO_DE_DADO NOT NULL ,  
    ...  
);
```

EXEMPLO DO COMANDO

```
CREATE TABLE ALUNO (  
    NOME VARCHAR(20) NOT NULL ,  
    CPF INT PRIMARY KEY ,  
    SEXO CHAR(1) NOT NULL ,  
    DATA_NASCIMENTO DATE NOT NULL ,  
    CIDADE VARCHAR(50) ,  
    VALOR_PAGO INT ,  
);
```

Note que, durante a criação da tabela, definimos nome, tipo e restrição para cada coluna e algumas delas como NOT NULL. A segunda maneira é por meio do comando ALTER TABLE:

SINTAXE DO COMANDO (MYSQL / ORACLE 10G EM DIANTE)

```
ALTER TABLE ALUNO  
MODIFY CIDADE VARCHAR(50) NOT NULL;
```

Com a aplicação da restrição NOT NULL, a tabela ALUNO passa a exigir que as colunas NOME, CPF, SEXO, DATA_NASCIMENTO e CIDADE sejam obrigatoriamente preenchidas em qualquer inserção de dados. A única exceção é o campo VALOR_PAGO, que pode ficar em branco, pois não possui essa



restrição. Vale lembrar que NULL não é o mesmo que zero ou espaços em branco; NULL indica ausência total de valor, ou seja, um campo efetivamente vazio.

UNIQUE

INCIDÊNCIA EM PROVA: BAIXA

A restrição UNIQUE garante que os valores de uma coluna, ou de um conjunto de colunas, não se repitam em uma tabela, assegurando a exclusividade dos dados. Diferente da chave primária, ela permite valores NULL, conforme o SGBD, desde que os demais valores não sejam duplicados. Por exemplo, em uma coluna de e-mail com UNIQUE, o banco pode aceitar registros com NULL, mas rejeita valores repetidos. Essa restrição é comum em campos como CPF, telefone ou nome de usuário. Quando aplicada a múltiplas colunas, exige que a combinação entre os valores dessas colunas seja única em cada linha da tabela.

EXEMPLO DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
CREATE TABLE ALUNO (  
    NOME          VARCHAR(20)      NOT NULL      ,  
    CPF           INT              PRIMARY KEY    ,  
    SEXO          CHAR(1)         NOT NULL      ,  
    DATA_NASCIMENTO DATE         NOT NULL      ,  
    CIDADE        VARCHAR(50)     ,  
    MATRICULA     INT              UNIQUE        ,  
    VALOR_PAGO    INT              ,  
);
```

EXEMPLO DO COMANDO (MYSQL)

```
CREATE TABLE ALUNO (  
    NOME          VARCHAR(20)      NOT NULL      ,  
    CPF           INT              PRIMARY KEY    ,  
    SEXO          CHAR(1)         NOT NULL      ,  
    DATA_NASCIMENTO DATE         NOT NULL      ,  
    CIDADE        VARCHAR(50)     ,  
    MATRICULA     INT              ,  
    VALOR_PAGO    INT              ,  
    UNIQUE (MATRICULA)  
);
```

Note acima que a coluna MATRICULA é UNIQUE! Logo, essa coluna não pode ter valores repetidos. Um outro ponto interessante é que nós podemos dar um nome a uma restrição de unicidade:

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS  
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1    TIPO_DE_DADO    PRIMARY KEY    ,  
    NOME_COLUNA2    TIPO_DE_DADO    NOT NULL      ,  
    NOME_COLUNA3    TIPO_DE_DADO    ,  
    NOME_COLUNA4    TIPO_DE_DADO    NOT NULL      ,  
    NOME_COLUNA5    TIPO_DE_DADO    ,  
    CONSTRAINT NOME_DA_RESTRIÇÃO UNIQUE (NOME_COLUNA3, NOME_COLUNA5)  
    ...  
);
```



É possível também adicionar uma restrição de unicidade a uma coluna de uma tabela pré-existente por meio da sintaxe apresentada a seguir:

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ADD UNIQUE (NOME_DA_COLUNA);
```

Ao tentar adicionar uma restrição UNIQUE a uma coluna que já contém valores duplicados, o SGBD realiza uma verificação prévia para garantir que todos os dados existentes atendem à exigência de exclusividade. Caso identifique duplicatas, a operação é abortada e a restrição não é aplicada até que os dados sejam corrigidos. Assim como é possível incluir uma restrição UNIQUE, também é permitido removê-la posteriormente, caso se torne desnecessária. A seguir, vejamos a sua sintaxe:

SINTAXE DO COMANDO (MYSQL)

```
ALTER TABLE NOME_DA_TABELA  
DROP INDEX NOME_DA_RESTRICAO;
```

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
DROP CONSTRAINT NOME_DA_RESTRICAO;
```

PRIMARY KEY

INCIDÊNCIA EM PROVA: MÉDIA

A restrição de chave primária é usada para identificar de forma única cada registro em uma tabela. Ela exige que os valores da coluna, ou do conjunto de colunas, sejam únicos e não nulos, garantindo que cada linha tenha um identificador exclusivo e válido. Cada tabela pode ter apenas uma chave primária, que pode ser definida em uma única coluna ou em múltiplas colunas, formando uma chave composta. Essa restrição é fundamental para manter a integridade e a unicidade dos dados na tabela.

CARACTERÍSTICAS	DESCRIÇÃO
UNICIDADE	A principal característica da chave primária é que ela assegura que cada valor na coluna (ou nas colunas, caso seja uma chave composta) seja único. Isso significa que nenhum registro na tabela pode ter o mesmo valor para a chave primária.
NÃO NULA	A chave primária não pode conter valores NULL. Isso ocorre porque um valor nulo em uma chave primária significaria que não há uma maneira única de identificar aquele registro, o que violaria o propósito de integridade da chave primária.
IDENTIFICADOR ÚNICO	A chave primária é usada para identificar de forma exclusiva cada linha de uma tabela. Por exemplo, em uma tabela de clientes, um campo ID pode ser a chave primária, garantindo que cada cliente tenha um identificador único, sem duplicação ou valores nulos.
ÍNDICE AUTOMÁTICO	Quando você define uma chave primária, o banco de dados automaticamente cria um índice para essa coluna (ou conjunto de colunas). Isso ajuda a melhorar o desempenho das



consultas, especialmente nas operações de busca ou de join, já que o banco pode acessar as linhas de maneira mais eficiente.

PRIMARY KEY = NOT NULL + UNIQUE

- Uma coluna **UNIQUE** jamais poderá se repetir, mas poderá ser nula;
- Uma coluna **NOT NULL** jamais poderá ser nula, mas poderá se repetir;
- Uma coluna **PRIMARY KEY** jamais poderá ser nula e jamais poderá se repetir.

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1          TIPO_DE_DADO      PRIMARY KEY ,  
    NOME_COLUNA2          TIPO_DE_DADO      RESTRIÇÕES ,  
    NOME_COLUNA3          TIPO_DE_DADO      RESTRIÇÕES ,  
    ...  
);
```

SINTAXE DO COMANDO (MYSQL)

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1          TIPO_DE_DADO      RESTRIÇÕES ,  
    NOME_COLUNA2          TIPO_DE_DADO      RESTRIÇÕES ,  
    NOME_COLUNA3          TIPO_DE_DADO      RESTRIÇÕES ,  
    ...  
    PRIMARY KEY (NOME_COLUNA1)  
);
```

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS  
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1          TIPO_DE_DADO      RESTRIÇÕES ,  
    NOME_COLUNA2          TIPO_DE_DADO      RESTRIÇÕES ,  
    NOME_COLUNA3          TIPO_DE_DADO      RESTRIÇÕES ,  
    ...  
    CONSTRAINT NOME_DA_RESTRICAO PRIMARY KEY (NOME_COLUNA1, NOME_COLUNA2)  
);
```

Uma tabela poderá ter apenas uma chave primária, seja ela simples (quando contém apenas uma coluna) ou composta (quando contém mais de uma coluna). Dito isso, na sintaxe apresentada acima, nós temos uma - e apenas uma - chave primária (composta). É possível também adicionar uma restrição PRIMARY KEY a uma tabela pré-existente. Para tal, utiliza-se a sintaxe a seguir:

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ADD PRIMARY KEY (NOME_COLUNA1);
```

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)



-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS

```
ALTER TABLE NOME_DA_TABELA  
ADD CONSTRAINT NOME_DA_RESTRICAO PRIMARY KEY (NOME_COLUNA1, NOME_COLUNA2);
```

Por fim, da mesma forma que é possível adicionar uma restrição PRIMARY KEY a uma determinada coluna, é também possível retirá-la por meio da seguinte sintaxe:

SINTAXE DO COMANDO (MYSQL)

```
ALTER TABLE NOME_DA_TABELA  
DROP PRIMARY KEY;
```

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
DROP CONSTRAINT NOME_DA_RESTRICAO;
```

FOREIGN KEY

INCIDÊNCIA EM PROVA: MÉDIA

No modelo relacional, os dados são organizados em tabelas interligadas para garantir a coerência do sistema. Essa conexão é feita por meio das chaves estrangeiras, que relacionam uma coluna da tabela filha a uma chave, geralmente primária, da tabela pai. Esse vínculo garante a integridade referencial, exigindo que o valor presente na tabela filha exista na tabela pai. Dessa forma, o banco impede a inserção de dados incoerentes ou desconectados, mantendo a consistência entre as tabelas. A chave estrangeira é, portanto, o elemento que assegura a integração e a coordenação entre os dados do sistema.

CARACTERÍSTICAS	DESCRIÇÃO
INTEGRIDADE REFERENCIAL	Essa restrição garante que, ao inserir, atualizar ou excluir dados na tabela filha, o valor da chave estrangeira sempre se refira a um valor existente na tabela pai. Caso contrário, o banco de dados gera um erro e a operação é impedida. Isso evita dados órfãos (referências inválidas) nas tabelas.
RELACIONAMENTO ENTRE TABELAS	Essa restrição estabelece uma relação entre as tabelas. Por exemplo: em um sistema de Gerenciamento de Pedidos, pode-se ter uma Tabela CLIENTES e uma Tabela PEDIDOS, onde a Tabela PEDIDOS contém uma coluna de CLIENTE_ID que é uma chave estrangeira que se refere à ID da Tabela CLIENTES. Isso garante que todo pedido seja associado a um cliente válido.
REGRAS DE ATUALIZAÇÃO/ EXCLUSÃO	A chave estrangeira pode ser configurada com regras de atualização e exclusão. Isso define o que acontece com os registros da tabela filha quando um registro da tabela pai é atualizado ou excluído. As opções mais comuns são: CASCADE, SET NULL e RESTRICT.

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1 TIPO_DE_DADO PRIMARY KEY ,  
    NOME_COLUNA2 TIPO_DE_DADO RESTRIÇÕES ,  
    NOME_COLUNA3 TIPO_DE_DADO FOREIGN KEY REFERENCES TABELA_REFERENCIADA (CHAVE)  
);
```



SINTAXE DO COMANDO (MYSQL)

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1 TIPO_DE_DADO RESTRIÇÕES ,  
    NOME_COLUNA2 TIPO_DE_DADO RESTRIÇÕES ,  
    NOME_COLUNA3 TIPO_DE_DADO RESTRIÇÕES ,  
    PRIMARY KEY (NOME_COLUNA1) ,  
    FOREIGN KEY (NOME_COLUNA2) REFERENCES TABELA_REFERENCIADA (CHAVE)  
);
```

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS

```
CREATE TABLE NOME_DA_TABELA (  
    NOME_COLUNA1 TIPO_DE_DADO RESTRIÇÕES ,  
    NOME_COLUNA2 TIPO_DE_DADO RESTRIÇÕES ,  
    NOME_COLUNA3 TIPO_DE_DADO RESTRIÇÕES ,  
    CONSTRAINT NOME_DA_RESTRICAO FOREIGN KEY (NOME_COLUNA1)  
    REFERENCES TABELA_REFERENCIADA (CHAVE)  
);
```

Essa restrição é aplicada a uma coluna da tabela filha e estabelece uma referência a uma coluna da tabela pai, normalmente uma chave primária. Essa relação garante que alterações em uma tabela impactem corretamente a outra, preservando a consistência dos dados. Um aspecto importante é que a chave estrangeira não precisa ser definida no momento da criação da tabela; ela pode ser adicionada posteriormente, conforme as necessidades do sistema evoluem. Vejamos:

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ADD FOREIGN KEY (NOME_COLUNA1) REFERENCES TABELA_REFERENCIADA (CHAVE);
```

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS

```
ALTER TABLE NOME_DA_TABELA  
ADD CONSTRAINT NOME_DA_RESTRICAO  
FOREIGN KEY (NOME_COLUNA1)  
REFERENCES TABELA_REFERENCIADA (CHAVE);
```

Por fim, da mesma forma que é possível adicionar uma restrição FOREIGN KEY a uma determinada coluna, é também possível retirá-la por meio da seguinte sintaxe:

SINTAXE DO COMANDO (MYSQL)

```
ALTER TABLE NOME_DA_TABELA  
DROP FOREIGN KEY;
```

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
DROP CONSTRAINT NOME_DA_RESTRICAO;
```



Ao definir uma chave estrangeira, cria-se uma relação de dependência entre a tabela filha e a tabela pai, garantindo a integridade dos dados. No entanto, se um registro da tabela pai for excluído, pode ocorrer a geração de dados órfãos na tabela filha, ou seja, registros sem referência válida, comprometendo a integridade referencial do banco.

Para evitar esse problema, os SGBDs oferecem mecanismos de controle que definem o que deve acontecer quando um registro da tabela pai é atualizado ou excluído. Esses comportamentos são configurados por cláusulas específicas na definição da chave estrangeira, assegurando a manutenção automática e segura da integridade entre as tabelas.

TABELA PROFESSOR		TABELA DISCIPLINA		
CPF PROFESSOR	NOME PROFESSOR	COD	NOME DISCIPLINA	CPF PROFESSOR
111.111.111-11	DÉCIO TERROR	101	PORTUGUÊS	111.111.111-11
222.222.222-22	GUILHERME NEVES	102	DIREITO CONSTITUCIONAL	333.333.333-33
333.333.333-33	RICARDO VALE	103	DIREITO ADMINISTRATIVO	555.555.555-55

CHAVE PRIMÁRIA (TABELA PROFESSOR)

CHAVE PRIMÁRIA (TABELA DISCIPLINA)

CHAVE ESTRANGEIRA (REFERENCIA A TABELA PROFESSOR)

Vejam o exemplo anterior: se excluíssemos o registro da Tabela PROFESSOR cuja chave primária é 111.111.111-11, a disciplina cujo código é 101 da Tabela DISCIPLINA ficaria sem referência. E como podemos resolver esse problema? Podemos utilizar a cláusula ON DELETE CASCADE. Essa cláusula basicamente obriga a exclusão dos registros correspondentes das tabelas filhas que referenciam o registro excluído da Tabela Pai.

Agora eu deixei um erro de propósito nas tabelas acima. Pensem um minutinho aí para ver se vocês conseguem identificá-lo...

...

...

...

Notem que o terceiro registro da Tabela DISCIPLINA tem uma chave estrangeira 555.555.555-55. Essa chave estrangeira não está referenciando nenhuma linha da Tabela PROFESSOR. Logo, o registro da disciplina Direito Administrativo está órfão, já que sua chave estrangeira não encontra correspondência na chave primária da tabela Professor. Esse tipo de situação é um exemplo clássico de violação de integridade referencial em bancos de dados relacionais.

Voltemos ao exemplo: a exclusão do registro da TABELA PROFESSOR cuja chave primária é 111.111.111-11 excluiria também o registro cujo código é 101 da TABELA DISCIPLINA.

SINTAXE DO COMANDO

CREATE TABLE NOME_DA_TABELA (



```
NOME_COLUNA1 TIPO_DE_DADO RESTRIÇÕES ,
NOME_COLUNA2 TIPO_DE_DADO RESTRIÇÕES ,
NOME_COLUNA3 TIPO_DE_DADO RESTRIÇÕES ,
...
CONSTRAINT NOME_DA_RESTRICAO FOREIGN KEY (NOME_COLUNA1, ...)
REFERENCES TABELA_REFERENCIADA (CHAVE1, ...)
ON DELETE CASCADE
);
```

Existe também a cláusula ON UPDATE, que permite realizar algumas ações quando há uma alteração na tabela pai: CASCADE, RESTRICT, NO ACTION, SET NULL e SET DEFAULT.

CHECK

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Essa restrição é utilizada para garantir que os valores inseridos ou atualizados em uma coluna obedecem a uma condição específica. Ela funciona como uma validação automática, impedindo que dados fora do padrão sejam armazenados. Em resumo, você define uma regra lógica (uma expressão booleana), e o banco só aceita os valores que satisfazem essa regra. É uma forma eficiente de manter a qualidade e a coerência dos dados dentro da tabela, sem depender apenas da aplicação para fazer essas verificações.

CARACTERÍSTICAS	DESCRIÇÃO
VALIDAÇÃO DE DADOS	Essa restrição garante que os dados inseridos em uma coluna atendam a uma condição específica. Por exemplo, você pode usá-la para garantir que os valores em uma coluna de idade sejam maiores ou iguais a 18, ou que o valor de salário não seja negativo.
EXPRESSÕES BOOLEANAS	Essa restrição aceita qualquer expressão booleana que possa ser avaliada como verdadeira ou falsa. Essa expressão pode envolver operadores lógicos, comparações e até funções. Por exemplo: você pode verificar se um campo de idade está dentro de um intervalo de valores válidos ou se um campo de status é igual a um valor específico.
APLICAÇÃO	Pode ser aplicada em colunas individuais ou em tabelas inteiras. Em colunas, você pode, por exemplo, garantir que os valores sejam positivos ou que sigam algum padrão específico. Em tabelas, pode-se garantir que a combinação de valores em várias colunas siga certas regras.

EXEMPLO DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
CREATE TABLE ALUNO (
    NOME          VARCHAR(20)      NOT NULL ,
    CPF           INT              PRIMARY KEY ,
    SEXO          CHAR(1)         NOT NULL ,
    CIDADE        VARCHAR(50)      ,
    MATRICULA     INT              UNIQUE ,
    IDADE         INT              CHECK (IDADE >= 18)
);
```

EXEMPLO DO COMANDO (MYSQL)

```
CREATE TABLE ALUNO (
    NOME          VARCHAR(20)      NOT NULL ,
    CPF           INT              PRIMARY KEY ,
    SEXO          CHAR(1)         NOT NULL ,
    CIDADE        VARCHAR(50)      ,
    MATRICULA     INT              UNIQUE ,
```



```
IDADE INT ,  
CHECK (IDADE >= 18)  
);
```

EXEMPLO DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS

```
CREATE TABLE ALUNO (  
    NOME VARCHAR(20) NOT NULL ,  
    CPF INT PRIMARY KEY ,  
    SEXO CHAR(1) NOT NULL ,  
    CIDADE VARCHAR(50) ,  
    MATRICULA INT UNIQUE ,  
    IDADE INT ,  
    CONSTRAINT NOME_DA_RESTRICAO CHECK (IDADE >= 18 AND SEXO = 'F')  
);
```

No último exemplo, vimos uma restrição composta, que impõe duas condições ao mesmo tempo: a IDADE deve ser maior ou igual a 18 e o SEXO deve ser 'F'. Ou seja, a tabela só aceitará registros de mulheres maiores de idade. Assim como outras restrições que já estudamos, o CHECK também pode ser adicionado depois da criação da tabela – o que é útil quando você precisa adaptar regras com o tempo. A seguir, veremos como aplicar essa restrição em uma tabela já existente, utilizando a sintaxe apropriada:

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ADD CHECK (CONDICAO);
```

SINTAXE DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

-- PARA NOMEAR UMA RESTRIÇÃO OU DEFINI-LA PARA MÚLTIPLAS COLUNAS

```
ALTER TABLE NOME_DA_TABELA  
ADD CONSTRAINT NOME_DA_RESTRICAO CHECK (CONDICAO1 OPERADOR CONDIÇÃO2 ...);
```

Por fim, da mesma forma que é possível adicionar uma restrição de checagem, é também possível retirá-la por meio da seguinte sintaxe:

SINTAXE DO COMANDO (MYSQL)

```
ALTER TABLE NOME_DA_TABELA  
DROP CHECK NOME_DA_RESTRICAO;
```

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
DROP CONSTRAINT NOME_DA_RESTRICAO;
```

Essa restrição é usada para validar se os valores inseridos ou atualizados em uma coluna estão de acordo com condições estabelecidas pelas regras de negócio. Sua função é impedir o armazenamento de dados inválidos ou incoerentes, contribuindo para a qualidade e a confiabilidade das informações. Uma de suas



vantagens é a flexibilidade, permitindo que as regras sejam alteradas ou incluídas posteriormente, sem impactar os dados já existentes. No entanto, sua aplicação se limita a validações simples dentro da mesma tabela. Para regras mais complexas, que envolvem múltiplas tabelas ou lógica elaborada, são necessários recursos como triggers ou stored procedures, que serão tratados em outro momento.

DEFAULT

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Essa restrição é utilizada para definir um valor padrão para uma coluna quando nenhum valor for informado na hora da inserção dos dados. Logo, se o usuário não preencher aquele campo, o banco de dados automaticamente insere o valor definido como padrão. Isso ajuda a evitar campos vazios e também simplifica o processo de inserção, especialmente quando há um valor que, na maioria dos casos, pode ser assumido como regra. Vamos ver agora algumas características dessa restrição:

CARACTERÍSTICAS	DESCRIÇÃO
VALOR PADRÃO AUTOMÁTICO	Essa restrição permite que você defina um valor que será inserido automaticamente se nenhum valor for fornecido para a coluna durante a inserção de dados. Isso garante que a coluna sempre tenha um valor, mesmo que o usuário ou sistema não forneça um valor explícito para ela.
VALORES CONSTANTES OU EXPRESSÕES	O valor atribuído à coluna DEFAULT pode ser um valor constante (como um número ou uma string) ou até mesmo uma expressão que será avaliada no momento da inserção. Por exemplo, você pode definir um valor padrão como CURRENT_DATE para uma coluna de data, garantindo que a data atual seja inserida por padrão.
FACILIDADE DE MANUTENÇÃO	Definir valores padrão para colunas pode ajudar a manter a consistência no banco de dados e reduzir a necessidade de intervenção do usuário ao inserir dados.

EXEMPLO DO COMANDO (MYSQL / SQL SERVER / ORACLE / MS-ACCESS)

```
CREATE TABLE ALUNO (  
  NOME          VARCHAR(20)      NOT NULL      ,  
  CPF           INT             PRIMARY KEY    ,  
  SEXO          CHAR(1)        NOT NULL      ,  
  CIDADE        VARCHAR(50)    DEFAULT 'BRASÍLIA' ,  
  MATRICULA     INT            UNIQUE        ,  
);
```

No exemplo acima, todo registro que não tenha especificado um valor para a coluna CIDADE será automaticamente preenchido com o valor BRASÍLIA pelo próprio sistema.

SINTAXE DO COMANDO (MYSQL)

```
ALTER TABLE NOME_DA_TABELA  
ADD NOME_COLUNA SET DEFAULT VALOR;
```

SINTAXE DO COMANDO (SQL SERVER)

```
ALTER TABLE NOME_DA_TABELA  
ADD CONSTRAINT NOME_DA_RESTRICAO DEFAULT VALOR FOR NOME_COLUNA;
```

SINTAXE DO COMANDO (MS-ACCESS)



```
ALTER TABLE NOME_DA_TABELA  
ALTER COLUMN NOME_COLUNA SET DEFAULT VALOR;
```

SINTAXE DO COMANDO (ORACLE)

```
ALTER TABLE NOME_DA_TABELA  
MODIFY NOME_COLUNA DEFAULT VALOR;
```

Por fim, da mesma forma que é possível adicionar um valor padrão, é também possível retirá-lo por meio da seguinte sintaxe:

SINTAXE DO COMANDO (MYSQL)

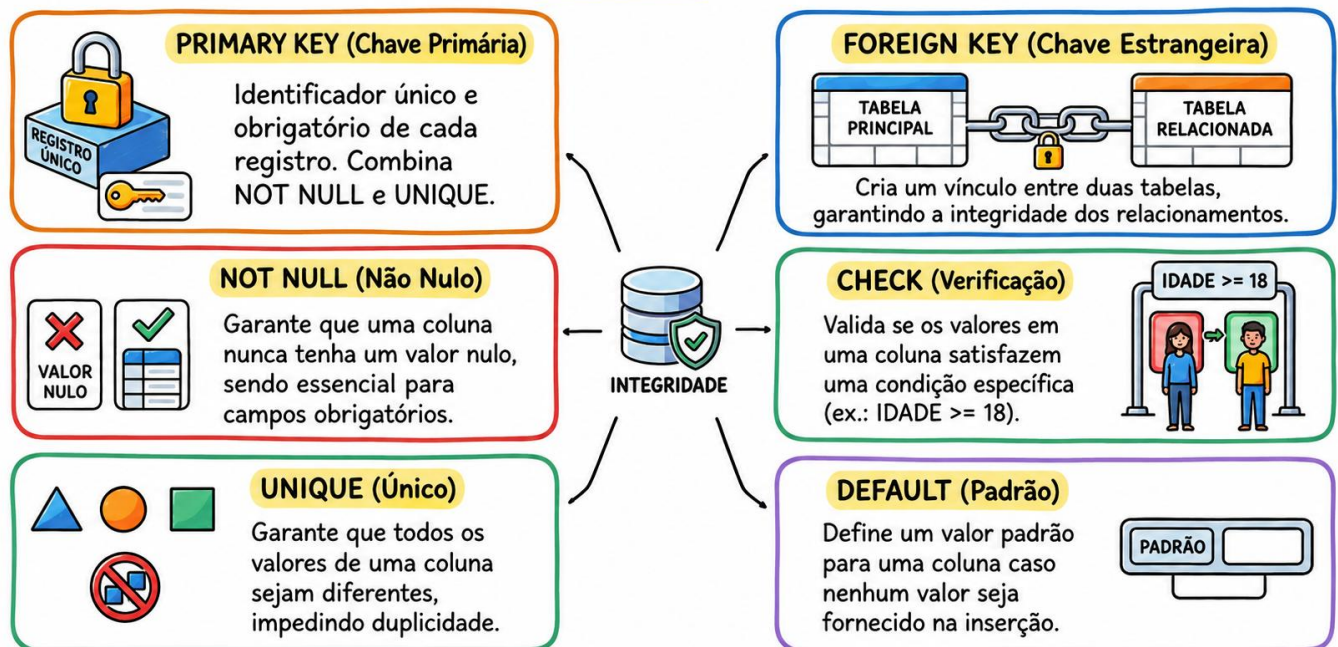
```
ALTER TABLE NOME_DA_TABELA  
ALTER NOME_COLUNA DROP DEFAULT;
```

SINTAXE DO COMANDO (SQL SERVER / ORACLE / MS-ACCESS)

```
ALTER TABLE NOME_DA_TABELA  
ALTER COLUMN NOME_COLUNA DROP DEFAULT;
```

Restrições SQL: As Regras de Ouro do seu Banco de Dados

Garanta a integridade e a **confiabilidade** dos dados no seu banco de dados.



DML (DATA MANIPULATION LANGUAGE)

DML (DATA MANIPULATION LANGUAGE)

A DML (Data Manipulation Language) é a parte do SQL destinada à manipulação direta dos dados armazenados no banco, permitindo inserir, consultar, atualizar e excluir informações dentro das tabelas. Por meio de seus comandos, a DML possibilita que usuários e aplicações interajam com os registros de forma dinâmica, realizando desde consultas simples até operações complexas que alteram o conteúdo do banco.



Agora o foco passa da estrutura da tabela para a manipulação dos dados armazenados. Esse papel é desempenhado pela DML (Data Manipulation Language), ou Linguagem de Manipulação de Dados. A DML reúne os comandos responsáveis por inserir, consultar, atualizar e excluir informações nas tabelas, permitindo a interação direta com os dados do banco. A seguir, serão apresentados esses comandos.

COMANDOS DML	DESCRIÇÃO
SELECT	Comando utilizado para realizar consultas a dados de uma ou mais tabelas do banco de dados.
INSERT	Comando utilizado para inserir um registro em uma tabela do banco de dados.
UPDATE	Comando utilizado para mudar valores de dados de registros de uma tabela do banco de dados.
DELETE	Comando utilizado para remover registros de uma tabela do banco de dados.

Os comandos da DML não alteram a estrutura das tabelas – ou seja, eles não mexem nas colunas, nas chaves ou em outros objetos do banco. O foco aqui é só nos dados que estão dentro das tabelas. A DML é essencial para interagir com os dados em tempo real, permitindo que usuários e sistemas consultem, insiram, atualizem ou excluam informações conforme necessário. É com ela que o banco de dados realmente “ganha vida” no dia a dia.

Comandos DML

INSERT²

INCIDÊNCIA EM PROVA: ALTA

Esse comando é usado para adicionar novos registros a uma tabela no banco de dados. Ele pode ser aplicado de duas formas: inserção completa ou incompleta. Na inserção completa, são informados valores para todas as colunas, seguindo a ordem padrão da tabela, sem necessidade de especificar os nomes das colunas. Já na inserção incompleta, quando apenas algumas colunas serão preenchidas, é obrigatório indicar os nomes dessas colunas, na mesma ordem dos valores fornecidos.

SINTAXE DO COMANDO I

-- INSERÇÃO DE TODOS OS VALORES PRESCINDE DA ESPECIFICAÇÃO DAS COLUNAS
INSERT INTO NOME_DA_TABELA
VALUES (VALOR_1, VALOR_2, VALOR_3, ...)

SINTAXE II DO COMANDO II

-- INSERÇÃO DE TODOS OS VALORES PRECISA DA ESPECIFICAÇÃO DAS COLUNAS
INSERT INTO NOME_DA_TABELA (NOME_COLUNA1, NOME_COLUNA2, NOME_COLUNA3, ...)
VALUES (VALOR_1, VALOR_2, VALOR_3, ...)

EXEMPLOS DO COMANDO I

INSERT INTO ALUNO_ESTRATEGIA

² O nome do comando pode ser apenas **INSERT** ou **INSERT INTO**.



```
VALUES ('ALICE', '11111111111', 'ALICE@ALICE.COM', '01-01-2001', 'BRASÍLIA', 200.00);

INSERT INTO ALUNO_ESTRATEGIA
VALUES ('BRUNO', '22222222222', 'BRUNO@BRUNO.COM', '02-02-2002', 'SÃO PAULO', 100.00);

INSERT INTO ALUNO_ESTRATEGIA
VALUES ('CAIO', '33333333333', 'CAIO@CAIO.COM', '03-03-2003', 'GOIÂNIA', 150.00);

INSERT INTO ALUNO_ESTRATEGIA
VALUES ('DIEGO', '44444444444', 'DIEGO@DIEGO.COM', '04-04-2004', 'SALVADOR', 250.00);

INSERT INTO ALUNO_ESTRATEGIA
VALUES ('ELIS', '55555555555', 'ELIS@ELIS.COM', '05-05-2005', 'BRASÍLIA', 50.00);
```

EXEMPLOS DO COMANDO II

```
INSERT INTO ALUNO_ESTRATEGIA (NOME, CPF, EMAIL, DATA_NASCIMENTO, CIDADE, VALOR_PAGO)
VALUES ('FABIO', '66666666666', 'FABIO@FABIO.COM', '06-06-2006', 'SALVADOR', 125.00);

INSERT INTO ALUNO_ESTRATEGIA (NOME, CPF, EMAIL, DATA_NASCIMENTO, CIDADE, VALOR_PAGO)
VALUES ('GABI', '77777777777', 'GABI@GABI.COM', '07-07-2007', 'BRASÍLIA', 225.00);

INSERT INTO ALUNO_ESTRATEGIA (NOME, CPF, EMAIL, DATA_NASCIMENTO, CIDADE, VALOR_PAGO)
VALUES ('HUGO', '88888888888', 'HUGO@HUGO.COM', '08-08-2008', 'BRASÍLIA', 50.00);

INSERT INTO ALUNO_ESTRATEGIA (NOME, CPF, EMAIL, DATA_NASCIMENTO, CIDADE, VALOR_PAGO)
VALUES ('IGOR', '99999999999', 'IGOR@IGOR.COM', '09-09-2009', 'RECIFE', 75.00);

INSERT INTO ALUNO_ESTRATEGIA (NOME, CPF, EMAIL, DATA_NASCIMENTO, CIDADE, VALOR_PAGO)
VALUES ('JOÃO', '00000000000', 'JOAO@JOAO.COM', '10-10-2010', 'NATAL', 175.00);
```

RESULTADO DOS COMANDOS

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIEGO	44444444444	DIEGO@DIEGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
IGOR	99999999999	IGOR@IGOR.COM	09-09-2009	RECIFE	75.00
JOÃO	00000000000	JOAO@JOAO.COM	10-10-2010	NATAL	175.00

UPDATE

INCIDÊNCIA EM PROVA: ALTA

Esse comando é usado para atualizar registros já existentes em uma tabela do banco de dados. Você pode atualizar todos os registros de uma vez ou apenas alguns específicos. Para isso, usamos a cláusula WHERE, que permite filtrar quais registros devem ser modificados com base em certas condições. Não se



preocupe: vamos ver a cláusula WHERE com mais detalhes mais adiante. Por enquanto, só lembre que ela é essencial para evitar que todas as linhas da tabela sejam alteradas acidentalmente. Vejamos:

SINTAXE DO COMANDO

```
UPDATE NOME_DA_TABELA  
SET NOME_DA_COLUNA_1 = VALOR_1, NOME_COLUNA2 = VALOR_2 ...  
WHERE LISTA_DE_CONDICICOES
```

EXEMPLO DO COMANDO

```
UPDATE ALUNO_ESTRATEGIA  
SET NOME = 'DIOGO', EMAIL = 'DIOGO@DIOGO.COM'  
WHERE CPF = 444444444444
```

RESULTADO DOS COMANDOS

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
IGOR	99999999999	IGOR@IGOR.COM	09-09-2009	RECIFE	75.00
JOÃO	00000000000	JOAO@JOAO.COM	10-10-2010	NATAL	175.00

Vamos imaginar um erro comum: o nome de um aluno foi cadastrado como Diego, mas o correto seria Diogo. Para corrigir isso, usamos o comando UPDATE na tabela ALUNO_ESTRATEGIA, alterando o NOME para DIOGO e o EMAIL para DIOGO@DIOGO.COM, mas apenas para o registro com o CPF 444444444444. Essa condição, baseada no CPF, é essencial – ela garante que apenas o aluno correto tenha seus dados alterados.

É uma forma segura de atualizar informações específicas sem correr o risco de mexer nos dados errados. E aí vem o alerta importante: se você não usar nenhuma condição (WHERE), o comando vai atualizar todos os registros da tabela. Por isso, muito cuidado ao usar UPDATE – sempre verifique se está aplicando a condição certa!

DELETE

INCIDÊNCIA EM PROVA: ALTA

O comando DELETE é usado para remover registros de uma tabela no banco de dados. Assim como no UPDATE, é possível excluir todos os registros ou apenas alguns, conforme definido pela cláusula WHERE. Essa cláusula é essencial para limitar a exclusão a registros específicos, garantindo segurança na operação. Se o DELETE for executado sem o WHERE, todos os dados da tabela serão apagados, mas sua estrutura permanecerá intacta, pronta para novas inserções – comportamento semelhante ao do TRUNCATE. A seguir, será apresentado um exemplo prático para ilustrar o uso desse comando.



SINTAXE DO COMANDO

DELETE FROM NOME_DA_TABELA **WHERE** LISTA_DE_CONDIÇÕES

EXEMPLO DO COMANDO

DELETE FROM ALUNO_ESTRATEGIA **WHERE** VALOR_PAGO = 175.00 **OR** CIDADE = 'RECIFE';

RESULTADO DO COMANDO

ALUNO_ESTRATEGIA					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

No exemplo apresentado, usamos o comando **DELETE** para remover registros da tabela **ALUNO_ESTRATEGIA** que atendem a um critério específico. A cláusula **WHERE** foi usada para definir essas condições – e somente os registros que cumprem pelo menos uma delas foram excluídos. Mais exatamente, o comando removeu:

- Registros com **VALOR_PAGO** = 175.00, ou
- Registros em que a **CIDADE** seja **RECIFE**.

Isso significa que basta uma das condições ser verdadeira para que o registro seja deletado. Foi o caso, por exemplo, dos alunos **JOÃO** e **IGOR**, cujas linhas foram removidas da tabela. Observação importante: a palavra-chave **OR** indica que as condições são alternativas. Se uma ou outra (ou ambas) forem verdadeiras, o registro será afetado pela operação.

SELECT

INCIDÊNCIA EM PROVA: ALTÍSSIMA

ESSE É O COMANDO MAIS IMPORTANTE DA AULA

Esse comando é usado para consultar e recuperar informações armazenadas em um banco de dados, permitindo a visualização dos dados contidos nas tabelas. Ele oferece recursos como filtros, ordenação e agrupamentos, conforme os critérios definidos pelo usuário. É essencial para extração e análise de dados, sendo amplamente utilizado em buscas e relatórios em sistemas que exigem acesso dinâmico às informações.

Essa prática de consulta está fundamentada em uma área teórica chamada Álgebra Relacional, que define operações formais para manipular dados em tabelas (ou relações, nesse contexto). A Álgebra Relacional



é a base matemática dos bancos de dados relacionais e serve de fundamento para linguagens como o SQL, que transforma esses conceitos em comandos mais acessíveis.

Entre as operações básicas da álgebra estão a Projeção, o Produto Cartesiano e a Seleção. Cada uma possui um correspondente no SQL, mas é importante evitar confusões – especialmente com a operação de seleção da álgebra relacional, que não se refere diretamente à cláusula SELECT do SQL. A seguir, será detalhado o que cada uma dessas operações realiza e como se relaciona com os comandos da linguagem SQL.

- PROJEÇÃO ↔ SELECT

Na Álgebra Relacional, a operação de projeção serve para selecionar apenas algumas colunas de uma tabela – ou melhor, de uma relação. Ela “projeta” um subconjunto de colunas, deixando de fora aquelas que não são relevantes para a consulta.

No SQL, isso corresponde à cláusula SELECT, onde você define quais colunas deseja visualizar no resultado. Por exemplo, se você quiser ver apenas os nomes e os e-mails de uma tabela de alunos, usaria a projeção para mostrar só essas colunas – exatamente como fazemos com o SELECT no SQL.

- PRODUTO CARTESIANO ↔ FROM

O produto cartesiano é uma operação da Álgebra Relacional que combina todas as linhas de duas tabelas entre si. Ou seja, cada linha da primeira tabela é combinada com todas as linhas da segunda, formando todas as combinações possíveis de registros.

No SQL, essa operação acontece de forma implícita quando usamos a cláusula FROM com mais de uma tabela sem definir nenhuma condição de junção. O resultado será exatamente esse: um produto cartesiano, que pode gerar um número muito grande de linhas se as tabelas forem grandes.

Por isso, na prática, quase sempre usamos condições de junção (como o JOIN ou WHERE) para controlar esse cruzamento e evitar resultados indesejados.

- SELEÇÃO ↔ WHERE

Na Álgebra Relacional, a operação de seleção é usada para filtrar as linhas de uma tabela com base em uma condição específica. Ela retorna apenas os registros que atendem a esse critério, deixando de fora os que não correspondem.

No SQL, essa operação corresponde à cláusula WHERE, onde você define exatamente quais registros deseja selecionar com base em determinada condição. É assim que fazemos consultas mais refinadas, buscando apenas os dados que realmente interessam.

SINTAXE DO COMANDO

```
-- AS CLÁUSULAS SÃO OPCIONAIS  
SELECT LISTA_DE_COLUNAS FROM LISTA_DE_TABELAS CLAUSULAS;
```

EXEMPLO DO COMANDO

```
SELECT * FROM ALUNO_ESTRATEGIA;
```



RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

Percebam no exemplo acima o uso do asterisco (*): esse símbolo é uma forma prática e rápida de indicar que queremos selecionar todas as colunas de uma tabela. Em vez de escrever o nome de cada coluna individualmente, usamos o * no comando SELECT para dizer ao banco: "Traga tudo!". Isso vale tanto para uma tabela única quanto para conjuntos de tabelas em consultas mais complexas.

É uma comodidade útil, especialmente quando você quer visualizar todos os dados disponíveis sem se preocupar com os nomes das colunas. Vamos ver agora um exemplo prático de como isso funciona.

EXEMPLO DE COMANDO			
SELECT	*	FROM	ALUNO_ESTRATEGIA
INTERPRETAÇÃO			
SELECIONE	TODAS AS COLUNAS	DA TABELA	ALUNO_ESTRATEGIA

E se eu quiser selecionar colunas específicas da tabela? Não há problema: nós podemos especificar, por exemplo, as colunas NOME e DATA_NASCIMENTO:

EXEMPLO DO COMANDO

```
SELECT NOME, DATA_NASCIMENTO FROM ALUNO_ESTRATEGIA;
```

RESULTADO DO COMANDO

RESULTADO	
NOME	DATA_NASCIMENTO
ALICE	01-01-2001
BRUNO	02-02-2002
CAIO	03-03-2003
DIOGO	04-04-2004
ELIS	05-05-2005
FABIO	06-06-2006
GABI	07-07-2007
HUGO	08-08-2008

COMANDO			
SELECT	NOME, DATA_NASCIMENTO	FROM	ALUNO_ESTRATEGIA



INTERPRETAÇÃO

SELECIONE	AS COLUNAS NOME E DATA_NASCIMENTO	DA TABELA	ALUNO ESTRATEGIA
-----------	-----------------------------------	-----------	------------------

Em algumas situações, não faz sentido retornar valores repetidos em uma consulta – e é aí que entra a palavra-chave **DISTINCT**. Ela é usada para eliminar duplicatas nos resultados, garantindo que apenas valores únicos sejam exibidos, mesmo que a tabela contenha registros repetidos. O **DISTINCT** é especialmente útil quando você quer ver quais são os diferentes valores presentes em uma coluna, sem se preocupar com quantas vezes cada um aparece. Vejamos um exemplo:

ALUNO ESTRATEGIA					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

Consideremos a nossa tabela de referência cima. Eu poderia selecionar a coluna cidade da tabela por meio do seguinte comando apresentado a seguir:

EXEMPLO DO COMANDO

```
SELECT CIDADE FROM ALUNO_ESTRATEGIA;
```

RESULTADO DO COMANDO

RESULTADO
CIDADE
BRASÍLIA
SÃO PAULO
GOIÂNIA
SALVADOR
BRASÍLIA
SALVADOR
BRASÍLIA
BRASÍLIA

Note que o resultado retornou cheio de duplicatas! E se essa tabela tivesse 1.000 registros? Ficaria inviável! Eu quero saber apenas quais são as cidades – sem repetições. Logo, posso fazer:

SINTAXE DO COMANDO

-- AS CLÁUSULAS SÃO OPCIONAIS

```
SELECT DISTINCT LISTA_DE_COLUNAS FROM LISTA_DE_TABELAS CLAUSULAS;
```

EXEMPLO DO COMANDO



```
SELECT DISTINCT CIDADE FROM ALUNO_ESTRATEGIA;
```

RESULTADO DO COMANDO

RESULTADO
CIDADE
BRASÍLIA
SÃO PAULO
GOIÂNIA
SALVADOR

Ao utilizar a palavra-chave **DISTINCT** em uma consulta SQL, os registros duplicados são eliminados, exibindo apenas valores únicos no resultado. Além desse recurso, outro elemento útil é o uso de aliases – apelidos temporários atribuídos a colunas ou tabelas durante a execução da consulta. Eles tornam o resultado mais legível e facilitam a escrita de comandos, especialmente em casos de nomes longos ou consultas complexas com múltiplas tabelas.

Esses aliases existem apenas enquanto a consulta é executada e servem para simplificar a identificação dos elementos envolvidos. O alias é criado com a palavra-chave **AS**, embora seu uso seja opcional. Pode-se usar **AS** para tornar o código mais claro, ou apenas inserir o apelido diretamente após o nome da coluna ou tabela. A seguir, será apresentada a sintaxe para utilização de aliases em SQL.

SINTAXE DO COMANDO

-- ALIAS PARA O NOME DA TABELA

```
SELECT NOME_COLUNA FROM NOME_DA_TABELA AS APELIDO CLAUSULAS;
```

-- ALIAS PARA O NOME DA COLUNA

```
SELECT NOME_COLUNA AS APELIDO FROM NOME_DA_TABELA CLAUSULAS;
```

EXEMPLO DO COMANDO

```
SELECT NOME AS N, DATA_NASCIMENTO AS DN FROM ALUNO_ESTRATEGIA AS AE;
```

O uso de aliases com nomes curtos, como mostrado no exemplo, melhora significativamente a leitura da consulta. No cotidiano, esses apelidos temporários são especialmente úteis em situações como consultas com várias tabelas (em junções), uso de funções no **SELECT** com necessidade de nomear os resultados, colunas com nomes longos ou pouco intuitivos, e quando há combinação de colunas em uma só, exigindo um novo nome para o resultado.

Além disso, outro recurso comum em SQL – inclusive em provas – é o uso de constantes nas consultas. Isso permite que, junto aos dados reais das tabelas, sejam retornados valores fixos definidos pelo próprio usuário. Na prática, a consulta exibe esse valor constante repetido em cada linha do resultado. Esse recurso é útil para testes, comparações ou para adicionar informações fixas ao resultado. Por exemplo, ao consultar a tabela **ALUNO_ESTRATÉGIA**, é possível incluir o número 1 como uma constante em cada linha da consulta.

EXEMPLO DO COMANDO

```
SELECT 1 AS RESULTADO FROM ALUNO_ESTRATEGIA;
```



RESULTADO DO COMANDO

RESULTADO
1
1
1
1
1
1
1
1
1
1

Ao executar essa consulta, o banco de dados não busca dados específicos de uma coluna, mas retorna o valor **1** repetidamente: uma vez para cada registro presente na tabela. Esse foi um exemplo com número, mas também podemos inserir textos (strings) nas consultas. Por exemplo, se você quiser que cada registro retorne o texto 'FLAMENGO É O MELHOR' em vez de dados de uma coluna, a consulta seria:

EXEMPLO DO COMANDO

```
SELECT 'FLAMENGO É O MELHOR' RESULTADO FROM ALUNO_ESTRATEGIA;
```

RESULTADO DO COMANDO

RESULTADO
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR
FLAMENGO É O MELHOR

Eu decidi deixar um tópico separado para as cláusulas por dois motivos. Primeiro, porque elas são importantíssimas e despençam em prova; segundo porque algumas delas são transversais e podem ser utilizadas em vários comandos de manipulação de dados (Ex: FROM e WHERE). Levantem-se, deem uma espreguiçada boa, tomem um café e vamos continuar porque ainda tem muita teoria pela frente. A seguir, uma lista das cláusulas que vamos estudar...



SQL: Comandos Essenciais da DML



INSERT

Adiciona novos registros (linhas) em uma tabela.



DELETE

Remove registros específicos de uma tabela.

A DML (Linguagem de Manipulação de Dados) interage com os dados em um banco (inserir, atualizar, excluir, consultar), sem alterar a estrutura das tabelas.



UPDATE

Modifica registros que já existem na tabela.



SELECT*

Consulta e recupera dados de uma ou mais tabelas.

Dicas Essenciais



SELECT é o comando mais importante: É a base para buscar, filtrar e visualizar qualquer informação no banco de dados.



Cuidado com UPDATE e DELETE: Use sempre a cláusula WHERE para especificar quais registros alterar ou apagar.



Foco apenas nos dados: A DML não modifica a estrutura da tabela (colunas e tipos de dados), apenas os registros dentro dela.

* Algumas bancas classificam o SELECT na DQL (Data Query Language).

Cláusulas

FROM

INCIDÊNCIA EM PROVA: ALTÍSSIMA

Essa cláusula é essencial em uma consulta SQL, pois define a origem dos dados que serão recuperados. É nela que se especificam as tabelas ou visões (views) utilizadas como fonte. Quando a consulta envolve múltiplas tabelas, o FROM também é o local onde se estabelecem as conexões entre elas, por meio das junções. Em resumo, essa cláusula marca o ponto de partida da consulta, indicando de onde os dados serão extraídos. A seguir, será apresentada sua sintaxe para melhor compreensão.

SINTAXE DO COMANDO

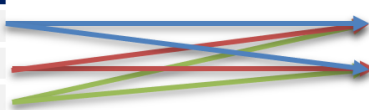
SELECT LISTA_DE_COLUNAS **FROM** TABELA1, TABELA2, ... **CLAUSULAS**;

Observem que é possível especificar mais de uma tabela separada por vírgula! Quando isso ocorre, temos um Produto Cartesiano. Vejamos um exemplo:

EXEMPLO DO COMANDO

SELECT * **FROM** TABELA_PROFESSOR, TABELA_DISCIPLINA;

TABELA_PROFESSOR	
NOME_PROFESSOR	CPF
DIEGO CARVALHO	111.111.111-11
RENATO DA COSTA	222.222.222-22
RICARDO VALE	333.333.333-33



TABELA_DISCIPLINA	
NOME_DISCIPLINA	CÓDIGO
INFORMÁTICA	101
DIREITO CONSTITUCIONAL	102



RESULTADO DO COMANDO

RESULTADO			
NOME PROFESSOR	CPF	NOME DISCIPLINA	CÓDIGO
DIEGO CARVALHO	111.111.111-11	INFORMÁTICA	101
RENATO DA COSTA	222.222.222-22	INFORMÁTICA	101
RICARDO VALE	333.333.333-33	INFORMÁTICA	101
DIEGO CARVALHO	111.111.111-11	DIREITO CONSTITUCIONAL	102
RENATO DA COSTA	222.222.222-22	DIREITO CONSTITUCIONAL	102
RICARDO VALE	333.333.333-33	DIREITO CONSTITUCIONAL	102

Quando utilizamos o asterisco (*) em uma consulta que envolve mais de uma tabela, o resultado inclui todas as colunas de todas as tabelas listadas. As linhas resultantes são formadas por combinações de cada linha de uma tabela com todas as da outra – operação conhecida como produto cartesiano. Nesse caso, o total de linhas é o produto entre as quantidades de linhas das tabelas envolvidas. Por exemplo, se uma tabela tem 3 linhas e a outra tem 2, o resultado será 6 linhas.

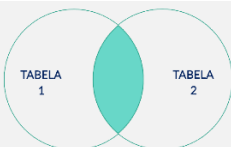
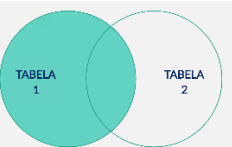
Em consultas desse tipo, os aliases se tornam ainda mais importantes. Isso porque, ao combinar tabelas que possuem colunas com nomes iguais, como CÓDIGO em PRODUTO e EMPRESA, pode haver ambiguidade. Sem indicar a origem de cada coluna, a consulta pode gerar erro ou resultados incorretos. Ao atribuir apelidos às tabelas, como P para PRODUTO e E para EMPRESA, é possível referenciar as colunas de forma clara e precisa – por exemplo, P.CODIGO para o código do produto e E.CODIGO para o código da empresa. Isso evita confusões e torna a consulta mais legível e segura.

JOIN

INCIDÊNCIA EM PROVA: ALTA

Essa cláusula é utilizada para combinar linhas/registros de duas ou mais tabelas, com base em uma coluna em comum entre elas. Em bancos de dados relacionais, as tabelas frequentemente têm dados que se referenciam mutuamente. A cláusula JOIN permite combinar essas tabelas de maneira eficiente para retornar dados completos a partir de informações distribuídas em várias tabelas.

O JOIN é geralmente utilizado em conjunto com FROM para definir as tabelas a serem unidas e a condição de combinação. As junções são baseadas em condições de relacionamento, geralmente utilizando chaves primárias e estrangeiras. O uso correto das junções é fundamental para obter resultados completos e precisos ao consultar dados em bancos de dados relacionais. Elas podem ser de quatro tipos diferentes³:

INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER JOIN
			
Retorna registros que possuem valores correspondentes em ambas as tabelas	Retorna todos os registros da tabela da esquerda e seus correspondentes da tabela da direita	Retorna todos os registros da tabela da direita e seus correspondentes da tabela da esquerda	Retorna todos os registros quando há uma correspondência na tabela da esquerda ou da direita

³ De acordo com o Padrão ANSI SQL92, são apenas esses quatro tipos. Em raras questões, pode aparecer ainda o Cross Join (que é essencialmente um Produto Cartesiano) e o Self Join (quando uma tabela é associada consigo mesma).



De acordo com a padronização do SQL, formalizada inicialmente pelo ANSI com o padrão SQL-89 e aperfeiçoada ao longo dos anos (com versões como SQL-92 e SQL-99), existem cinco tipos principais de JOIN: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN e CROSS JOIN. O CROSS JOIN, no entanto, corresponde ao produto cartesiano, que já foi estudado anteriormente, por isso não foi listado como novidade.

Além desses cinco, há ainda o SELF JOIN, que não faz parte do padrão oficial, mas é útil em situações específicas – como quando uma tabela se relaciona consigo mesma, por exemplo, uma tabela de funcionários em que cada funcionário pode ter um gerente também listado na mesma tabela. No entanto, como o SELF JOIN raramente é cobrado em provas, o foco permanece nos cinco JOINS padronizados.

O INNER JOIN, também conhecido simplesmente como JOIN, é utilizado para selecionar registros que possuem correspondência entre as tabelas envolvidas, ou seja, ele retorna apenas os dados que têm valores coincidentes nas colunas relacionadas de ambas as tabelas. A seguir, será apresentada sua sintaxe.

SINTAXE DO COMANDO

```
SELECT NOME_DAS_COLUNAS  
FROM TABELA1  
INNER JOIN TABELA2  
ON TABELA1.NOME_COLUNA = TABELA2.NOME_COLUNA;
```

PEDIDOS				
ID_PEDIDO	ID_CLIENTE	ID_FUNCIONARIO	DATA	ID_ENTREGADOR
10308	2	7	18/09/1996	3
10309	37	3	19/09/1996	1
10310	77	8	20/09/1996	2

CLIENTES						
ID_CLIENTE	NOME_CLIENTE	ID_INDICACAO	ENDEREÇO	CIDADE	CEP	PAIS
1	Alfredo	2	Rua X, 58	Berlin	70.000-00	Alemanha
2	Ana	3	Rua Y, 72	Miami	71.000-00	EUA
3	Antonio		Rua Z, 94	Tijuana	72.000-00	Mexico

EXEMPLO DO COMANDO

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE  
FROM PEDIDOS  
INNER JOIN CLIENTES  
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

Vamos analisar passo a passo cada uma das linhas do exemplo acima:

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE
```

Nessa linha, o SELECT é a cláusula que define as colunas que queremos retornar como resultado da consulta, isto é, as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE.

```
FROM PEDIDOS
```



A cláusula FROM define a tabela de origem dos dados. No caso analisado, a tabela PEDIDOS é a tabela principal da consulta, e os dados serão extraídos dela inicialmente.

INNER JOIN CLIENTES

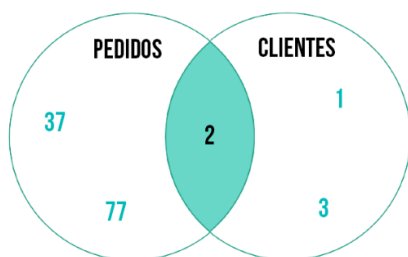
O INNER JOIN combina as duas tabelas, PEDIDOS e CLIENTES, com base em uma condição de relacionamento entre elas. Ele retorna apenas os registros em que há correspondência entre as duas tabelas, ou seja, ele somente inclui os pedidos que estão associados a um cliente na tabela CLIENTES. A tabela CLIENTES é unida à tabela PEDIDOS com base em uma condição definida na cláusula ON.

ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;

A cláusula ON especifica a condição de junção. Neste caso, a condição é que o ID_CLIENTE da tabela PEDIDOS deve ser igual ao ID_CLIENTE da tabela CLIENTES. Isso significa que a consulta irá combinar os registros da tabela PEDIDOS com os registros da tabela CLIENTES em que os valores do campo ID_CLIENTE em ambas as tabelas sejam iguais. Em outras palavras, ela retorna os pedidos e seus respectivos clientes.

Em suma, esse comando está dizendo para selecionar (SELECT) todas as linhas de ambas as tabelas (FROM PEDIDOS INNER JOIN CLIENTES) desde que exista uma correspondência entre as colunas (ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE) e, após isso, retornar as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE. O resultado do comando é apresentado na tabela a seguir:

RESULTADO DO COMANDO	
RESULTADO	
ID_PEDIDO	NOME_CLIENTE
10308	Ana



Vamos entender melhor? Note que a coluna em comum é ID_CLIENTE. A tabela PEDIDOS possui três valores para essa coluna: [2, 37, 77]; já a tabela CLIENTES também possui três valores para essa coluna: [1, 2, 3]. Como se trata de um INNER JOIN, a tabela resultante retornará apenas os registros que possuem correspondência em ambas as tabelas. Qual é o valor comum entre as tabelas? É o 2, que corresponde à cliente Ana. Lembrando também que é possível fazer um INNER JOIN com mais de duas tabelas, conforme mostra o exemplo seguinte (com três tabelas):

EXEMPLO DO COMANDO

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE, ENTREGADORES.NOME_ENTREGADOR  
FROM ((PEDIDOS  
INNER JOIN CLIENTES ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE)  
INNER JOIN ENTREGADORES ON PEDIDOS.ID_ENTREGADOR = ENTREGADORES.ID_ENTREGADOR);
```

Nós vimos que as tabelas que compõem um INNER JOIN devem possuir uma coluna em comum. Em geral, essas colunas em comum possuem o mesmo nome conforme vimos nos exemplos. No entanto, isso não é obrigatório - elas podem ter conteúdos correspondentes, mas nomes distintos. De toda forma, caso elas tenham o mesmo nome, é possível utilizar a palavra-chave USING para melhorar a leitura do código e sua compreensão. Vejamos como seria sua sintaxe:

SINTAXE DO COMANDO



```
SELECT LISTA_DE_COLUNAS  
FROM TABELA1  
INNER JOIN TABELA2  
USING (NOME_COLUNA_COMUM);
```

EXEMPLO DO COMANDO

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE  
FROM PEDIDOS  
INNER JOIN CLIENTES  
USING (ID_CLIENTE);
```

A cláusula USING especifica o nome da coluna comum entre as duas tabelas que será usada para realizar a junção. Em vez de escrever ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE, o USING permite simplificar a condição de junção, já que a coluna tem o mesmo nome em ambas as tabelas. Ao utilizá-lo, não é necessário repetir o nome da coluna, dado que o SQL já sabe que a junção deve ser feita com a coluna ID_CLIENTE, que tem o mesmo nome nas duas tabelas.

Já o LEFT JOIN (também chamado de LEFT OUTER JOIN) retorna todos os registros da tabela da esquerda, além dos registros correspondentes da tabela da direita. Vejamos a sintaxe:

SINTAXE DO COMANDO

```
SELECT NOME_DAS_COLUNAS  
FROM TABELA1  
LEFT JOIN TABELA2  
ON TABELA1.NOME_COLUNA = TABELA2.NOME_COLUNA;
```

EXEMPLO DO COMANDO

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE  
FROM PEDIDOS  
LEFT JOIN CLIENTES  
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

Vamos analisar passo a passo cada uma das linhas do exemplo acima:

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE
```

Nessa linha, o SELECT é a cláusula que define as colunas que queremos retornar como resultado da consulta, isto é, as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE.

```
FROM PEDIDOS
```

A cláusula FROM define a tabela de origem dos dados. No caso analisado, a tabela PEDIDOS é a tabela principal da consulta, e os dados serão extraídos dela inicialmente.

```
LEFT JOIN CLIENTES
```

O LEFT JOIN é utilizado para combinar registros de duas tabelas com base em uma condição especificada. Ele retorna todas as linhas da tabela à esquerda (PEDIDOS) e somente as linhas



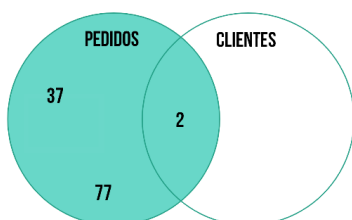
correspondentes da tabela à direita (CLIENTES). Se não houver uma correspondência na tabela CLIENTES para algum registro na tabela PEDIDOS, a consulta ainda retornará o registro da tabela PEDIDOS, mas as colunas da tabela CLIENTES terão valores NULL.

```
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

O ON define a condição de junção entre as tabelas. Ela especifica que a junção deve ser feita com base na correspondência entre as colunas ID_CLIENTE das tabelas PEDIDOS e CLIENTES. Isso significa que a consulta irá combinar os registros de PEDIDOS e CLIENTES em que o ID_CLIENTE de ambas as tabelas for igual. Se o ID_CLIENTE da tabela PEDIDOS não tiver uma correspondência na tabela CLIENTES, o valor da coluna NOME_CLIENTE será NULL para esse registro.

Em suma, esse comando está dizendo para selecionar (SELECT) todas as linhas da tabela da esquerda (FROM PEDIDOS LEFT JOIN), além dos registros da tabela da direita (CLIENTES), desde que exista uma correspondência entre as colunas (ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE) e, após isso, retornar as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE. O resultado é apresentado na tabela a seguir:

RESULTADO DO COMANDO	
RESULTADO	
ID_PEDIDO	NOME_CLIENTE
10308	Ana
10309	NULL
10310	NULL



Vamos entender melhor? Note que a coluna em comum é ID_CLIENTE. A tabela PEDIDOS possui três valores para essa coluna: [2, 37, 77]; já a tabela CLIENTES também possui três valores para essa coluna: [1, 2, 3]. Como se trata de um LEFT JOIN, a tabela resultante retornará todos os registros da tabela da esquerda e seus valores correspondentes da tabela da direita (se houver).

Já o RIGHT JOIN (também chamado de RIGHT OUTER JOIN) retorna todos os registros da tabela da direita, além dos registros correspondentes da tabela da esquerda. Vejamos a sintaxe:

SINTAXE DO COMANDO

```
SELECT NOME_DAS_COLUNAS  
FROM TABELA1  
RIGHT JOIN TABELA2  
ON TABELA1.NOME_COLUNA = TABELA2.NOME_COLUNA;
```

EXEMPLO DO COMANDO

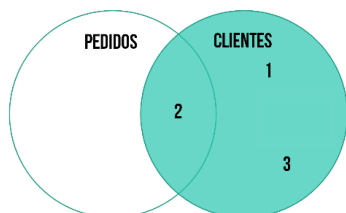
```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE  
FROM PEDIDOS  
RIGHT JOIN CLIENTES  
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

É muito semelhante ao anterior, mas agora pela direita. Ele está nos dizendo para selecionar (SELECT) todas as linhas da tabela da direita (CLIENTES), além dos registros da tabela da esquerda (FROM



PEDIDOS RIGHT JOIN) desde que exista uma correspondência entre as colunas (ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE) e, após isso, retornar as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE. O resultado é apresentado na tabela a seguir:

RESULTADO DO COMANDO	
RESULTADO	
ID_PEDIDO	NOME_CLIENTE
NULL	Alfredo
10308	Ana
NULL	Antonio



Vamos entender melhor? Note que a coluna em comum é ID_CLIENTE. A tabela PEDIDOS possui três valores para essa coluna: [2, 37, 77]; já a tabela CLIENTES também possui três valores para essa coluna: [1, 2, 3]. Como se trata de um RIGHT JOIN, a tabela resultante retornará todos os registros da tabela da direita e seus valores correspondentes da tabela da esquerda (se houver).

Já o FULL JOIN (também chamado de FULL OUTER JOIN) retorna todos os registros quando há uma correspondência da tabela esquerda com a direita ou da direita com a esquerda. Vejamos:

SINTAXE DO COMANDO

```
SELECT NOME_DAS_COLUNAS
FROM TABELA1
FULL OUTER JOIN TABELA2
ON TABELA1.NOME_COLUNA = TABELA2.NOME_COLUNA;
```

EXEMPLO DO COMANDO

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE
FROM PEDIDOS
FULL OUTER JOIN CLIENTES
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

Vamos analisar passo a passo cada uma das linhas do exemplo acima:

```
SELECT PEDIDOS.ID_PEDIDO, CLIENTES.NOME_CLIENTE
```

Nessa linha, o SELECT é a cláusula que define as colunas que queremos retornar como resultado da consulta, isto é, as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE.

```
FROM PEDIDOS
```

A cláusula FROM define a tabela de origem dos dados. No caso analisado, a tabela PEDIDOS é a tabela principal da consulta, e os dados serão extraídos dela inicialmente.

```
FULL OUTER JOIN CLIENTES
```

O FULL OUTER JOIN é um tipo de junção que retorna todos os registros de ambas as tabelas, PEDIDOS e CLIENTES, combinando-os onde houver correspondência. Para os registros onde não há



correspondência, o FULL OUTER JOIN ainda retorna esses registros, mas com valores NULL nas colunas da tabela onde não há correspondência. Em outras palavras, ele inclui:

- Todos os registros da tabela à esquerda (PEDIDOS), com valores correspondentes da tabela à direita (CLIENTES).
- Todos os registros da tabela à direita (CLIENTES), com valores correspondentes da tabela à esquerda (PEDIDOS).

Caso não haja correspondência entre as tabelas, a consulta ainda incluirá os registros e preencherá as colunas da tabela ausente com NULL.

```
ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE;
```

A cláusula ON define a condição de junção, que neste caso é PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE. Isso significa que a consulta está tentando combinar registros da tabela PEDIDOS com os registros da tabela CLIENTES onde o ID_CLIENTE de ambas as tabelas seja o mesmo.

Em suma, esse comando está dizendo para selecionar (SELECT) todas as linhas da tabela da direita e da esquerda (FROM PEDIDOS FULL OUTER JOIN CLIENTES) desde que exista uma correspondência entre as colunas (ON PEDIDOS.ID_CLIENTE = CLIENTES.ID_CLIENTE) e, após isso, retornar as colunas PEDIDOS.ID_PEDIDO e CLIENTES.NOME_CLIENTE. O resultado é apresentado na tabela a seguir:

RESULTADO DO COMANDO	
RESULTADO	
ID_PEDIDO	NOME_CLIENTE
10309	NULL
10308	Ana
10310	NULL
NULL	Alfredo
NULL	Antonio



Vamos entender melhor? Note que a coluna em comum é ID_CLIENTE. A tabela PEDIDOS possui três valores para essa coluna: [2, 37, 77]; já a tabela CLIENTES também possui três valores para essa coluna: [1, 2, 3]. Como se trata de um FULL OUTER JOIN, a tabela resultante retornará todos os registros da tabela da direita e da esquerda e seus valores correspondentes (se houver).

WHERE

INCIDÊNCIA EM PROVA: ALTÍSSIMA

Até aqui, já foi possível dominar a criação, alteração, exclusão e renomeação de tabelas, a aplicação de restrições para controle de dados, além das operações básicas de inserção, atualização, exclusão e consulta. Também foram explorados recursos como o SELECT com distinção de registros, uso de aliases e junções entre tabelas.

Agora, o próximo passo é compreender a cláusula WHERE, fundamental para filtrar registros com base em condições específicas. Com ela, é possível restringir os resultados de uma consulta, retornando apenas as linhas que atendem aos critérios definidos. Trata-se de um elemento central na personalização e precisão das buscas em SQL. A seguir, será apresentada sua aplicação prática.

ALUNO_ESTRATEGIA



NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...
FROM NOME_DA_TABELA1
WHERE CONDICAÇÃO;
```

EXEMPLO DO COMANDO

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE CIDADE = 'BRASÍLIA';
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

COMANDO					
SELECT	*	FROM	ALUNO_ESTRATEGIA	WHERE	CIDADE = 'BRASÍLIA'
INTERPRETAÇÃO					
ELECIONE	TODAS AS COLUNAS	DA TABELA	ALUNO_ESTRATEGIA	ONDE	A CIDADE SEJA BRASÍLIA

A cláusula WHERE não se aplica apenas ao comando SELECT – ela também é usada com os comandos UPDATE e DELETE para restringir quais registros devem ser alterados ou removidos. A condição usada nessa cláusula é uma expressão booleana, ou seja, uma expressão que resulta em TRUE ou FALSE. Com base nesse resultado, o SQL decide se a linha será incluída na operação. Para definir essas condições, utilizam-se operadores relacionais e lógicos, que comparam valores de diferentes formas.

OPERADORES RELACIONAIS

OPERADOR	DESCRIÇÃO	EXEMPLO
=	IGUAL	... WHERE NOME = 'DIEGO';
>	MAIOR	... WHERE VALOR_PAGO > 1000.00;
>=	MAIOR OU IGUAL	... WHERE IDADE >= 18;
<	MENOR	... WHERE DATA_NASCIMENTO < '01/01/2000';



<=	MENOR OU IGUAL	... WHERE VELOCIDADE <= 100;
<>	DIFERENTE	... WHERE CIDADE <> 'São Paulo';

OPERADORES AND, OR, NOT

No SQL, os operadores lógicos são usados para combinar múltiplas condições dentro de uma cláusula WHERE ou outras partes da consulta. Eles permitem realizar filtragens complexas e criar condições compostas. Vamos começar pelos operadores AND, OR e NOT:

- **AND (E):** esse operador é utilizado quando você quer que todas as condições especificadas sejam verdadeiras para que um registro seja incluído no resultado da consulta, ou seja, a consulta retorna registros onde todas as condições associadas ao AND sejam verdadeiras ao mesmo tempo.
- **OR (OU):** esse operador é utilizado quando você quer que qualquer uma das condições seja verdadeira para que o registro seja incluído na consulta, ou seja, a consulta retorna registros onde pelo menos uma das condições associadas ao OR seja verdadeira. Somente retorna falso se ambas as condições forem falsas.
- **NOT (NÃO):** esse operador é utilizado para negar uma condição, ou seja, ele inverte o resultado da condição. Se a condição original for verdadeira, o NOT a tornará falsa; se for falsa, ela se tornará verdadeira. Ele é útil quando você quer excluir registros que atendem a uma condição específica.

Agora nós vamos ver a sintaxe de cada operador e seus respectivos exemplos. Começamos pelo operador AND:

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE CONDICA01 AND CONDICA02 AND CONDICA03 ... ;
```

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE CIDADE = 'SALVADOR' AND VALOR_PAGO >= 200.00 ;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00

Vejam que o resultado filtrou a tabela de referência, retornando apenas os registros cuja cidade era SALVADOR e (AND) cujo VALOR_PAGO fosse maior ou igual a 200.00.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE CONDICA01 OR CONDICA02 OR CONDICA03 ... ;
```



EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE CIDADE = 'SALVADOR' OR VALOR_PAGO >= 200.00;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00

Vejam que o resultado filtrou a tabela de referência, retornando apenas os registros cuja cidade era SALVADOR ou (OR) cujo VALOR_PAGO fosse maior ou igual a 200.00.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE NOT CONDICA01;
```

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOT CIDADE = 'BRASÍLIA';
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00

Vejam que o resultado filtrou a tabela de referência, retornando apenas os registros cuja cidade não (NOT) era BRASÍLIA – perceba que ele retorna registros com SÃO PAULO, GOIÂNIA e SALVADOR.

É importante destacar também que é possível combinar esses operadores de diversas formas diferentes. Agora vamos falar sobre mais um operador: BETWEEN.

OPERADOR BETWEEN

Esse operador é usado para filtrar registros com base em um intervalo, permitindo selecionar valores que estejam entre dois limites, inclusive os próprios limites. Ele simplifica consultas que exigiriam múltiplas comparações, sendo aplicável a números, datas ou strings. Como é um operador inclusivo, ao definir um intervalo de 10 a 20, por exemplo, os valores 10 e 20 serão considerados. Com strings, a comparação é



feita em ordem alfabética; com datas, em ordem cronológica. É uma forma prática e eficiente de verificar se um valor está dentro de um determinado intervalo.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE NOME_COLUNA1 BETWEEN VALOR1 AND VALOR2;
```

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE VALOR_PAGO BETWEEN 150.00 AND 300.00;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00

Observem que o resultado contém valores entre 150.00 e 300.00 incluindo o 150.00 e o 300.00. Por essa razão, retornou o registro cujo nome é CAIO!

OPERADOR LIKE

Esse operador é usado na cláusula WHERE para buscar registros em colunas de texto que correspondam a um padrão específico, permitindo pesquisas parciais ou aproximadas. É útil quando o valor exato não é conhecido ou quando se deseja localizar dados com determinada estrutura textual. Para isso, o LIKE utiliza caracteres curingas (wildcards) que representam um ou mais caracteres dentro da string.

Esses curingas variam conforme o SGBD utilizado, por isso não serão detalhados neste momento. O importante é compreender que o LIKE facilita a localização de valores semelhantes, sem exigir correspondência exata. De toda forma, existem dois caracteres curingas principais que são frequentemente utilizados com o operador LIKE:

- **Porcentagem (%):** representa/substitui qualquer número de caracteres, incluindo nenhum - isso significa que ele pode substituir qualquer parte do texto.
- **Sublinhado (_):** representa/substitui um único caractere, ou seja, ele pode ser utilizado para substituir exatamente um caractere em uma string.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE NOME_COLUNA1 LIKE PADRAO
```



OPERADORES	DESCRIÇÃO
...WHERE NOME LIKE 'A%'	Retorna valores que comecem com "A".
...WHERE NOME LIKE '%A'	Retorna valores que terminem com "A".
...WHERE NOME LIKE '%IO%'	Retorna valores que possuam "IO" em qualquer posição.
...WHERE NOME LIKE '_R%'	Retorna valores que possuam um caractere e depois a letra "R".
...WHERE NOME LIKE '%A_'	Retorna valores que terminem com "A" mais apenas um caractere.
...WHERE NOME LIKE 'A_%'	Retorna valores que comecem com "A" e tem ao menos 3 caracteres.
...WHERE NOME LIKE '%A%O'	Retorna valores que possuam "A" depois "O" (imediatamente ou não).

Vejamos - para cada operador curinga - quais palavras poderiam ser aceitas (em verde) e quais não poderiam (em vermelho):

LIKE 'A%'	A	BOLA	PERSPECTIVA	GLÓRIA	PUXAR	ARCO
LIKE 'A%'	A	ACRÉSCIMO	ALVENARIA	ARCO	BOLA	PUXAR
LIKE '%IO%'	IO	CAIO	DIOGO	FABIO	ELIAS	LOURA
LIKE '_R%'	ARCO	IRMÃ	ORATÓRIA	URANIO	PUXAR	BARCO
LIKE '%A_'	AR	ARMAS	AURICULAR	BAÚ	A	ALÔ
LIKE 'A_%'	AVÔ	AVERIGUAR	ALÔ	AMOR	AR	A
LIKE '%A%O'	LAGO	AVÔ	AO	AVERIGUADO	LOBA	PIVOTAR

Dada a nossa clássica tabela apresentada abaixo, vamos ver como seriam diversos exemplos de comandos utilizando o operador LIKE e seus caracteres curingas:

ALUNO_ESTRATEGIA					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

EXEMPLO DO COMANDO

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE NOME LIKE 'A%';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno começa com a letra "A".

RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00

EXEMPLO DO COMANDO

```
SELECT *
```



```
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE '%A';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno termina com a letra "A" (não retorna registros).

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE '%IO';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno termina com as letras "IO".

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE '%R%';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno tem exatamente a segunda letra como "R" e, em seguida, qualquer sequência de caracteres.

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE '%A_';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno termina com a letra "A" seguida de exatamente um caractere qualquer (não retorna registros).

RESULTADO DO COMANDO



RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE 'A__%';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno começa com a letra "A" seguida de exatamente dois caracteres quaisquer.

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
WHERE NOME LIKE '%A%O';
```

O código acima seleciona todos os registros da tabela de referência em que o nome do aluno contém a letra "A" em qualquer posição e termina com a letra "O".

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00

OPERADOR IS NULL

Os operadores IS NULL e IS NOT NULL são utilizados para verificar a presença ou ausência de valores em uma coluna, uma vez que os operadores relacionais tradicionais não funcionam com valores nulos. Em SQL, NULL representa a ausência de dados e não é tratado como um valor comum, como números ou textos. Por isso, expressões como NULL = NULL não são consideradas verdadeiras, pois não é possível afirmar que duas ausências de valor sejam iguais.

Esse comportamento está relacionado à lógica de três valores do SQL, que considera, além de TRUE e FALSE, o valor UNKNOWN, usado quando não é possível determinar o resultado de uma comparação. Por isso, ao trabalhar com dados que podem estar ausentes, é necessário usar IS NULL para identificar registros sem valor, e IS NOT NULL para localizar registros que possuem algum conteúdo. Esses operadores são a forma correta e segura de lidar com verificações envolvendo NULL em consultas SQL.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1
```



WHERE NOME_COLUNA1 IS NULL;

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...
FROM NOME_DA_TABELA1
WHERE NOME_COLUNA1 IS NOT NULL;
```

OPERADOR IN

O operador IN é utilizado para filtrar registros com base em uma lista de valores possíveis para uma coluna. Ele simplifica a escrita de consultas que, de outra forma, exigiriam várias comparações usando o operador OR. Em vez de escrever múltiplas condições, o IN permite verificar se um valor pertence a um conjunto especificado, tornando a consulta mais clara e eficiente.

Esse operador pode ser aplicado a valores numéricos, textos e também em subconsultas, permitindo verificar se um valor existe em outra tabela ou conjunto de dados. Além disso, existe a forma NOT IN, que retorna os registros cujo valor não está na lista especificada. O uso do IN é especialmente útil para validar a existência de dados em múltiplos casos com uma única condição na cláusula WHERE.

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...
FROM NOME_DA_TABELA1
WHERE NOME_COLUNA1 IN (VALOR1, VALOR2,...);
```

EXEMPLO DO COMANDO

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE CIDADE IN ('SALVADOR', 'GOIANIA');
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00



Note que esse comando retorna todas as colunas da tabela em que a cidade seja SALVADOR OU GOIÂNIA. Aliás, esse operador é como a abreviação para várias condições OR. Vamos comparar:

COMPARATIVO DE COMANDOS

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE CIDADE = 'SALVADOR' OR CIDADE = 'GOIANIA';
```

=

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE CIDADE IN ('SALVADOR', 'GOIANIA');
```

Agora vamos falar sobre a utilização com subconsultas (também chamadas de subqueries ou consultas aninhadas), que são basicamente consultas dentro de outras consultas. Vejamos:

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...
FROM NOME_DA_TABELA1
WHERE NOME_COLUNA1 IN (SELECT ... FROM ...);
```

Vocês viram que existe uma consulta dentro de outra? Elas podem ser de dois tipos: correlacionadas ou não correlacionadas. As subconsultas correlacionadas são consultas que dependem e fazem referências às colunas de consultas externas a qual estão contidas; já as consultas não correlacionadas são consultas independentes das consultas externas nas quais estão contidas. Para ver como funciona, vamos adotar as tabelas seguintes:

CAPITAIS				
CODIGO	CAPITAL	ESTADO	SIGLA	REGIAO
001	MACEIÓ	ALAGOAS	AL	NORDESTE
002	SALVADOR	BAHIA	BA	NORDESTE
003	BELÉM	PARÁ	PA	NORTE
004	MANAUS	AMAZONAS	AM	NORTE
005	GOIÂNIA	GOIÁS	GO	CENTRO-OESTE
006	SÃO LUIS	MARANHÃO	MA	NORDESTE
007	CURITIBA	PARANÁ	PR	SUL
008	PORTO ALEGRE	RIO GRANDE DO SUL	RS	SUL

ALUNO_ESTRATEGIA					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00



FABIO	666666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	777777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	888888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

EXEMPLO DO COMANDO

```
SELECT *
FROM ALUNO_ESTRATEGIA
WHERE CIDADE IN (SELECT CAPITAL FROM CAPITAIS)
```

Trata-se de uma subconsulta não correlacionada. Note que primeiro executamos a consulta interna e depois executamos a consulta externa. Vejamos o resultado da consulta interna:

RESULTADO INTERMEDIÁRIO

RESULTADO
CAPITAL
MACEIÓ
SALVADOR
BELÉM
MANAUS
GOIÂNIA
SÃO LUIS
CURITIBA
PORTO ALEGRE

Agora vamos executar a consulta externa - observe que ela retornará todas as colunas da tabela ALUNO_ESTRATEGIA em que a cidade esteja dentre as cidades da tabela acima:

RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00

Já a subconsulta correlacionada será explicada dentro do contexto do operador EXISTS, logo vamos entendê-lo primeiro em detalhes...

OPERADOR EXISTS

O operador EXISTS é usado para verificar se uma subconsulta retorna pelo menos um registro. Se houver qualquer resultado, o EXISTS retorna TRUE; caso contrário, retorna FALSE. Ele é especialmente útil quando o objetivo é apenas confirmar a existência de dados relacionados, sem precisar analisar seu conteúdo. Também existe a forma NOT EXISTS, que retorna TRUE quando a subconsulta não retorna nenhuma linha.

Um ponto importante é sua eficiência: o EXISTS costuma ser mais rápido que o operador IN em subconsultas com muitos registros, pois ele interrompe a busca assim que encontra o primeiro resultado válido, enquanto o IN precisa processar toda a lista de valores. Trata-se, portanto, de um recurso ideal



para verificar a existência de dados de forma simples e performática. A seguir, será apresentada a sintaxe correspondente.

SINTAXE DO COMANDO

```
SELECT NOME_COLUNA1, NOME_COLUNA2, ...  
FROM NOME_DA_TABELA1  
WHERE EXISTS (SELECT ... FROM ... WHERE ...);
```

EXEMPLO DO COMANDO

```
SELECT *  
FROM CAPITAIS C  
WHERE EXISTS (SELECT CIDADE FROM ALUNO_ESTRATEGIA AE WHERE AE.CIDADE = C.CAPITAL);
```

RESULTADO DO COMANDO

RESULTADO				
CODIGO	CAPITAL	ESTADO	SIGLA	REGIAO
002	SALVADOR	BAHIA	BA	NORDESTE
005	GOIÂNIA	GOIÁS	GO	CENTRO-OESTE

O comando em questão retorna todos os registros da tabela CAPITAIS que também aparecem como cidades na tabela ALUNO_ESTRATEGIA, por meio de uma subconsulta correlacionada. Diferente da subconsulta não correlacionada – que pode ser executada de forma independente –, a subconsulta correlacionada depende de valores da consulta externa e, por isso, não pode ser executada isoladamente.

Nesse tipo de subconsulta, a execução ocorre uma vez para cada linha candidata da consulta externa. A cada iteração, valores da linha atual da consulta principal são usados para avaliar a subconsulta. O resultado final é construído com base no conjunto de execuções parciais da subconsulta correlacionada. Trata-se de uma técnica avançada, porém poderosa, especialmente útil em situações em que há dependência direta entre dados de duas tabelas. Imagine um cenário em que temos as duas tabelas abaixo e queremos retornar uma lista de clientes que já realizaram pedidos:

CLIENTES		
ID_CLIENTE	NOME_CLIENTE	CIDADE
1	JOÃO	SÃO PAULO
2	MARIA	RECIFE
3	PEDRO	SALVADOR

PEDIDOS		
ID_PEDIDO	ID_CLIENTE	VALOR_PEDIDO
101	1	250,00
102	2	300,00

EXEMPLO DO COMANDO

```
SELECT NOME_CLIENTE  
FROM CLIENTES C  
WHERE EXISTS (SELECT 1 FROM PEDIDOS P WHERE P.ID_CLIENTE = C.ID_CLIENTE);
```



No comando principal, nós estamos consultando a tabela CLIENTES (SELECT NOME_CLIENTE FROM CLIENTES C) e nós queremos encontrar clientes que já fizeram pedidos. Já a subconsulta está relacionada à tabela principal (CLIENTES). Isso significa que, para cada cliente da tabela CLIENTES, o SQL verifica na tabela PEDIDOS se existe pelo menos um pedido associado ao cliente. Aqui, P.ID_CLIENTE = C.ID_CLIENTE é o vínculo entre as duas tabelas.

A palavra-chave EXISTS verifica se a subconsulta retorna alguma linha. Se a subconsulta encontrar pelo menos um registro correspondente, o SQL entende que a condição é verdadeira e inclui o cliente na lista final. O SELECT 1 na subconsulta não retorna nenhuma informação específica (o valor 1 é apenas uma constante), porque o objetivo da subconsulta não é retornar dados, mas sim verificar se há alguma correspondência.

CLIENTES			PEDIDOS		
ID_CLIENTE	NOME_CLIENTE	CIDADE	ID_PEDIDO	ID_CLIENTE	VALOR_PEDIDO
1	JOÃO	SÃO PAULO	101	1	250,00
2	MARIA	RECIFE	102	2	300,00
3	PEDRO	SALVADOR			

- Na primeira linha da tabela CLIENTES, temos ID_CLIENTE = 1. Vamos procurar ID_CLIENTE = 1 na tabela PEDIDOS também? Está na primeira linha, logo nem precisamos verificar os outros registros da tabela PEDIDOS e já podemos retornar JOÃO.
- Na segunda linha da tabela CLIENTES, temos ID_CLIENTE = 2. Vamos procurar ID_CLIENTE = 2 na tabela PEDIDOS também? Está na segunda linha, logo nem precisamos verificar os outros registros da tabela PEDIDOS e já podemos retornar MARIA.
- Na terceira linha da tabela CLIENTES, temos ID_CLIENTE = 3. Vamos procurar ID_CLIENTE = 3 na tabela PEDIDOS também? Não consta em nenhuma linha da tabela PEDIDOS, logo não retornamos PEDRO.

Esse primeiro exemplo foi bem simples, mas esse costuma ser um assunto extremamente complexo. Dito isso, vejamos agora um caso mais difícil:

ARVORE_GENEALOGICA	
ASCENDENTE	DESCENDENTE
ALICE	LAURA
BRUNO	ELIS
BRUNO	HUGO
ELIS	CAIO
GABI	ALICE
HUGO	GABI
JUDITH	LAURA

Considerem a tabela acima com a base para os exemplos que vamos analisar. Note que a tabela possui duas colunas:

- Ascendente:** a pessoa que é considerada a ascendente.
- Descendente:** a pessoa que é considerada o descendente dessa ascendente.

Vejamos alguns exemplos de registros:



- A ascendente ALICE tem como descendente LAURA.
- O ascendente BRUNO tem como descendentes ELIS e HUGO.

Agora vamos analisar a seguinte consulta:

EXEMPLO DO COMANDO

```
SELECT *
FROM ARVORE_GENEALOGICA
WHERE EXISTS (SELECT * FROM ARVORE_GENEALOGICA AG
              WHERE AG.ASCENDENTE = 'BRUNO'
              AND AG.DESCENTENTE = ARVORE_GENEALOGICA.ASCENDENTE);
```

Em primeiro lugar, podemos notar que se trata de uma consulta aninhada, dado que temos uma consulta dentro de outra. Em segundo lugar, podemos notar que a consulta interna é uma subconsulta correlacionada, dado que temos uma referência a uma coluna da consulta externa. Dito isso, agora vamos analisar - para cada tupla da consulta externa - se a consulta interna retorna algum valor. Se sim, retornaremos também para a consulta externa; caso contrário, não.

Além disso, note que temos a consulta interna e a consulta externa tratando da mesma tabela, porém uma é chamada de ARVORE_GENEALOGICA e a outra tem um alias chamado AG:

ARVORE_GENEALOGICA	
ASCENDENTE	DESCENDENTE
ALICE	LAURA
BRUNO	ELIS
BRUNO	HUGO
ELIS	CAIO
GABI	ALICE
HUGO	GABI
JUDITH	LAURA

AG	
ASCENDENTE	DESCENDENTE
ALICE	LAURA
BRUNO	ELIS
BRUNO	HUGO
ELIS	CAIO
GABI	ALICE
HUGO	GABI
JUDITH	LAURA

Agora vamos pegar cada tupla da consulta externa e vamos validar a consulta interna, de modo que a tupla da consulta externa será retornada se, e somente se, a consulta interna retornar pelo menos uma tupla. Podemos ver que a consulta interna retornará todas as tuplas da tabela AG desde que a coluna ASCENDENTE da tabela AG seja "BRUNO" e a coluna DESCENDENTE da tabela AG seja igual à coluna ASCENDENTE da tabela ARVORE_GENEALOGICA. Agora vamos para a prática...

ARVORE_GENEALOGICA		AG	
ASCENDENTE	DESCENDENTE	ASCENDENTE	DESCENDENTE
ALICE	LAURA	ALICE	LAURA
BRUNO	ELIS	BRUNO	ELIS
BRUNO	HUGO	BRUNO	HUGO
ELIS	CAIO	ELIS	CAIO
GABI	ALICE	GABI	ALICE
HUGO	GABI	HUGO	GABI
JUDITH	LAURA	JUDITH	LAURA

A primeira tupla da tabela da consulta externa é (ALICE, LAURA). Essa tupla será retornada apenas se a consulta interna retornar alguma tupla. Vamos ver se ela retorna? A primeira tupla da consulta interna também é (ALICE, LAURA). Porém, a consulta interna só retornará essa tupla se AG.ASCENDENTE =



"BRUNO". Ora, acabamos de ver que - para essa tupla - AG.ASCENDENTE é "ALICE", logo nem precisamos ver o restante da consulta porque essa tupla já não será retornada.

Vamos agora para a segunda tupla da tabela da consulta interna: (BRUNO, ELIS). A consulta interna só retornará essa tupla se AG.ASCENDENTE = "BRUNO". É realmente "BRUNO", mas ainda não acabou: AG.DECENDENTE deve ser igual a ARVORE_GENEALOGICA.ASCENDENTE. Ora, AG.DECENDENTE é ELIS e ARVORE_GENEALOGICA.ASCENDENTE é ALICE. Logo, essa tupla também não será retornada porque não cumpriu as duas condições do operador AND. Viram que nós temos que fazer um por um?

Em tese, teríamos que fazer $7 \times 7 = 49$ avaliações. No entanto, é possível identificar alguns atalhos: o nome de AG.ASCENDENTE deve ser "BRUNO". Isso só ocorre em duas oportunidades:

ARVORE_GENEALOGICA		AG	
ASCENDENTE	DESCENDENTE	ASCENDENTE	DESCENDENTE
ALICE	LAURA	ALICE	LAURA
BRUNO	ELIS	BRUNO	ELIS
BRUNO	HUGO	BRUNO	HUGO
ELIS	CAIO	ELIS	CAIO
GABI	ALICE	GABI	ALICE
HUGO	GABI	HUGO	GABI
JUDITH	LAURA	JUDITH	LAURA

Vejam que ocorre somente nas tuplas (BRUNO, ELIS) e (BRUNO, HUGO). Nesses dois casos, temos AG.DECENDENTE = (ELIS, HUGO). Por fim, vejamos que temos ELIS ou HUGO como ASCENDENTE na tabela ARVORE_GENEALOGICA apenas em duas oportunidades: (ELIS, CAIO) e (HUGO, GABI). Logo, essas são as tuplas que serão retornadas pela consulta externa. Interessante, não? Vejamos como isso seria em termos de tabela:

ARVORE_GENEALOGICA		AG	
ASCENDENTE	DESCENDENTE	ASCENDENTE	DESCENDENTE
ALICE	LAURA	ALICE	LAURA
BRUNO	ELIS	BRUNO	ELIS
BRUNO	HUGO	BRUNO	HUGO
ELIS	CAIO	ELIS	CAIO
GABI	ALICE	GABI	ALICE
HUGO	GABI	HUGO	GABI
JUDITH	LAURA	JUDITH	LAURA

RESULTADO DO COMANDO	
ASCENDENTE	DESCENDENTE
ELIS	CAIO
HUGO	GABI

Em suma, nós podemos dizer que o código funciona da seguinte forma:

1. A consulta principal (consulta externa) está iterando sobre os registros da tabela ARVORE_GENEALOGICA.



2. Para cada registro dessa tabela, a subconsulta está verificando se esse ascendente é descendente de BRUNO.
3. Quando, por exemplo, o ascendente da linha sendo verificada é ELIS, a subconsulta vai verificar se ELIS é um descendente de BRUNO.
4. Se isso for verdadeiro (ou seja, se o descendente de BRUNO for ELIS), então o registro com ELIS como ascendente será incluído nos resultados finais da consulta.

A consulta irá retornar todos os registros da tabela ARVORE_GENEALOGICA em que o ascendente de um registro é descendente de BRUNO. Com base na tabela fornecida, a consulta retornaria:

- ELIS (descendente de BRUNO e ascendente de CAIO).
- HUGO (descendente de BRUNO e ascendente de GABI).

A consulta verifica se existe um registro em que o ascendente é um descendente de BRUNO, e, se esse registro existir, ele será incluído no conjunto de resultados. Conforme já disse, as provas costumam pegar pesado nesse assunto e ele pode ser considerado tranquilamente o assunto mais difícil de SQL. E mais: esse assunto realmente cai em prova! Se vocês não conseguiram entender, sugiro assistir a videoaula para ter uma noção mais visual do conteúdo.

GROUP BY

INCIDÊNCIA EM PROVA: MÉDIA

Essa cláusula é usada quando queremos agrupar registros de uma tabela com base em uma ou mais colunas, e aplicar funções agregadas sobre esses grupos. Isso significa que, ao usar o GROUP BY, você pode calcular somas, médias, contagens, valores máximos ou mínimos, entre outros – mas agora organizados por grupos, de acordo com os valores das colunas escolhidas.

Por exemplo, dá para saber quantos alunos existem por cidade, ou qual foi o valor médio pago em cada turma, e assim por diante. Vamos ver agora como é a sintaxe dessa cláusula na prática:

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS, FUNCAO_DE_AGREGACAO(COLUNA)
FROM NOME_DA_TABELA
WHERE CONDIÇÕES --OPCIONAL
GROUP BY LISTA_DE_COLUNAS;
```

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

EXEMPLO DO COMANDO



```
SELECT CIDADE, COUNT(CPF)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE;
```

COMANDO					
SELECT	CIDADE, COUNT(CPF)	FROM	ALUNO_ESTRATEGIA	GROUP BY	CIDADE
↓	↓	↓	↓	↓	↓
INTERPRETAÇÃO					
SELECIONE	AS COLUNAS CIDADE E COUNT(CPF)	DA TABELA	ALUNO_ESTRATEGIA	E AGRUPE POR	CIDADE

Essa cláusula buscará registros de uma tabela que possuem um valor em comum para um ou mais atributos e os agrupará baseado em algum critério de agrupamento (soma, média, quantidade, entre outros). No caso acima, ele buscará registros que tenham o mesmo valor para o atributo CIDADE e os agrupará pela quantidade (BRASÍLIA tem quatro aparições; SÃO PAULO tem uma aparição; GOIÂNIA tem uma aparição; e SALVADOR tem duas aparições). Vejam o resultado:

RESULTADO DO COMANDO	
RESULTADO	
CIDADE	COUNT(CPF)
BRASÍLIA	4
SÃO PAULO	1
GOIÂNIA	1
SALVADOR	2

Na tabela a seguir, podemos ver outras funções de agregação - lembrando que todas elas podem ser utilizadas com o operador DISTINCT! Em seguida, veremos outros exemplos com essas funções:

FUNÇÕES	AGREGAÇÃO	DESCRIÇÃO
COUNT()	Quantidade	Essa função conta a quantidade total de dados de um dado campo.
SUM()	Soma	Essa função soma valores numéricos de um dado campo ⁴ .
AVG()	Média	Essa função calcula a média aritmética simples de um conjunto de valores numéricos.
MAX()	Máximo	Essa função retorna o maior valor encontrado de um dado campo.
MIN()	Mínimo	Essa função retorna o menor valor encontrado de um dado campo.

EXEMPLO DO COMANDO

```
SELECT CIDADE, SUM(VALOR_PAGO)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE;
```

⁴ Um detalhe que cai sobre a função SUM(): é possível inserir um valor de tal modo que, para cada registro encontrado, esse valor seja somado. Por exemplo: SUM(2) somará duas unidades para cada registro encontrado. Logo, se encontrou 3 registros, retornará 2x3 = 6. Além disso, é importante mencionar que SUM(1) = COUNT(*) = COUNT(1).



RESULTADO DO COMANDO

RESULTADO	
CIDADE	SUM(VALOR_PAGO)
BRASÍLIA	525.00
SÃO PAULO	100.00
GOIÂNIA	150.00
SALVADOR	375.00

EXEMPLO DO COMANDO

```
SELECT CIDADE, AVG(VALOR_PAGO)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE;
```

RESULTADO DO COMANDO

RESULTADO	
CIDADE	AVG(VALOR_PAGO)
BRASÍLIA	131.25
SÃO PAULO	100.00
GOIÂNIA	150.00
SALVADOR	187.50

EXEMPLO DO COMANDO

```
SELECT CIDADE, MAX(VALOR_PAGO)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE;
```

RESULTADO DO COMANDO

RESULTADO	
CIDADE	MAX(VALOR_PAGO)
BRASÍLIA	225.00
SÃO PAULO	100.00
GOIÂNIA	150.00
SALVADOR	250.00

EXEMPLO DO COMANDO

```
SELECT CIDADE, MIN(VALOR_PAGO)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE;
```

RESULTADO DO COMANDO

RESULTADO	
CIDADE	MIN(VALOR_PAGO)
BRASÍLIA	50.00
SÃO PAULO	100.00



GOIÂNIA	150.00
SALVADOR	125.00

Lembrando que as funções de agregação podem ser utilizadas também em outras cláusulas sem necessariamente a presença do GROUP BY, como é mostrado a seguir:

EXEMPLO DO COMANDO

```
SELECT MAX(VALOR_PAGO)
FROM ALUNO_ESTRATEGIA
```

RESULTADO DO COMANDO

RESULTADO
MAX(VALOR_PAGO)
250.00

HAVING

INCIDÊNCIA EM PROVA: MÉDIA

A cláusula WHERE é usada para filtrar registros individualmente antes da aplicação do GROUP BY, e por isso não permite o uso de funções agregadas nem de aliases definidos no SELECT. Já a cláusula HAVING atua após a formação dos grupos pelo GROUP BY e permite filtrar esses grupos com base em condições que envolvem funções agregadas, como COUNT, SUM, AVG, entre outras. Por esse motivo, HAVING pode utilizar tanto funções agregadas quanto os apelidos definidos no SELECT.

Em um exemplo prático, se o objetivo for exibir apenas os grupos onde a contagem de CPF é maior que 1, o HAVING é quem permite aplicar essa condição após os dados serem agrupados.

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS, FUNCAO_DE_AGREGACAO(COLUNA)
FROM NOME_DA_TABELA
WHERE CONDIÇÕES --OPCIONAL
GROUP BY LISTA_DE_COLUNAS
HAVING CONDIÇÕES;
```

EXEMPLO DO COMANDO

```
SELECT CIDADE, COUNT(CPF)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE
HAVING COUNT(CPF) > 1;
```

RESULTADO DO COMANDO

RESULTADO	
CIDADE	COUNT(CPF)
BRASÍLIA	4
SALVADOR	2

COMANDO



SELECT	CIDADE, COUNT(CPF)	FROM	ALUNO_ESTRATEGIA	GROUP BY	CIDADE	HAVING	COUNT(CPF) >1
INTERPRETAÇÃO							
SELECIONE	AS COLUNAS CIDADE E COUNT(CPF)	DA TABELA	ALUNO_ESTRATEGIA	E AGRUPE POR	CIDADE	QUE TENHA	CONTAGEM >1

A coluna utilizada na cláusula HAVING deve necessariamente estar na lista de colunas selecionadas no SELECT ou estar contida dentro de uma função de agregação. Dessa forma, se fizermos um SELECT de CIDADE junto de uma função de agregação selecionada, a cláusula HAVING poderá filtrar por CIDADE, pela função de agregação selecionada ou por uma coluna (Ex: VALOR_PAGO) desde que ela esteja contida dentro de uma função de agregação. Exemplo:

EXEMPLO DO COMANDO

```
SELECT CIDADE, COUNT(CPF)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE
HAVING MAX(VALOR_PAGO) > 100;
```

Note que VALOR_PAGO não pode ser utilizado no HAVING porque não consta do SELECT, mas como está dentro de uma função de agregação (MAX), sua utilização é permitida.

RESULTADO DO COMANDO

RESULTADO	
CIDADE	COUNT(CPF)
BRASÍLIA	4
GOIÂNIA	1
SALVADOR	2

ORDER BY

INCIDÊNCIA EM PROVA: ALTA

A cláusula ORDER BY é utilizada para ordenar os registros retornados por uma consulta, permitindo uma visualização mais organizada e clara dos dados. Ela pode ordenar os resultados em ordem crescente (ASC), que é o padrão, ou em ordem decrescente (DESC), de acordo com o critério definido. Essa ordenação pode ser aplicada a colunas simples ou a resultados de funções agregadas, sendo especialmente útil após o uso de GROUP BY e HAVING, quando se deseja destacar ou facilitar a leitura dos dados agrupados. A seguir, será demonstrada sua aplicação prática:

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS, FUNCAO_DE_AGREGACAO(COLUNA)
FROM NOME_DA_TABELA
WHERE CONDIÇÕES --OPCIONAL
GROUP BY LISTA_DE_COLUNAS --OPCIONAL
HAVING CONDIÇÕES --OPCIONAL
ORDER BY COLUNA1 ASC | DESC, COLUNA2 ASC | DESC, ...;
```

EXEMPLO DO COMANDO C/ ASC



```
SELECT CIDADE, COUNT(CPF)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE
HAVING COUNT(CPF) > 1
ORDER BY COUNT(CPF) ASC;
```

RESULTADO DO COMANDO C/ ASC

RESULTADO	
CIDADE	COUNT(CPF)
SALVADOR	2
BRASÍLIA	4

EXEMPLO DO COMANDO C/ DESC

```
SELECT CIDADE, COUNT(CPF)
FROM ALUNO_ESTRATEGIA
GROUP BY CIDADE
HAVING COUNT(CPF) > 1
ORDER BY COUNT(CPF) DESC;
```

RESULTADO DO COMANDO C/ DESC

RESULTADO	
CIDADE	COUNT(CPF)
BRASÍLIA	4
SALVADOR	2

INTERPRETAÇÃO									
SELECT	CIDADE, COUNT(CPF)	FROM	ALUNO_ESTRATEGIA	GROUP BY	CIDADE	HAVING	COUNT(CPF) >1	ORDER BY	ASC
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
INTERPRETAÇÃO									
SELECIONE	AS COLUNAS CIDADE E COUNT(CPF)	DA TABELA	ALUNO_ESTRATEGIA	E AGRUPE POR	CIDADE	QUE TENHA	CONTAGEM >1	EM ORDEM	CRESC

A coluna utilizada para ordenação na cláusula ORDER BY deve necessariamente estar na lista de colunas do SELECT, em uma função de agregação qualquer ou ainda em uma coluna definida em uma tabela do FROM. Dessa forma, se fizermos um SELECT de NOME e CIDADE, a cláusula ORDER BY poderá ordenar por NOME, por CIDADE ou por outra coluna (Ex: VALOR_PAGO) desde que ela esteja contida dentro de uma função de agregação.

Essa cláusula organiza os resultados de uma consulta com base no valor de uma ou mais colunas. A ordenação pode ser feita de duas formas:

- **Crescente (ASC):** os valores são organizados do menor para o maior, ou seja, em ordem alfabética (para textos) ou numérica (para números).
- **Decrescente (DESC):** os valores são organizados do maior para o menor, ou seja, em ordem alfabética inversa (para textos) ou numérica (para números).

Note que, quando não for indicado a forma de ordenação, será considerada a forma ascendente como padrão. Além disso, você pode ordenar os dados com base em mais de uma coluna. Quando isso é feito, a primeira coluna especificada é usada para a ordenação principal, e as colunas subsequentes são usadas como critérios secundários, caso haja empates na primeira coluna.



Por fim, percebam que várias dessas cláusulas são opcionais, logo é possível utilizar o ORDER BY apenas com SELECT e FROM. É possível também representar a coluna responsável pela ordenação por meio de um número que indique a ordem da coluna (Ex: 1 é primeira coluna, 2 é segunda coluna, 3 é terceira coluna, entre outros). Agora vejamos um exemplo:

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
ORDER BY 6,1;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00

Note que ele ordenou pela coluna #6 (VALOR_PAGO), mas - como houve um empate de VALOR_PAGO de 50.00 - ele ordenou pela coluna #1 (NOME).

LIMIT

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Essa cláusula é usada para restringir a quantidade de registros retornados por uma consulta, sendo útil quando se deseja visualizar apenas as primeiras linhas do resultado, como os 5 primeiros alunos cadastrados. Caso o número total de registros seja menor que o limite definido, o banco retorna apenas os registros disponíveis, sem gerar erro.

É importante observar que LIMIT não faz parte do padrão SQL oficial. Seu uso varia conforme o SGBD: no MySQL e PostgreSQL utiliza-se LIMIT; no SQL Server, o equivalente é a cláusula TOP; e no Oracle, utiliza-se ROWNUM. Apesar das diferenças de sintaxe, todos esses mecanismos servem ao mesmo propósito: limitar a quantidade de dados retornados pela consulta. Em seguida, será apresentada a aplicação prática dessa cláusula.

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS  
FROM NOME_DA_TABELA  
LIMIT QTD_LINHAS;
```

EXEMPLO DO COMANDO

```
SELECT *  
FROM ALUNO_ESTRATEGIA  
LIMIT 2;
```



RESULTADO DO COMANDO

RESULTADO					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00

Por fim, vamos falar de um operador chamado **UNION**! Não se trata de uma cláusula, trata-se de um operador utilizado para combinar os resultados de duas ou mais instruções **SELECT**. Para que funcione, cada uma dessas instruções deve conter o mesmo número de colunas, sendo que as colunas devem ter tipos de dados semelhantes - além de ter a mesma ordem. Vamos ver como é a sua sintaxe:

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS  
FROM NOME_DA_TABELA1
```

UNION

```
SELECT LISTA_DE_COLUNAS  
FROM NOME_DA_TABELA2;
```

Esse comando elimina eventuais linhas duplicadas. Caso se queira permitir linhas duplicadas, utiliza-se a instrução **UNION ALL**:

SINTAXE DO COMANDO

```
SELECT LISTA_DE_COLUNAS  
FROM NOME_DA_TABELA1
```

UNION ALL

```
SELECT LISTA_DE_COLUNAS  
FROM NOME_DA_TABELA2;
```

O resultado é muito simples: esse comando basicamente une/junta as linhas das tabelas que compõem a união. Se tinham 10 linhas em cada tabela, teremos 20 (exceto se houver duplicatas).

IMPORTANTE



CLÁSULAS	DESCRIÇÃO
FROM	Comando utilizado para indicar de onde os dados devem ser selecionados.
JOIN	Comando utilizado para combinar linhas tabelas, com base em uma coluna em comum entre elas.
WHERE	Comando utilizado para filtrar os dados.
GROUP BY	Comando utilizado para agregar um conjunto de dados.
HAVING	Comando utilizado para filtrar dados agregados.

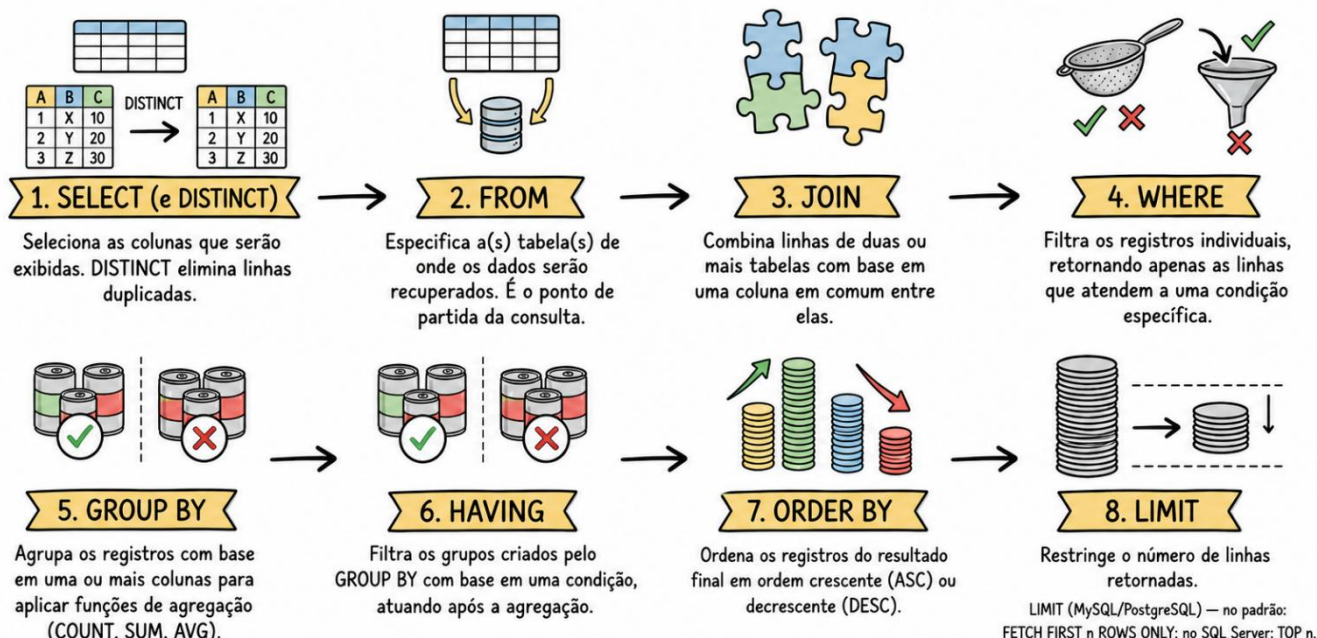


ORDER BY Comando utilizado para ordenar os dados recuperados.

LIMIT Comando utilizado para limitar a quantidade de resultados.

A única cláusula obrigatória do SELECT é o FROM – todas as outras são opcionais! A única ressalva é que elas devem vir na ordem acima e, se houver um HAVING, antes deve existir um GROUP BY.

A Anatomia de uma Consulta SQL



TCL (TRANSACTION CONTROL LANGUAGE)

Comandos TCL

TCL (TRANSACTION CONTROL LANGUAGE)

A TCL (Transaction Control Language) é responsável pelo controle de transações, garantindo que operações no banco de dados sejam executadas de forma segura, consistente e recuperável. Por meio de seus comandos, ela permite confirmar alterações permanentes, desfazê-las em caso de erros ou salvar pontos intermediários dentro de uma transação. Assim, ela assegura integridade e confiabilidade, principalmente em sistemas que exigem precisão e atomicidade nas operações, como aplicações financeiras, bancárias e corporativas.

O TCL (Transaction Control Language) é um subconjunto do SQL usado para controlar transações dentro do banco de dados. Ele é essencial para garantir que operações mais complexas, que envolvem várias etapas, sejam tratadas como uma única unidade de trabalho – tudo ou nada. Transações são especialmente importantes quando precisamos realizar várias inserções, atualizações ou deleções de uma vez, mantendo a consistência dos dados, mesmo que algo dê errado no meio do caminho.

O objetivo do TCL é garantir que: se tudo der certo, a transação seja confirmada (committed); se algo falhar, a transação seja revertida (rolled back) – como se nada tivesse acontecido. Suas características são:

- **Atômica:** uma transação deve ser executada por completo ou totalmente desfeita. Não existe “meio-termo”.



- **Isolada:** cada transação precisa ser executada sem interferência de outras transações em andamento, garantindo independência.
- **Persistente:** uma vez concluída com sucesso, a transação grava as alterações de forma permanente no banco de dados.

Esses princípios são fundamentais para manter a confiabilidade e integridade dos dados em qualquer sistema que use banco de dados relacional.

COMANDOS TCL	DESCRIÇÃO
COMMIT	Comando utilizado para finalizar/confirmar uma transação dentro de um SGBD.
ROLLBACK	Comando utilizado para descartar mudanças nos dados desde o último COMMIT ou ROLLBACK.

Há ainda os comandos SAVEPOINT e SET TRANSACTION, mas nós não vamos entrar em detalhes sobre eles porque eles raramente caem em prova.

COMMIT

INCIDÊNCIA EM PROVA: BAIXA

Esse comando é utilizado para confirmar e salvar permanentemente no banco de dados todas as alterações feitas por comandos como INSERT, UPDATE ou DELETE. Essas alterações, inicialmente, ficam em um estado temporário, aguardando a confirmação final. Ao executar o COMMIT, você informa ao banco que todas as modificações devem ser registradas de forma definitiva.

Esse comando garante que todas as ações realizadas desde o último COMMIT ou ROLLBACK sejam aplicadas de forma permanente, encerrando a transação atual. Um exemplo prático seria uma transferência bancária: você insere os dados, visualiza o resumo e, ao confirmar, a operação é efetivada – esse momento corresponde ao COMMIT no banco de dados. Sem ele, nenhuma alteração é de fato salva.

SINTAXE DO COMANDO

COMMIT;

ESTADO INICIAL

ALUNO_ESTRATEGIA					
NOME	CPE	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00

TRANSAÇÃO DML



```
INSERT INTO ALUNO_ESTRATEGIA
VALUES ('ZICO', 12345678901, 'ZICO@ZICO.COM', '03-03-1953', 'RIO DE JANEIRO', 500.00);
```

ESTADO INTERMEDIÁRIO

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
ZICO	12345678901	ZICO@ZICO.COM	03-03-1953	RIO DE JANEIRO	500.00

CONFIRMAÇÃO DA TRANSAÇÃO

```
COMMIT;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
ZICO	12345678901	ZICO@ZICO.COM	03-03-1953	RIO DE JANEIRO	500.00

ROLLBACK

INCIDÊNCIA EM PROVA: BAIXA

O comando ROLLBACK é utilizado para desfazer alterações realizadas em uma transação, retornando o banco ao estado anterior ao início da transação ou até um ponto de salvamento (SAVEPOINT), se definido. Ele opera com base no log de transações do SGBD, sem necessidade de backups para realizar a reversão. Esse comando é útil em casos em que foram feitas inserções, atualizações ou exclusões, mas se identificou algum erro ou foi necessário cancelar a operação.

O ROLLBACK anula todas as modificações feitas desde o último COMMIT ou ROLLBACK, funcionando como um "botão de desfazer", preservando a integridade e a consistência do banco.

SINTAXE DO COMANDO

```
ROLLBACK;
```



ESTADO INICIAL

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
ZICO	12345678901	ZICO@ZICO.COM	03-03-1953	RIO DE JANEIRO	500.00

TRANSAÇÃO DML

```
UPDATE ALUNO_ESTRATEGIA  
SET NOME = 'ARTHUR'  
WHERE CPF = 12345678901;
```

ESTADO INTERMEDIÁRIO

ALUNO_ESTRATEGIA					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
ARTHUR	12345678901	ZICO@ZICO.COM	03-03-1953	RIO DE JANEIRO	500.00

CANCELAMENTO DA TRANSAÇÃO

```
ROLLBACK;
```

RESULTADO DO COMANDO

RESULTADO					
NOME	CPF	EMAIL	DATA_NASCIMENTO	CIDADE	VALOR_PAGO
ALICE	11111111111	ALICE@ALICE.COM	01-01-2001	BRASÍLIA	200.00
BRUNO	22222222222	BRUNO@BRUNO.COM	02-02-2002	SÃO PAULO	100.00
CAIO	33333333333	CAIO@CAIO.COM	03-03-2003	GOIÂNIA	150.00
DIOGO	44444444444	DIOGO@DIOGO.COM	04-04-2004	SALVADOR	250.00
ELIS	55555555555	ELIS@ELIS.COM	05-05-2005	BRASÍLIA	50.00
FABIO	66666666666	FABIO@FABIO.COM	06-06-2006	SALVADOR	125.00
GABI	77777777777	GABI@GABI.COM	07-07-2007	BRASÍLIA	225.00
HUGO	88888888888	HUGO@HUGO.COM	08-08-2008	BRASÍLIA	50.00
ZICO	12345678901	ZICO@ZICO.COM	03-03-1953	RIO DE JANEIRO	500.00



TCL: O Controle de Transações no SQL

A TCL (Transaction Control Language) gerencia transações no SQL, tratando um conjunto de operações como uma única unidade de trabalho ("tudo ou nada"). Garante a integridade da transação: em caso de sucesso, as alterações são confirmadas (COMMIT); em caso de erro são revertidas (ROLLBACK).

COMMIT: CONFIRMANDO A

TRANSAÇÃO



Salva permanentemente todas as alterações.

Finaliza a transação, tornando as mudanças definitivas no banco de dados.

Exemplo: Inserir um novo aluno

ID	Nome
101	Maria Silva
102	Ana Costa

O COMMIT salva permanentemente as alterações.

Novo registro confirmado e salvo no banco de dados.

ROLLBACK: DESFAZENDO

A TRANSAÇÃO



Descarta todas as alterações recentes.

Reverte a transação, retornando o banco de dados ao seu último estado salvo.

Exemplo: Atualizar um nome por engano

ANTES		DURANTE A TRANSAÇÃO		APÓS O ROLLBACK	
ID	Nome	ID	Nome	ID	Nome
101	João Souza	101	João Pereira	101	João Souza

DESFAZER

O ROLLBACK descarta as alterações e retorna ao último estado salvo.

DCL (DATA CONTROL LANGUAGE)

Comandos DCL

DCL (DATA CONTROL LANGUAGE)

A DCL (Data Control Language) é a parte do SQL responsável pela administração de permissões e controle de acesso aos dados dentro do banco. Por meio de seus comandos, ela define quais usuários ou roles podem consultar, inserir, alterar ou excluir informações, garantindo segurança e conformidade com regras organizacionais. Ela é essencial para proteger dados sensíveis, estabelecer diferentes níveis de privilégio e assegurar que apenas usuários autorizados executem operações específicas, contribuindo diretamente para a política de governança e segurança da informação no ambiente de banco de dados.

O DCL (Data Control Language) é um subconjunto do SQL voltado para o controle de acesso e segurança dentro do banco de dados. Com os comandos da DCL, é possível definir quem pode fazer o quê, ou seja, você controla quais usuários têm permissão para acessar, modificar ou visualizar determinados dados – e em que nível. Essa parte da linguagem é essencial para proteger as informações armazenadas, garantindo que somente pessoas autorizadas possam realizar certas ações. Em sistemas com muitos usuários, o DCL é fundamental para manter a segurança e integridade dos dados.

COMANDOS DCL	DESCRIÇÃO
GRANT	Comando utilizado para conceder permissão a um usuário em relação a algum objeto.
REVOKE	Comando utilizado para remover/restringir a capacidade de um usuário de executar operações.

GRANT

INCIDÊNCIA EM PROVA: BAIXA



O comando GRANT é usado para conceder permissões a usuários ou grupos de usuários no banco de dados. Essas permissões determinam o que cada usuário pode ou não fazer, como por exemplo: ler dados (SELECT), inserir novos registros (INSERT), atualizar informações (UPDATE), excluir registros (DELETE), e outras ações relacionadas à manipulação e controle de objetos no banco.

No total, existem nove tipos principais de permissões que podem ser concedidas com o GRANT: SELECT, INSERT, UPDATE, DELETE, REFERENCES, USAGE, UNDER, TRIGGER, EXECUTE. Esse comando é essencial para garantir que cada usuário tenha exatamente o nível de acesso que precisa – nada a mais, nada a menos.

SINTAXE DO COMANDO

```
GRANT LISTA_DE_PRIVILEGIOS ON OBJETO TO LISTA_DE_USUARIOS;
```

EXEMPLO DO COMANDO

```
GRANT SELECT      ON ALUNO_ESTRATEGIA TO PROFESSOR;  
GRANT INSERT      ON ALUNO_ESTRATEGIA TO GERENTE;  
GRANT UPDATE      ON ALUNO_ESTRATEGIA TO HEBER;  
GRANT DELETE      ON ALUNO_ESTRATEGIA TO RICARDO;  
GRANT REFERENCES  ON ALUNO_ESTRATEGIA TO DUDU;
```

RESULTADO DO COMANDO

--TODOS OS COMANDOS GARANTEM ALGUM TIPO DE PERMISSÃO À TABELA ALUNO_ESTRATEGIA

```
SELECT    - PERMITE QUE PROFESSOR CONSULTE DADOS;  
INSERT    - PERMITE QUE GERENTE INSIRA DADOS;  
UPDATE    - PERMITE QUE HEBER MODIFIQUE DADOS;  
DELETE    - PERMITE QUE RICARDO DELETE DADOS;  
REFERENCES - PERMITE QUE DUDU REFERENCIE OUTRA TABELA;
```

REVOKE

INCIDÊNCIA EM PROVA: BAIXA

Comando usado para revogar permissões a usuários em relação a objetos. Há nove funções: SELECT, INSERT, UPDATE, DELETE, REFERENCES, USAGE, UNDER, TRIGGER e EXECUTE.

SINTAXE DO COMANDO

```
REVOKE LISTA_DE_PRIVILEGIOS ON OBJETO FROM LISTA_DE_USUARIOS;
```

EXEMPLO DO COMANDO

```
REVOKE SELECT      ON ALUNO_ESTRATEGIA FROM PROFESSOR;  
REVOKE INSERT      ON ALUNO_ESTRATEGIA FROM GERENTE;  
REVOKE UPDATE      ON ALUNO_ESTRATEGIA FROM HEBER;  
REVOKE DELETE      ON ALUNO_ESTRATEGIA FROM RICARDO;  
REVOKE REFERENCES  ON ALUNO_ESTRATEGIA FROM DUDU;
```

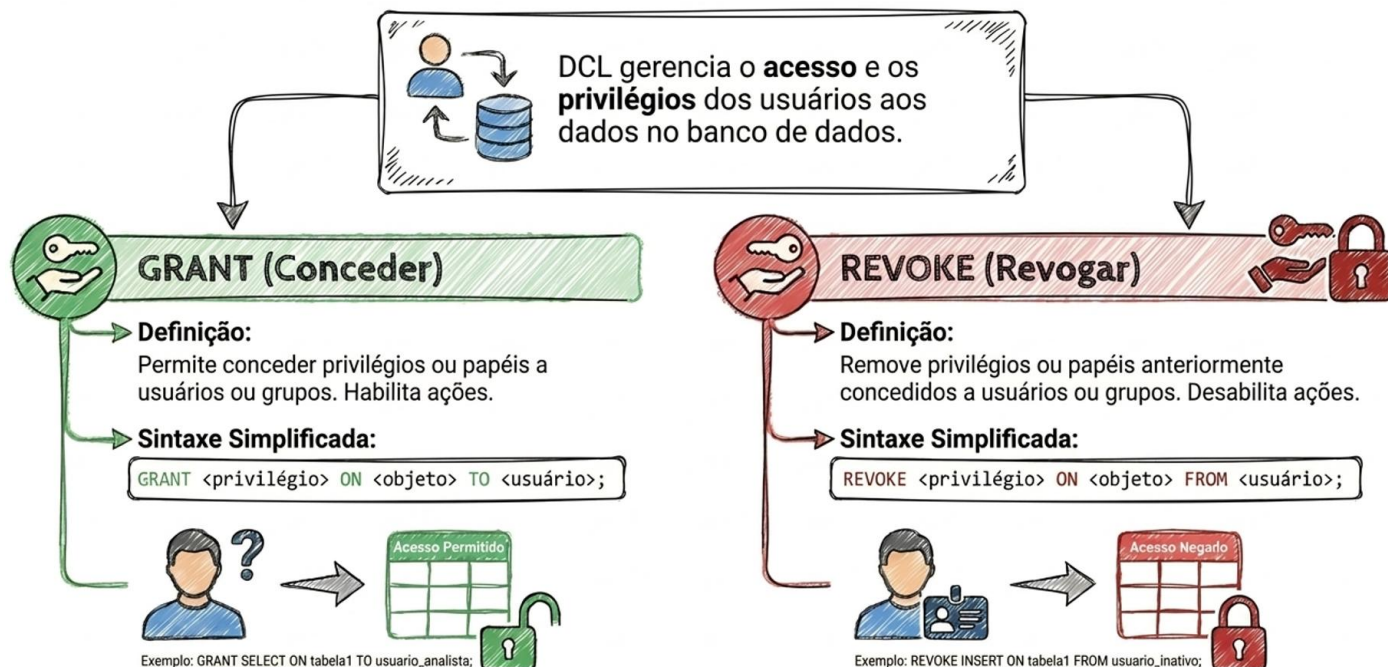
RESULTADO DO COMANDO



//TODOS OS COMANDOS REVOGAM ALGUM TIPO DE PERMISSÃO À TABELA ALUNO ESTRATEGIA

SELECT	- REVOGA A PERMISSÃO DE QUE PROFESSOR CONSULTE DADOS;
INSERT	- REVOGA A PERMISSÃO DE QUE GERENTE INSIRA DADOS;
UPDATE	- REVOGA A PERMISSÃO DE QUE HEBER MODIFIQUE DADOS;
DELETE	- REVOGA A PERMISSÃO DE QUE RICARDO DELETE DADOS;
REFERENCES	- REVOGA A PERMISSÃO DE QUE DUDU REFERENCIE OUTRA TABELA;

DCL: Linguagem de Controle de Dados (Data Control Language)



QUESTÕES COMENTADAS

departamento	
id	nome_departamento
1	TI
2	RH
3	marketing

funcionario				
id	nome	id_departamento	salario	data_admissao
1	João Oliveira	1	4500.00	2022-05-10
2	Maria Silva	2	4000.00	2024-11-18
3	Carlos Andrade	1	5000.00	2021-08-15
4	Ana Costa	3	3500.00	2023-03-01

1. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

A execução da seguinte consulta SQL retornará o nome dos funcionários e seus respectivos departamentos e salários.

```
SELECT f.nome, d.nome_departamento, f.salario  
FROM funcionarios f INNER JOIN departamentos d  
ON f.id_departamento = d.id;
```

Comentários:

A consulta SQL apresentada realiza um INNER JOIN entre as tabelas funcionarios (f) e departamentos (d) com base na chave estrangeira f.id_departamento, que referencia a chave primária d.id da tabela de departamentos. O INNER JOIN retorna apenas os registros em que há correspondência entre as duas tabelas. A consulta seleciona:

- f.nome → Nome do funcionário;
- d.nome_departamento → Nome do departamento ao qual o funcionário pertence;
- f.salario → Salário do funcionário.

Logo, a consulta realmente retorna o nome dos funcionários, seus respectivos departamentos e salários. A questão foi anulada porque o nome da tabela é "funcionário" e, não, "funcionários".

Gabarito: Anulada

2. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

A execução da seguinte consulta SQL resultará uma lista com a média de salários de cada departamento.

```
SELECT d.nome_departamento, AVG(f.salario) AS  
salario_medio  
FROM funcionarios f  
INNER JOIN departamentos d  
ON f.id_departamento = d.id  
WHERE salario_medio > 4000.00  
GROUP BY d.nome_departamento;
```

Comentários:

A consulta tenta calcular a média de salários por departamento usando a função de agregação AVG(f.salario) e agrupando pelos departamentos (GROUP BY d.nome_departamento). No entanto, há um erro na cláusula WHERE.

A cláusula WHERE salario_medio > 4000.00 está errada, porque salario_medio é um alias (apelido) definido na consulta, mas ele só está disponível após a agregação. Em SQL, quando se deseja filtrar os resultados após o cálculo de agregação, deve-se usar HAVING em vez de WHERE. O correto seria:

```
SELECT d.nome_departamento, AVG(f.salario) AS salario_medio  
FROM funcionarios f  
INNER JOIN departamentos d  
ON f.id_departamento = d.id  
GROUP BY d.nome_departamento  
HAVING AVG(f.salario) > 4000.00;
```

Dessa forma, a consulta retornará apenas os departamentos cuja média salarial seja superior a 4000.00. A questão foi anulada porque o nome da tabela é "funcionário" e não, "funcionários".

Gabarito: Anulada

3. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

O resultado da seguinte consulta SQL será uma lista com os departamentos cadastrados e a quantidade de empregados de cada um deles.

```
SELECT d.nome_departamento AS departamento,  
COUNT(f.id) AS total_employees  
FROM departamentos d  
LEFT JOIN funcionarios f  
ON d.id = f.id_departamento  
GROUP BY d.id, d.nome_departamento;
```

Comentários:

A consulta SQL faz um LEFT JOIN entre as tabelas departamentos (d) e funcionarios (f) e conta a quantidade de funcionários em cada departamento. Vamos analisar sua estrutura:

- LEFT JOIN: garante que todos os departamentos sejam retornados, mesmo aqueles sem funcionários (neste caso, o COUNT(f.id) retornará 0 para esses departamentos);
- COUNT(f.id): conta os funcionários em cada departamento;
- GROUP BY d.id, d.nome_departamento: agrupa os resultados por departamento, garantindo que cada departamento apareça apenas uma vez na saída.

A consulta retorna: a lista de todos os departamentos (por causa do LEFT JOIN) e a quantidade de empregados em cada departamento (COUNT(f.id)). A questão foi anulada porque o nome da tabela é "funcionário" e, não, "funcionários".

Gabarito: Anulada

4. (CEBRASPE / EMBRAPA - 2025) No que concerne a linguagem SQL e banco de dados NoSQL, julgue o item a seguir.

A execução da instrução SQL a seguir criará uma tabela employees com uma chave primária composta pelas colunas employee_id e department_id.

```
CREATE TABLE employees (  
    employee_id INT NOT NULL,  
    department_id INT NOT NULL,  
    name VARCHAR(100),  
    PRIMARY KEY (employee_id,  
    department_id)  
);
```

Comentários:

A instrução SQL apresentada define apenas uma chave primária na coluna `employee_id`. Para que a chave primária fosse composta, seria necessário incluir ambas as colunas (`employee_id` e `department_id`) na definição da chave primária.

Gabarito: Errado

5. (CEBRASPE / EMBRAPA - 2025) No que concerne a linguagem SQL e banco de dados NoSQL, julgue o item a seguir.

A consulta a seguir retorna todos os clientes que tenham realizado pelo menos um pedido na tabela `orders`, utilizando o comando `INNER JOIN` para garantir que apenas clientes com pedidos sejam incluídos no resultado.

```
SELECT
    customers.customer_id,
    customers.name
FROM
    customers
INNER JOIN
    orders
ON customers.customer_id = orders.customer_id;
```

Comentários:

A consulta SQL apresentada utiliza o `INNER JOIN` corretamente para combinar as tabelas "customers" e "orders", garantindo que apenas os clientes que possuem pedidos sejam retornados. Assim, a consulta está bem estruturada e atende ao que foi solicitado.

Gabarito: Correto

6. (CEBRASPE / EMBRAPA - 2025) Julgue o item subsequente, relativo a sistemas de banco de dados.

No SQL, a instrução `DELETE` pode ser usada para remover colunas inteiras de uma tabela e manter o restante dos dados sem alteração.

Comentários:

A instrução `DELETE` no SQL é utilizada para remover linhas de uma tabela, não colunas. Para remover colunas, utiliza-se a instrução `ALTER TABLE` com `DROP COLUMN`, que altera a estrutura da tabela.

Gabarito: Errado

7. (CEBRASPE / BDMG - 2025) Julgue o próximo item, relativos a visualização de dados, banco de dados e BI (business intelligence).

Índices em colunas frequentemente usadas em cláusulas WHERE podem melhorar significativamente a performance de consultas SQL em bancos de dados relacionais.

Comentários:

A utilização de índices em colunas frequentemente acessadas nas cláusulas WHERE permite que o banco de dados localize rapidamente as linhas relevantes, reduzindo o tempo de execução das consultas e melhorando a performance geral.

Gabarito: Correto

8. (CEBRASPE / BDMG - 2025) No que se refere à administração de banco de dados, julgue o item que se segue.

Uma instância de um banco de dados SQL compreende a organização lógica dos dados com seus atributos e seus domínios correspondentes.

Comentários:

A instância de um banco de dados SQL refere-se ao ambiente de execução do sistema de gerenciamento de banco de dados (SGBD), e não à organização lógica dos dados. A organização lógica é representada pelo esquema do banco de dados.

Gabarito: Errado

9. (CEBRASPE / ANM - 2025) Julgue o item a seguir, relativo às tecnologias CSV e JSON, à Linguagem SQL e ao modelo CRISP-DM.

Para excluir uma tabela na linguagem SQL e liberar os recursos alocados por ela, inclusive índices e triggers associados, deve-se fazer uso do comando DELETE TABLE, pois ele remove tanto os dados quanto a estrutura da tabela.

Comentários:

O comando correto para excluir uma tabela e liberar os recursos alocados é o comando DROP TABLE, e não DELETE TABLE. O comando DELETE é utilizado para remover dados de uma tabela, mas não a estrutura dela.

Gabarito: Errado

10.(CEBRASPE / ANM - 2025) Julgue o item a seguir, relativo às tecnologias CSV e JSON, à Linguagem SQL e ao modelo CRISP-DM.

A cláusula GROUP BY, do SQL, pode ser utilizada sem a presença de funções agregadas, desde que haja uma cláusula ORDER BY para organizar os resultados da consulta.

Comentários:

A cláusula GROUP BY é utilizada para agrupar resultados com base em uma ou mais colunas, e sua função principal é trabalhar em conjunto com funções agregadas. Sem funções agregadas, o uso do GROUP BY não faz sentido, mesmo com ORDER BY.

Gabarito: Errado

11.(CEBRASPE / ANM - 2025) A respeito de banco de dados, julgue o próximo item.

Em SQL, a instrução mais simples para selecionar registros de uma tabela é composta pelas cláusulas SELECT e WHERE.

Comentários:

A instrução mais simples para selecionar registros em SQL é composta apenas pela cláusula SELECT, sem a necessidade da cláusula WHERE, que é utilizada para filtrar resultados.

Gabarito: Errado

12.(CEBRASPE / SEFAZ-SE - 2025) A tabela VENDAS tem a seguinte estrutura simplificada.

```
VENDAS(  
    ID_VENDA    INT,  
    ID_CLIENTE  INT,  
    VALOR       DECIMAL(10,2),  
    DATA_VENDA DATE  
);
```

Dessa tabela, deseja-se obter, para cada cliente, o total de vendas realizadas e, em seguida, apresentar o ranking desses clientes, ordenado do maior para o menor valor total. A partir das informações precedentes, assinale a opção na qual é apresentada a consulta SQL que atenderá ao desejado.

a)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
ORDER BY ID_CLIENTE;

b)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL
FROM VENDAS
ORDER BY SUM(VALOR) DESC;

c)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
GROUP BY ID_CLIENTE
ORDER BY TOTAL DESC;

d)
SELECT ID_CLIENTE,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
ORDER BY POSICAO;

e)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY TOTAL DESC) AS POSICAO
FROM VENDAS
ORDER BY TOTAL DESC;

Comentários:

(a) Errado. A consulta tenta usar RANK() com SUM(VALOR) sem utilizar GROUP BY, o que causa erro, pois SUM(VALOR) é uma função agregadora aplicada sem agregação prévia por cliente;

(b) Errado. A consulta realiza corretamente a soma por cliente, mas não inclui o ranking, que é parte essencial do solicitado no enunciado;

(c) Correto. A consulta agrupa corretamente por cliente, soma os valores e utiliza a função RANK() com OVER(ORDER BY SUM(VALOR) DESC) para gerar o ranking, conforme desejado;

(d) Errado. O ranking está sendo calculado com base em SUM(VALOR) mas sem o GROUP BY, o que gera erro de agregação, pois SUM está sendo usado diretamente na cláusula OVER sem agregação por cliente;

(e) Errado. A função RANK() está sendo usada com ORDER BY TOTAL, mas TOTAL é um alias definido na própria seleção. Isso não é permitido dentro da função RANK(), pois o alias ainda não está disponível no momento do cálculo.

Gabarito: Letra C

```
CREATE TABLE empresas (  
    id_empresa INT PRIMARY KEY,  
    nome_empresa VARCHAR(100),  
    cnpj VARCHAR(18)  
);
```

```
CREATE TABLE auditorias (  
    id_auditoria INT PRIMARY KEY,  
    id_empresa INT,  
    data_auditoria DATE,  
    FOREIGN KEY (id_empresa)  
        REFERENCES empresas(id_empresa)  
);
```

13.(CEBRASPE / SEFAZ-SE - 2025) A partir das tabelas precedentes, referentes a empresas em que foram realizadas auditorias, assinale a opção em que a consulta SQL apresentada retorna corretamente a lista de empresas que foram objeto de nenhuma ou de apenas uma auditoria no ano de 2025.

a)
SELECT e.id_empresa, e.nome_empresa
FROM empresas e
LEFT JOIN auditorias a
 ON e.id_empresa = a.id_empresa
WHERE a.data_auditoria >= '2025-01-01'
 AND a.data_auditoria <= '2025-12-31'
GROUP BY e.id_empresa, e.nome_empresa
HAVING COUNT(*) <= 1;

b)
SELECT e.id_empresa, e.nome_empresa
FROM empresas e
LEFT JOIN auditorias a

```
ON e.id_empresa = a.id_empresa  
AND a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY 1, 2  
HAVING COUNT(a.id_auditoria) <= 1;
```

c) (observação: o original tinha erros como "id empresa" sem underline)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
INNER JOIN auditorias a  
ON e.id_empresa = a.id_empresa  
AND a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

d)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
ON e.id_empresa = a.id_empresa  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

e)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
ON e.id_empresa = a.id_empresa  
WHERE a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

Comentários:

(a) Errado. A cláusula WHERE filtra pelas datas de auditoria após o LEFT JOIN, eliminando os registros NULL, o que impede a seleção de empresas sem auditorias;

(b) Correto. A condição de data é aplicada na cláusula JOIN, o que mantém os registros NULL resultantes do LEFT JOIN, permitindo identificar corretamente empresas com 0 ou 1 auditoria em 2025;

(c) Errado. Usa INNER JOIN, o que exclui empresas que não tenham auditoria em 2025, contrariando o requisito de incluir empresas com nenhuma auditoria no ano;

(d) Errado. A ausência de filtro de data faz com que sejam consideradas auditorias de todos os anos, não apenas de 2025, o que foge ao escopo da questão;

(e) Errado. Assim como na alternativa (a), a cláusula WHERE elimina os NULLs do LEFT JOIN, removendo da seleção as empresas sem auditorias no período.

Gabarito: Letra B

14.(CEBRASPE / Polícia Federal - 2025) Julgue o itens subsecutivo, a respeito de bancos de dados.

Em SQL, as junções externas (LEFT OUTER JOIN, RIGHT OUTER JOIN e FULL OUTER JOIN) permitem incluir registros de uma ou de ambas as tabelas, mesmo quando não há correspondência entre as chaves de junção.

Comentários:

As junções externas (LEFT, RIGHT e FULL OUTER JOIN) permitem combinar registros mesmo sem correspondência entre as chaves. O LEFT OUTER JOIN inclui todos os registros da tabela da esquerda; o RIGHT, da direita; e o FULL, de ambas, preenchendo com NULL os dados ausentes da tabela oposta.

Gabarito: Correto

15.(CEBRASPE / Polícia Federal - 2025) Julgue o item subsequente, relativos à teoria da informação e a banco de dados.

No contexto de banco de dados, dados estruturados são aqueles que não possuem um formato fixo e são armazenados em sistemas como bancos NoSQL, ao passo que dados não estruturados possuem um formato rígido e são armazenados em tabelas relacionais.

Comentários:

Dados estruturados possuem um formato fixo e são armazenados em bancos de dados relacionais, enquanto dados não estruturados não têm um formato rígido e são frequentemente armazenados em bancos NoSQL.

Gabarito: Errado

16.(CEBRASPE / SEFAZ-RJ - 2025) A principal função da cláusula WITH do SQL é

- a) agrupar registros com base em valores comuns em uma ou mais colunas.
- b) combinar registros de duas ou mais tabelas com base em colunas relacionadas.
- c) filtrar registros com base em condições específicas.

- d) ordenar o resultado de uma consulta por uma ou mais colunas.
- e) definir blocos de consulta nomeados que podem ser referenciados múltiplas vezes.

Comentários:

- (a) Errado. A cláusula GROUP BY é a responsável por agrupar registros com base em valores comuns em colunas, não a cláusula WITH.
- (b) Errado. A cláusula JOIN é utilizada para combinar registros de tabelas relacionadas, enquanto a cláusula WITH não tem essa função.
- (c) Errado. A cláusula WHERE é a que filtra registros com base em condições específicas, não a cláusula WITH.
- (d) Errado. A cláusula ORDER BY é utilizada para ordenar resultados de consultas, enquanto a cláusula WITH serve para definir blocos de consulta.
- (e) Correto. A cláusula WITH permite definir subconsultas nomeadas que podem ser referenciadas várias vezes na consulta principal, facilitando a leitura e manutenção do código.

Gabarito: Letra E

17.(CEBRASPE / FUB - 2025) Julgue o seguinte item, que versa sobre administração de banco de dados.

Considere que os comandos SQL a seguir tenham sido executados de forma bem-sucedida.

```
CREATE TABLE TABELA_TESTE (a integer, b varchar);  
INSERT INTO TABELA_TESTE (a) VALUES (5);  
INSERT INTO TABELA_TESTE (b) VALUES ('FUB');  
INSERT INTO TABELA_TESTE (a) VALUES (7);
```

Nessa situação, o resultado da execução da consulta `SELECT COUNT(a), AVG(a) FROM TABELA_TESTE;` será o apresentado a seguir.

count	avg
3	4

Comentários:

A tabela possui 3 registros, mas apenas duas linhas com valores na coluna a: 5 e 7. A função COUNT(a) não conta valores nulos, portanto retorna 2, e AVG(a) calcula a média desses dois valores: $(5 + 7) / 2 = 6$. O resultado exibido (count = 3, avg = 4) está incorreto.

Gabarito: Errado

18.(CEBRASPE / FUB - 2025) Julgue o seguinte item, que versa sobre administração de banco de dados.

A cláusula HAVING opera em um único conjunto de dados de entrada, sem agregação.

Comentários:

Essa cláusula é utilizada em consultas SQL para filtrar resultados de funções de agregação, como SUM ou COUNT, após a execução do GROUP BY. Logo, ela opera em conjuntos de dados que já foram agregados.

Gabarito: Errado

19.(CEBRASPE / BANRISUL - 2025) Assinale a opção em que é corretamente apresentado o comando DDL que permite criar a tabela de nome itens_pedido e relacionar cada pedido a seus produtos, de modo que: pedido_id e produto_id sejam chaves estrangeiras que referenciem a tabela; quantidade seja um inteiro maior que zero; preco_unitario seja um decimal maior que zero; e a combinação (pedido_id, produto_id) seja a chave primária, garantindo unicidade e não null.

a)

```
CREATE TABLE itens_pedido (  
    pedido_id INTEGER REFERENCES pedidos(id),  
    produto_id INTEGER REFERENCES produtos(id),  
    quantidade INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    UNIQUE (pedido_id, produto_id)  
);
```

b)

```
CREATE itens_pedido (  
    pedido_id INTEGER REFERENCES pedidos(id),  
    produto_id INTEGER REFERENCES produtos(id),  
    quantidade INTEGER CHECK quantidade > 0,  
    preco_unitario DECIMAL(10,2) CHECK preco_unitario > 0,
```

```
PRIMARY KEY (pedido_id, produto_id)  
);
```

c)

```
CREATE TABLE itens_pedido (  
    pedido_id INTEGER,  
    produto_id INTEGER,  
    quantidade INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    FOREIGN KEY (pedido_id) pedidos(id),  
    FOREIGN KEY (produto_id) produtos(id),  
    PRIMARY KEY (pedido_id, produto_id)  
);
```

d)

```
CREATE TABLE itens_pedido (  
    pedido_id INTEGER REFERENCES pedidos(id),  
    produto_id INTEGER REFERENCES produtos(id),  
    quantidade INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    PRIMARY KEY (pedido_id, produto_id)  
);
```

e)

```
CREATE TABLE itens_pedido (  
    item_id INTEGER PRIMARY KEY,  
    pedido_id INTEGER REFERENCES pedidos(id),  
    produto_id INTEGER REFERENCES produtos(id),  
    quantidade INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK  
(preco_unitario > 0),  
    UNIQUE (pedido_id, produto_id)  
);
```

Comentários:

(a) Errado. UNIQUE (pedido_id, produto_id) garante unicidade, mas não define essa combinação como chave primária. A questão exige expressamente PRIMARY KEY, que também impõe NOT NULL;

- (b) Errado. A sintaxe está incorreta. O comando deveria iniciar com CREATE TABLE, e as restrições CHECK devem ser escritas com a expressão entre parênteses, como CHECK (quantidade > 0);
- (c) Errado. As declarações das chaves estrangeiras estão incompletas. O correto seria FOREIGN KEY (pedido_id) REFERENCES pedidos(id) e FOREIGN KEY (produto_id) REFERENCES produtos(id);
- (d) Correto. A sintaxe está adequada: pedido_id e produto_id referenciam as tabelas correspondentes, quantidade e preco_unitario possuem restrições CHECK, e a combinação (pedido_id, produto_id) é definida como chave primária composta;
- (e) Errado. A chave primária foi definida como item_id. A combinação (pedido_id, produto_id) aparece apenas como UNIQUE, contrariando o requisito do enunciado.

Gabarito: Letra D

20.(CEBRASPE / BANRISUL - 2025) Assinale a opção em que são corretamente apresentados os comandos necessários para a criação, em um banco de dados relacional, da tabela de nome funcionários, contendo as colunas: id de inteiros e chave primária; nome com VARCHAR(100), obrigatório; salario com DECIMAL(10,2); e data_admissao de DATE.

a)

```
CREATE funcionarios (  
  id      INTEGER PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  salario DECIMAL(10,2),  
  data_admissao DATE  
);
```

b)

```
INSERT TABLE funcionarios (  
  id      INTEGER PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  salario DECIMAL(10,2),  
  data_admissao DATE  
);
```

c)

```
CREATE TABLE funcionarios (  
  id      INTEGER PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  salario DECIMAL(10,2),  
  data_admissao DATE
```

);

d)

```
DEFINE TABLE funcionarios (  
  id      INTEGER PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL,  
  salario DECIMAL(10,2),  
  data_admissao DATE  
);
```

e)

```
CREATE TABLE funcionarios (  
  id      INTEGER KEY,  
  nome    VARCHAR(100) REQUIRED,  
  salario DECIMAL(10,2),  
  data_admissao DATE  
);
```

Comentários:

(a) Errado. O comando CREATE deve incluir a palavra "TABLE" para definir a criação de uma tabela, portanto, a sintaxe está incorreta.

(b) Errado. O comando INSERT não é utilizado para criar tabelas, mas sim para inserir dados nelas. A sintaxe está completamente equivocada.

(c) Correto. O comando "CREATE TABLE" é a sintaxe correta para criar uma tabela em SQL, e todas as colunas estão definidas corretamente, incluindo a chave primária e a obrigatoriedade do nome.

(d) Errado. O comando "DEFINE" não é uma instrução SQL válida para a criação de tabelas. A sintaxe correta deve ser "CREATE TABLE".

(e) Errado. A palavra "REQUIRED" não é uma palavra-chave válida em SQL; o correto é "NOT NULL" para definir que a coluna é obrigatória.

Gabarito: Letra C

21.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

A criação de uma view que exiba o total de sinistros por cliente é considerada operação de DML, pois envolve a estrutura lógica do banco.

Comentários:

A criação de uma view é uma operação de DDL (Data Definition Language), pois envolve a definição de uma nova estrutura no banco de dados. DML (Data Manipulation Language) refere-se a operações de manipulação de dados, como SELECT, INSERT, UPDATE e DELETE.

Gabarito: Errado

22.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

O comando SELECT nome, idade FROM CLIENTE; pertence à DML, pois consulta os dados armazenados no banco para eventual tomada de decisão.

Comentários:

O comando SELECT pertence à DQL (Data Query Language), que é responsável por consultar dados em um banco de dados. A DML (Data Manipulation Language) inclui comandos que manipulam dados, como INSERT, UPDATE e DELETE.

Gabarito: Errado

23.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

A DDL permite a criação de restrições de integridade referencial entre as tabelas, como, por exemplo, a definição de uma chave estrangeira (foreign key) na tabela SINISTRO referenciando o código da APOLICE.

Comentários:

A DDL (Data Definition Language) é responsável pela definição da estrutura do banco de dados, incluindo a criação de tabelas e a definição de restrições, como chaves estrangeiras, que garantem a integridade referencial entre tabelas.

Gabarito: Correto

24.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

Para recuperar os nomes de todos os beneficiários vinculados a apólices ativas, usa-se a DQL com cláusulas como INNER JOIN e WHERE, classificando-se tal operação como consulta de dados.

Comentários:

A DQL (Data Query Language) é utilizada para consultas em bancos de dados, e a combinação de INNER JOIN e WHERE permite recuperar dados relacionados entre tabelas, como os nomes de beneficiários de apólices ativas.

Gabarito: Correto

25.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

O comando UPDATE pode ser utilizado para alterar a definição de colunas em uma tabela, como o tipo de dado de uma coluna data_nascimento, por exemplo, substituindo VARCHAR por DATE.

Comentários:

O comando UPDATE é utilizado para modificar os dados existentes em uma tabela, não para alterar a definição de colunas. Para isso, deve-se usar o comando ALTER TABLE, que permite modificar a estrutura da tabela, incluindo tipos de dados.

Gabarito: Errado

26.(CEBRASPE / FUNPRES-P-EXE - 2025) Acerca de banco de dados, julgue o próximo item.

Considerando que CIDADANIA seja uma tabela com as colunas NOME_PESSOA, DATA_NASCIMENTO e NOME_PAIS, a execução do comando SQL SELECT NOME_PAIS, COUNT(*) FROM CIDADANIA GROUP BY NOME_PAIS apresentará quantas pessoas nasceram em cada PAIS.

Comentários:

O comando SQL apresentado utiliza a cláusula GROUP BY para agrupar os resultados pela coluna NOME_PAIS, permitindo contar quantas pessoas (COUNT(*)) nasceram em cada país listado na tabela CIDADANIA.

Gabarito: Correto

27.(CEBRASPE / FUNPRES-P-EXE - 2025) Acerca de banco de dados, julgue o próximo item.

O termo CRUD refere-se aos comandos de INSERT, UPDATE, SELECT e DELETE, respectivamente, em um banco de dados.

Comentários:

O uso do termo “respectivamente” estabeleceu uma correspondência com a ordem errada de comandos descrita no enunciado (mapeando Read para UPDATE e Update para SELECT).

Gabarito: Errado

28.(CEBRASPE / FUNPRES-P- 2025) Acerca de banco de dados, julgue o próximo item.

Em um banco de dados, os comandos de SELECT, UPDATE, INSERT e DELETE estão agrupados como DML (data manipulation language).

Comentários:

Questão polêmica! O CEBRASPE considera, em algumas questões, que o SELECT faz parte da DQL (Data Query Language) e, não, da DML.

Gabarito: Errado

29.(CEBRASPE / FUNPRES-P- 2025) Acerca de banco de dados, julgue o próximo item.

DCL (data control language) é responsável pelas tarefas administrativas de controle do próprio banco de dados; em SQL, comandos de GRANT, REVOKE e DENY são exemplos de comandos utilizados para esse fim.

Comentários:

A DCL (Data Control Language) é realmente responsável por controlar o acesso e as permissões em um banco de dados. Os comandos GRANT, REVOKE e DENY são utilizados para conceder, revogar e negar permissões, respectivamente, confirmando a afirmação.

Gabarito: Correto

30.(CEBRASPE / FUNPRES-P- 2025) Acerca de banco de dados, julgue o próximo item.

A execução do comando SELECT * FROM PESSOA LIMIT 5, 10 apresentará 10 registros da tabela PESSOA, a partir do 6.º registro dessa tabela.

Comentários:

O comando LIMIT em SQL, quando utilizado com dois parâmetros, define o número de registros a serem retornados e a partir de qual registro iniciar. Assim, LIMIT 5, 10 retorna 10 registros a partir do 6.º registro, conforme afirmado.

Gabarito: Correto

LISTA DE QUESTÕES

departamento	
id	nome_departamento
1	TI
2	RH
3	marketing

funcionario				
id	nome	id_departamento	salario	data_admissao
1	João Oliveira	1	4500.00	2022-05-10
2	Maria Silva	2	4000.00	2024-11-18
3	Carlos Andrade	1	5000.00	2021-08-15
4	Ana Costa	3	3500.00	2023-03-01

1. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

A execução da seguinte consulta SQL retornará o nome dos funcionários e seus respectivos departamentos e salários.

```
SELECT f.nome, d.nome_departamento, f.salario
FROM funcionarios f INNER JOIN departamentos d
ON f.id_departamento = d.id;
```

2. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

A execução da seguinte consulta SQL resultará uma lista com a média de salários de cada departamento.

```
SELECT d.nome_departamento, AVG(f.salario) AS
salario_medio
FROM funcionarios f
INNER JOIN departamentos d
ON f.id_departamento = d.id
WHERE salario_medio > 4000.00
GROUP BY d.nome_departamento;
```

3. (CEBRASPE / TRF - 6ª REGIÃO - 2025) A partir das tabelas departamento e funcionario precedentes, julgue o item a seguir, relativo a SQL.

O resultado da seguinte consulta SQL será uma lista com os departamentos cadastrados e a quantidade de empregados de cada um deles.

```
SELECT d.nome_departamento AS departamento,  
COUNT(f.id) AS total_empregados  
FROM departamentos d  
LEFT JOIN funcionarios f  
ON d.id = f.id_departamento  
GROUP BY d.id, d.nome_departamento;
```

4. (CEBRASPE / EMBRAPA - 2025) No que concerne a linguagem SQL e banco de dados NoSQL, julgue o item a seguir.

A execução da instrução SQL a seguir criará uma tabela employees com uma chave primária composta pelas colunas employee_id e department_id.

```
CREATE TABLE employees (  
    employee_id INT NOT NULL,  
    department_id INT NOT NULL,  
    name VARCHAR(100),  
    PRIMARY KEY (employee_id)  
);
```

5. (CEBRASPE / EMBRAPA - 2025) No que concerne a linguagem SQL e banco de dados NoSQL, julgue o item a seguir.

A consulta a seguir retorna todos os clientes que tenham realizado pelo menos um pedido na tabela orders, utilizando o comando INNER JOIN para garantir que apenas clientes com pedidos sejam incluídos no resultado.

```
SELECT  
    customers.customer_id,  
    customers.name  
FROM  
    customers  
INNER JOIN  
    orders  
ON customers.customer_id = orders.customer_id;
```

6. (CEBRASPE / EMBRAPA - 2025) Julgue o item subsequente, relativo a sistemas de banco de dados.

No SQL, a instrução DELETE pode ser usada para remover colunas inteiras de uma tabela e manter o restante dos dados sem alteração.

7. (CEBRASPE / BDMG - 2025) Julgue o próximo item, relativos a visualização de dados, banco de dados e BI (business intelligence).

Índices em colunas frequentemente usadas em cláusulas WHERE podem melhorar significativamente a performance de consultas SQL em bancos de dados relacionais.

8. (CEBRASPE / BDMG - 2025) No que se refere à administração de banco de dados, julgue o item que se segue.

Uma instância de um banco de dados SQL compreende a organização lógica dos dados com seus atributos e seus domínios correspondentes.

9. (CEBRASPE / ANM - 2025) Julgue o item a seguir, relativo às tecnologias CSV e JSON, à Linguagem SQL e ao modelo CRISP-DM.

Para excluir uma tabela na linguagem SQL e liberar os recursos alocados por ela, inclusive índices e triggers associados, deve-se fazer uso do comando DELETE TABLE, pois ele remove tanto os dados quanto a estrutura da tabela.

10.(CEBRASPE / ANM - 2025) Julgue o item a seguir, relativo às tecnologias CSV e JSON, à Linguagem SQL e ao modelo CRISP-DM.

A cláusula GROUP BY, do SQL, pode ser utilizada sem a presença de funções agregadas, desde que haja uma cláusula ORDER BY para organizar os resultados da consulta.

11.(CEBRASPE / ANM - 2025) A respeito de banco de dados, julgue o próximo item.

Em SQL, a instrução mais simples para selecionar registros de uma tabela é composta pelas cláusulas SELECT e WHERE.

12.(CEBRASPE / SEFAZ-SE - 2025) A tabela VENDAS tem a seguinte estrutura simplificada.

```
VENDAS(  
    ID_VENDA    INT,  
    ID_CLIENTE  INT,  
    VALOR       DECIMAL(10,2),  
    DATA_VENDA DATE  
);
```

Dessa tabela, deseja-se obter, para cada cliente, o total de vendas realizadas e, em seguida, apresentar o ranking desses clientes, ordenado do maior para o menor valor total. A partir das informações precedentes, assinale a opção na qual é apresentada a consulta SQL que atenderá ao desejado.

a)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
ORDER BY ID_CLIENTE;

b)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL
FROM VENDAS
ORDER BY SUM(VALOR) DESC;

c)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
GROUP BY ID_CLIENTE
ORDER BY TOTAL DESC;

d)
SELECT ID_CLIENTE,
RANK() OVER (ORDER BY SUM(VALOR) DESC) AS POSICAO
FROM VENDAS
ORDER BY POSICAO;

e)
SELECT ID_CLIENTE, SUM(VALOR) AS TOTAL,
RANK() OVER (ORDER BY TOTAL DESC) AS POSICAO
FROM VENDAS
ORDER BY TOTAL DESC;

CREATE TABLE empresas (
id_empresa INT PRIMARY KEY,
nome_empresa VARCHAR(100),
cnpj VARCHAR(18)
);

```
CREATE TABLE auditorias (  
  id_auditoria INT PRIMARY KEY,  
  id_empresa INT,  
  data_auditoria DATE,  
  FOREIGN KEY (id_empresa)  
    REFERENCES empresas(id_empresa)  
);
```

13.(CEBRASPE / SEFAZ-SE - 2025) A partir das tabelas precedentes, referentes a empresas em que foram realizadas auditorias, assinale a opção em que a consulta SQL apresentada retorna corretamente a lista de empresas que foram objeto de nenhuma ou de apenas uma auditoria no ano de 2025.

a)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
  ON e.id_empresa = a.id_empresa  
WHERE a.data_auditoria >= '2025-01-01'  
  AND a.data_auditoria <= '2025-12-31'  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

b)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
  ON e.id_empresa = a.id_empresa  
  AND a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY 1, 2  
HAVING COUNT(a.id_auditoria) <= 1;
```

c) (observação: o original tinha erros como "id empresa" sem underline)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
INNER JOIN auditorias a  
  ON e.id_empresa = a.id_empresa  
  AND a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

d)

```
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
    ON e.id_empresa = a.id_empresa  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

```
e)  
SELECT e.id_empresa, e.nome_empresa  
FROM empresas e  
LEFT JOIN auditorias a  
    ON e.id_empresa = a.id_empresa  
WHERE a.data_auditoria BETWEEN '2025-01-01' AND '2025-12-31'  
GROUP BY e.id_empresa, e.nome_empresa  
HAVING COUNT(*) <= 1;
```

14.(CEBRASPE / Polícia Federal - 2025) Julgue o itens subsecutivo, a respeito de bancos de dados.

Em SQL, as junções externas (LEFT OUTER JOIN, RIGHT OUTER JOIN e FULL OUTER JOIN) permitem incluir registros de uma ou de ambas as tabelas, mesmo quando não há correspondência entre as chaves de junção.

15.(CEBRASPE / Polícia Federal - 2025) Julgue o item subsequente, relativos à teoria da informação e a banco de dados.

No contexto de banco de dados, dados estruturados são aqueles que não possuem um formato fixo e são armazenados em sistemas como bancos NoSQL, ao passo que dados não estruturados possuem um formato rígido e são armazenados em tabelas relacionais.

16.(CEBRASPE / SEFAZ-RJ - 2025) A principal função da cláusula WITH do SQL é

- a) agrupar registros com base em valores comuns em uma ou mais colunas.
- b) combinar registros de duas ou mais tabelas com base em colunas relacionadas.
- c) filtrar registros com base em condições específicas.
- d) ordenar o resultado de uma consulta por uma ou mais colunas.
- e) definir blocos de consulta nomeados que podem ser referenciados múltiplas vezes.

17.(CEBRASPE / FUB - 2025) Julgue o seguinte item, que versa sobre administração de banco de dados.

Considere que os comandos SQL a seguir tenham sido executados de forma bem-sucedida.

```
CREATE TABLE TABELA_TESTE (a integer, b varchar);  
INSERT INTO TABELA_TESTE (a) VALUES (5);  
INSERT INTO TABELA_TESTE (b) VALUES ('FUB');  
INSERT INTO TABELA_TESTE (a) VALUES (7);
```

Nessa situação, o resultado da execução da consulta `SELECT COUNT(a), AVG(a) FROM TABELA_TESTE;` será o apresentado a seguir.

count	avg
3	4

18.(CEBRASPE / FUB - 2025) Julgue o seguinte item, que versa sobre administração de banco de dados.

A cláusula `HAVING` opera em um único conjunto de dados de entrada, sem agregação.

19.(CEBRASPE / BANRISUL - 2025) Assinale a opção em que é corretamente apresentado o comando DDL que permite criar a tabela de nome `itens_pedido` e relacionar cada pedido a seus produtos, de modo que: `pedido_id` e `produto_id` sejam chaves estrangeiras que referenciem a tabela; `quantidade` seja um inteiro maior que zero; `preco_unitario` seja um decimal maior que zero; e a combinação (`pedido_id`, `produto_id`) seja a chave primária, garantindo unicidade e não null.

a)

```
CREATE TABLE itens_pedido (  
    pedido_id    INTEGER REFERENCES pedidos(id),  
    produto_id   INTEGER REFERENCES produtos(id),  
    quantidade   INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    UNIQUE (pedido_id, produto_id)  
);
```

b)

```
CREATE TABLE itens_pedido (  
    pedido_id    INTEGER REFERENCES pedidos(id),  
    produto_id   INTEGER REFERENCES produtos(id),  
    quantidade   INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    PRIMARY KEY (pedido_id, produto_id)  
);
```

c)

```
CREATE TABLE itens_pedido (  
    pedido_id    INTEGER,  
    produto_id   INTEGER,  
    quantidade   INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id),  
    FOREIGN KEY (produto_id) REFERENCES produtos(id),  
    PRIMARY KEY (pedido_id, produto_id)  
);
```

d)

```
CREATE TABLE itens_pedido (  
    pedido_id    INTEGER REFERENCES pedidos(id),  
    produto_id   INTEGER REFERENCES produtos(id),  
    quantidade   INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    PRIMARY KEY (pedido_id, produto_id)  
);
```

e)

```
CREATE TABLE itens_pedido (  
    item_id      INTEGER PRIMARY KEY,  
    pedido_id    INTEGER REFERENCES pedidos(id),  
    produto_id   INTEGER REFERENCES produtos(id),  
    quantidade   INTEGER CHECK (quantidade > 0),  
    preco_unitario DECIMAL(10,2) CHECK (preco_unitario > 0),  
    UNIQUE (pedido_id, produto_id)  
);
```

20.(CEBRASPE / BANRISUL - 2025) Assinale a opção em que são corretamente apresentados os comandos necessários para a criação, em um banco de dados relacional, da tabela de nome funcionários, contendo as colunas: id de inteiros e chave primária; nome com VARCHAR(100), obrigatório; salario com DECIMAL(10,2); e data_admissao de DATE.

a)

```
CREATE funcionarios (  
    id          INTEGER PRIMARY KEY,  
    nome        VARCHAR(100) NOT NULL,  
    salario     DECIMAL(10,2),  
    data_admissao DATE  
);
```

b)

```
INSERT TABLE funcionarios (  
    id          INTEGER PRIMARY KEY,  
    nome        VARCHAR(100) NOT NULL,  
    salario     DECIMAL(10,2),  
    data_admissao DATE  
);
```

c)

```
CREATE TABLE funcionarios (  
    id          INTEGER PRIMARY KEY,  
    nome        VARCHAR(100) NOT NULL,  
    salario     DECIMAL(10,2),  
    data_admissao DATE  
);
```

d)

```
DEFINE TABLE funcionarios (  
    id          INTEGER PRIMARY KEY,  
    nome        VARCHAR(100) NOT NULL,  
    salario     DECIMAL(10,2),  
    data_admissao DATE  
);
```

e)

```
CREATE TABLE funcionarios (  
    id          INTEGER KEY,  
    nome        VARCHAR(100) REQUIRED,  
    salario     DECIMAL(10,2),  
    data_admissao DATE  
);
```

21.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

A criação de uma view que exiba o total de sinistros por cliente é considerada operação de DML, pois envolve a estrutura lógica do banco.

22.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

O comando `SELECT nome, idade FROM CLIENTE`; pertence à DML, pois consulta os dados armazenados no banco para eventual tomada de decisão.

23.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

A DDL permite a criação de restrições de integridade referencial entre as tabelas, como, por exemplo, a definição de uma chave estrangeira (foreign key) na tabela SINISTRO referenciando o código da APOLICE.

24.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

Para recuperar os nomes de todos os beneficiários vinculados a apólices ativas, usa-se a DQL com cláusulas como `INNER JOIN` e `WHERE`, classificando-se tal operação como consulta de dados.

25.(CEBRASPE / SUSEP - 2025) A respeito da situação hipotética apresentada e da linguagem SQL a ela relacionada, julgue o item a seguir.

O comando `UPDATE` pode ser utilizado para alterar a definição de colunas em uma tabela, como o tipo de dado de uma coluna `data_nascimento`, por exemplo, substituindo `VARCHAR` por `DATE`.

26.(CEBRASPE / FUNPRESPEX - 2025) Acerca de banco de dados, julgue o próximo item.

Considerando que `CIDADANIA` seja uma tabela com as colunas `NOME_PESSOA`, `DATA_NASCIMENTO` e `NOME_PAIS`, a execução do comando SQL `SELECT NOME_PAIS, COUNT(*) FROM CIDADANIA GROUP BY NOME_PAIS` apresentará quantas pessoas nasceram em cada PAIS.

27.(CEBRASPE / FUNPRESPEX - 2025) Acerca de banco de dados, julgue o próximo item.

O termo CRUD refere-se aos comandos de `INSERT`, `UPDATE`, `SELECT` e `DELETE`, respectivamente, em um banco de dados.

28.(CEBRASPE / FUNPRESPEX - 2025) Acerca de banco de dados, julgue o próximo item.

Em um banco de dados, os comandos de `SELECT`, `UPDATE`, `INSERT` e `DELETE` estão agrupados como DML (data manipulation language).

29.(CEBRASPE / FUNPRESPEX - 2025) Acerca de banco de dados, julgue o próximo item.

DCL (data control language) é responsável pelas tarefas administrativas de controle do próprio banco de dados; em SQL, comandos de GRANT, REVOKE e DENY são exemplos de comandos utilizados para esse fim.

30.(CEBRASPE / FUNPRESP-EXE - 2025) Acerca de banco de dados, julgue o próximo item.

A execução do comando `SELECT * FROM PESSOA LIMIT 5, 10` apresentará 10 registros da tabela PESSOA, a partir do 6.º registro dessa tabela.

GABARITO

1. LETRA A
2. LETRA A
3. LETRA A
4. ERRADO
5. CORRETO
6. ERRADO
7. CORRETO
8. ERRADO
9. ERRADO
10. ERRADO
11. ERRADO
12. LETRA C
13. LETRA B
14. CORRETO
15. ERRADO
16. LETRA E
17. ERRADO
18. ERRADO
19. LETRA D
20. LETRA C
21. ERRADO
22. ERRADO
23. CORRETO
24. CORRETO
25. ERRADO
26. CORRETO
27. ERRADO
28. ERRADO
29. CORRETO
30. CORRETO

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.